

ROS 2 二维机器人 运动规划实践

从仿真到定位、规划与控制

本书随代码逐章成长。每个例子都有明确的运行入口，
每个公式都能在对应函数和实验中找到用途。

阅读方法 先运行、看见现象，再读公式和关键代码，最后完成一个小练习。

版本范围 当前实际实现范围以仓库进度表为准；本书只收录已实现内容。

主机 ROS 2 Lyrical / RViz2

独立算法 C++17, ROS 节点 C++20

2026 年 9 月

目录

第一章 环境搭建与第一个圆盘场景	1
1.1 先认识四个名字	1
1.2 确认主机环境	1
1.3 构建并启动	2
1.4 圆盘为什么用直径?	3
1.5 发布一次, 为什么晚启动也能看到?	3
1.6 动手练习与验收	4
第二章 二维坐标、时间与接口	5
2.1 一个点可以有不同坐标	5
2.2 从公式到一个小函数	6
2.3 逆变换与变换链	6
2.4 运行与理解数值误差	6
2.5 练习: 自己写一次逆变换	7
2.6 仿真时间与真实等待时间	7
2.7 把坐标放到 ROS 的 TF 树中	8
2.8 暂停、单步与重置实验	8
第三章 一个可复现的二维世界	10
3.1 先看世界, 再谈算法	10
3.2 圆心不碰撞, 不代表圆盘不碰撞	11
3.3 点到线段的距离	11
3.4 随机多边形为什么从凸包开始?	12
3.5 先检查可达, 再讲最短路径	12
3.6 参数、练习与验收	13
第四章 让圆盘运动: 位置、速度与力	14
4.1 圆心状态与圆盘外形	14
4.2 先让机器人到一个点	14
4.3 扫过的空间也要安全	15
4.4 理想到点不产生真实 IMU	16

4.5	冻结试验与重新开始	16
4.6	给速度：先分清它在哪个坐标系	17
4.7	限速、超时与圆弧安全	18
4.8	给力：质量必须真正起作用	19
4.9	从恒加速度到带阻尼的离散模型	20
4.10	惯性、超时和碰撞如何一起工作	21
4.11	实验：质量、步长和制动距离	22
4.12	连续理想参考：给后续传感器一条平滑轨迹	23
4.13	运动中的初始条件与导数核对	24
4.14	接口、练习与验收	25
第五章	一束光到一圈扫描：CPU 激光雷达	26
5.1	先让一束光碰到障碍	26
5.2	射线与圆：别忘了第二个根	27
5.3	射线与线段：平行不一定没交点	28
5.4	均匀角度采样：一圈不要重复起点	29
5.5	量程、无效读数与采样时间	30
5.6	在 RViz2 同时观察光束和命中点	31
5.7	高斯测距噪声：先有回波，再加误差	32
5.8	噪声验收：看统计量，也看真实消息	33
5.9	束数实验：看不见不等于没有障碍	34
5.10	性能实验：先说清楚计时范围	35
第六章	机器人感觉到什么：IMU 与传感器时间	36
6.1	静止的加速度计为什么不为零	36
6.2	用三行公式读懂理想 IMU	37
6.3	先手算，再运行五个场景	38
6.4	让每个采样点带上一白噪声	39
6.5	原始消息不提供航向答案	40
6.6	用统计量验收噪声	41
6.7	零均值误差也会积成漂移	42
6.8	在 RViz 中同时观察激光与 IMU	43
6.9	采样时间不是回调到达时间	44
6.10	先接受运动区间，再产生观测	45
6.11	不对齐频率也要逐帧可验证	46

第七章 位姿已知时，如何从观测建图	47
7.1 先给建图一个已知的位姿	47
7.2 同一条里程计消息里的两种坐标系	48
7.3 运行真值适配器并核对 TF	49
7.4 从距离到二维点云	50
7.5 时间对齐比点云画出来更重要	51
7.6 格子有面积，未知也是一种状态	52
7.7 DDA：沿射线依次走过哪些格子	53
7.8 把射线变成概率证据	54
7.9 有限回波、无回波和无效读数	55
7.10 在 RViz 中检查“看见了什么”	56
7.11 独立真值栅格：只能用来评估	57
7.12 实验一：分辨率、噪声与评价指标	58
7.13 实验二：墙到底变厚了多少米	59
7.14 复现实验与本章练习	60
第八章 让两帧扫描对齐：激光里程计	61
8.1 先对齐两组已知形状的点	61
8.2 点到点残差的两行雅可比	62
8.3 点到线：沿墙滑动和穿墙不同	62
8.4 配准失败必须有名字	63
8.5 把成功扫描留在一个小窗口中	64
8.6 用估计里程计替换真值入口	64
8.7 实验：束数、噪声与被拒绝的帧	65
第九章 让 IMU 预测，让激光纠偏	67
9.1 先单独验证滤波器	67
9.2 八个数描述平面状态	67
9.3 不确定性如何沿着状态传播	68
9.4 一次位姿校正为什么也能改变速度	69
9.5 偏置与一秒缺测实验	69
9.6 校正发生在采样时刻，不是收到消息时	71
9.7 高频预测与扫描位姿各走一个出口	72
9.8 真实扫描：转得更快、少扫半秒	73
第十章 回到起点之后：关键帧与位姿图	74
10.1 从一条链变成一张图	74
10.2 为什么必须固定第一帧	74
10.3 两端位姿各贡献一个雅可比	75

10.4 矩形实验与可独立检验的答案	76
10.5 从关键帧中寻找旧地点	77
10.6 回环更新地图坐标，不改写局部里程计	78
10.7 实际激光与 IMU 的矩形回放	79
10.8 观察重建地图，并检查失败分支	80
第十一章 给圆盘找一条路：配置空间与 A*	81
11.1 先运行一张带窄门的地图	81
11.2 圆盘变成一个中心点	81
11.3 A* 的三个数与两种邻接	82
11.4 未知不是免费的捷径	83
11.5 删掉折点之前，检查整条线段	83
11.6 半径、邻接与窄门实验	84
11.7 从观测地图接入 ROS 与 RViz	85
11.8 复现一条来自真实消息的路径	86
第十二章 离障碍多远：二维距离场	87
12.1 先看一个圆的距离场	87
12.2 先确定你计算的是哪一种距离	87
12.3 欧氏距离变换：先平方，后开方	87
12.4 一维最小值为什么可以快速求	88
12.5 连续位置的值与梯度	88
12.6 把距离与安全判定分开	89
12.7 核对距离、梯度与离散误差	89
12.8 分辨率实验与练习	90
12.9 ROS 距离通道与梯度显示	91
12.10 读一次真实距离消息	92

第一章 环境搭建与第一个圆盘场景

本章目标 在主机 ROS 2 上编译两个小包，启动一个节点，在 RViz2 看见半径为 0.20 m 的圆盘。

先修 会打开终端、编辑文本，读懂基本 C++ 变量和函数。

1.1 先认识四个名字

节点 (node) 是执行计算并收发消息的程序。话题 (topic) 是带名字和类型的数据通道，例如本章的 `/visualization/robot`。包 (package) 组织源码、依赖和安装规则。工作空间把多个包放到一起编译；它不是新的 ROS 发行版。

本章只有一个发布节点和一个可视化程序：



RViz2 显示收到的数据，不负责机器人动力学。本章圆盘是一个静态图形。

1.2 确认主机环境

本课程验证平台是 Ubuntu 26.04 和 ROS 2 Lyrical。已有主机不用重装。打开 `bash`，先运行：

```
source /opt/ros/lyrical/setup.bash
printenv ROS_DISTRO
ros2 pkg prefix rviz2
colcon version-check --help
```

前三项应分别加载环境、输出 `lyrical`、找到 `RViz2`。最后一项只是检查 `colcon` 命令，后文构建才是实际工具链验收。

干净主机应先按 [ROS 官方安装步骤](#) 配置 UTF-8、Universe 与 `ros-apt-source`。已配好软件源的 Ubuntu 26.04 可安装 `ros-lyrical-desktop`、`ros-dev-tools`、`libeigen3-dev`。不要把其他 Ubuntu 版本与 Lyrical 的二进制包混用。

Lyrical 的 ROS 接口按 C++20 编译；本课程独立几何和数学代码采用 C++17。读者不必先学习 `concepts` 等高级特性。ROS 的 Python 包使用系统 Python，构建时显式指定 `/usr/bin/python3`。

1.3 构建并启动

以下路径是当前主机仓库位置；换机器后改成自己的路径。每个新终端都要先加载主机环境，再加载课程工作空间。

```
source /opt/ros/lyrical/setup.bash
cd /home/zdp/ForCodex/ros2_motion_planning_course/ros2_ws
colcon build --symlink-install --cmake-args \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DPython3_EXECUTABLE=/usr/bin/python3
source install/setup.bash
ros2 launch motion2d_bringup ch01.launch.py
```

motion2d 装算法和节点；motion2d_bringup 装配置、launch 与 RViz2 视图。--symlink-install 方便开发时编辑配置。参数只在启动时读取，所以修改 YAML 后重启节点。

RViz2 应显示浅色背景、0.5 m 网格和原点的青色圆盘，Fixed Frame 为 map。无图形界面可加 rviz:=false；这不能替代图形验收。

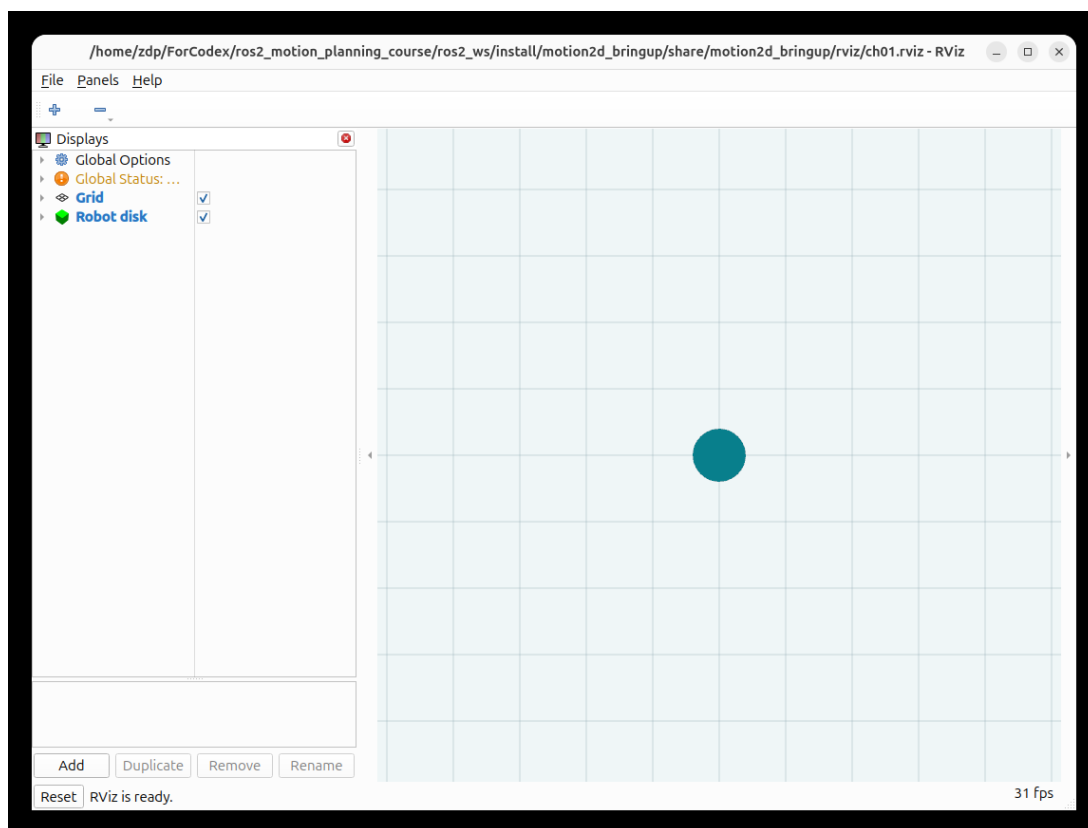


图 1.1: 主机实际运行画面。圆盘直径为 0.40 m，小于一个 0.5 m 网格。

此时 Global Status 可能提示尚无 TF 数据，因为本章还没有发布坐标树。Marker 已直接在 map 帧中，所以仍可正确显示；下一章建立 TF 后该提示消失。

1.4 圆盘为什么用直径?

用一个很薄的 CYLINDER 表示圆盘。Marker 的 scale 是物体的完整尺寸，因此

$$d_x = d_y = 2r, \quad r = 0.20 \text{ m} \Rightarrow d_x = d_y = 0.40 \text{ m}. \quad (1.1)$$

scale.z=0.05 只是显示厚度，后续平面碰撞只使用半径。单位四元数的 w 分量为 1；alpha 为 0 的图形完全透明。

下面直接读取实际源码的标记区域。修改代码后重新编译本书，摘录会同步更新：

```

33 visualization_msgs::msg::Marker disk;
34 disk.header.frame_id = "map";
35 disk.header.stamp = now();
36 disk.ns = "robot";
37 disk.id = 0;
38 disk.type = visualization_msgs::msg::Marker::CYLINDER;
39 disk.action = visualization_msgs::msg::Marker::ADD;
40 disk.pose.position.x = x;
41 disk.pose.position.y = y;
42 disk.pose.position.z = 0.025;
43 disk.pose.orientation.w = 1.0;
44 disk.scale.x = 2.0 * radius;
45 disk.scale.y = 2.0 * radius;
46 disk.scale.z = 0.05;
47 disk.color.r = 0.03F;
48 disk.color.g = 0.50F;
49 disk.color.b = 0.55F;
50 disk.color.a = 1.0F;
51 //

```

1.5 发布一次，为什么晚启动也能看到?

本节点使用 reliable + transient-local QoS，保留最后一条 Marker。RViz2 订阅同样配置持久性，晚启动时可取到保留的消息。节点退出后，不应假设消息仍会被保留。

第二个终端加载环境，进入工作空间后运行：

```

ros2 topic echo /visualization/robot --once \
--qos-durability transient_local
/usr/bin/python3 ../scripts/check_ch01.py

```

检查程序确实作为晚加入的订阅者接收消息，并核对帧、类型与默认尺寸。此时消息 stamp 用系统时间；下一章再引入仿真时钟。

1.6 动手练习与验收

1. **理解：**节点与话题的区别是什么？发布者先启动、RViz2 后启动时，为什么仍能收到圆盘？
2. **代码：**完成 `exercises/ch01/starter/marker_scale.hpp` 中的 `EXERCISE(ch01-1)`。检查方法、分级提示和独立参考答案位于同目录。
3. **实验：**在 `ch01.yaml` 中把半径改成 0.35 m、x 改为 1.0 m，重启。先预测消息和网格上的尺寸，再观察；检查默认配置前恢复参数。

应看到直径 0.70 m、圆心 (1,0) 的圆盘。若图形不见了，依次检查 Fixed Frame、话题、alpha 与 QoS。若找不到包，检查是否加载课程的 install 环境。

独立编译练习

练习的输入是半径，输出是 Marker 的 x/y 尺寸。下面的 starter 故意保留一个需要你修改的返回值；它没有被链接到正式演示中。

```
#pragma once

/** @brief Return Marker x/y size (m) for a disk of positive radius (m). */
inline double markerDiameter(double radius)
{
    // EXERCISE(ch01-1): RViz's cylinder scale describes its full width.
    return radius;
}
```

从仓库根目录运行：

```
g++ -std=c++17 -Wall -Wextra -Iexercises/ch01/starter \
    exercises/ch01/check.cpp -o /tmp/ch01_exercise
/tmp/ch01_exercise
```

未完成时检查输出 FAIL；改对后输出 PASS。检查包含两个半径，避免只为一个例子填写常数。若卡住，先看同目录的 hints，再读 solutions 的解释；把编译命令中的 starter 换成 solutions 即可验证参考解。

本章完成后，你能解释图形由哪个节点发布、消息中哪些量决定尺寸，以及如何让另一个程序接收这条消息。

第二章 二维坐标、时间与接口

目标 正确旋转、平移、求逆，并用一个仿真时钟驱动坐标树。

先修 第 1 章；二维向量和正弦、余弦。矩阵只需要理解本章的两行乘法。

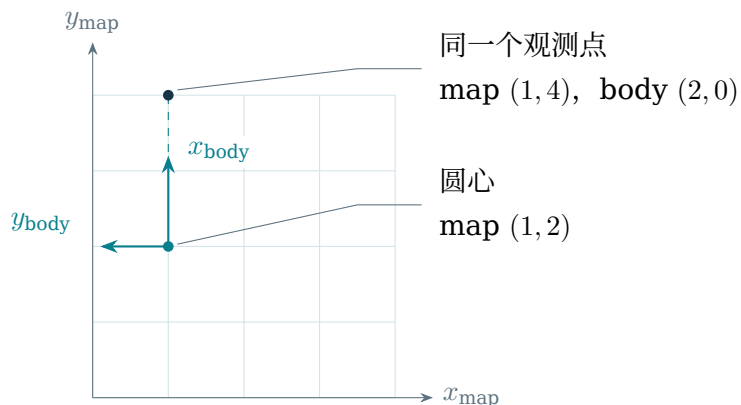
2.1 一个点可以有不同坐标

机器人位于地图坐标 (1, 2)，朝向正北。雷达观测到正前方 2 m 的点。它在机体系是 (2, 0)，在地图系是 (1, 4)。坐标变了，物理上的点没有变。

我们采用右手系：机体 x 轴向前、y 轴向左、z 轴向上，逆时针偏航为正，角度使用弧度。一个平面位姿记作 $T_{A \leftarrow B} = (t, \psi)$ ：它把 B 系的坐标换成 A 系坐标。

$$R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix}, \quad p_A = R(\psi)p_B + t. \quad (2.1)$$

先旋转向量，再加上原点位置。不能先加 t 再旋转，那样会把平移也绕 A 系原点转动。



2.2 从公式到一个小函数

Pose2D 只保存一个二维向量和一个角度。几何库不包含 ROS 头文件，可以单独用 C++17 编译、调试和测试。

```

13 Eigen::Matrix2d rotation(double yaw)
14 {
15     const double c = std::cos(yaw);
16     const double s = std::sin(yaw);
17     Eigen::Matrix2d result;
18     result << c, -s, s, c;
19     return result;
20 }
21
22 Eigen::Vector2d transformPoint(const Pose2D & pose, const Eigen::Vector2d & point)
23 {
24     return rotation(pose.yaw) * point + pose.position;
25 }
26 //

```

2.3 逆变换与变换链

从式 (2.1) 两侧减去 t ，再左乘 $R^\top$ ：

$$p_B = R^\top(p_A - t) = R^\top p_A - R^\top t. \quad (2.2)$$

因此逆变换的平移是 $-R^\top t$ 。旋转矩阵的逆为转置，这是二维旋转保持长度和夹角的结果。

若雷达相对机体有固定外参 $T_{B \leftarrow L}$ ，机体相对地图为 $T_{M \leftarrow B}$ ，则

$$T_{M \leftarrow L} = T_{M \leftarrow B} T_{B \leftarrow L}. \quad (2.3)$$

从右向左应用：先把雷达点放到机体系，再放到地图系。课程默认传感器位于圆心，外参为单位变换；保留显式外参能让后续实验改变安装位置。

2.4 运行与理解数值误差

按第 1 章的方法重新构建、加载环境，在工作空间运行：

```

ros2 run motion2d se2_demo
colcon test --packages-select motion2d \
  --event-handlers console_direct+
colcon test-result --verbose

```

示例输出机体点 (2, 0)、地图点 (1, 4)，再恢复到 (2, 0)。浮点运算中的 $\cos(\pi/2)$ 可能是接近零的小数，所以比较使用绝对误差容限 10^{-12} ，不是要求每个计算结果逐位相同。

角度是另一个边界问题。179° 到 -179° 的最短旋转只有 2°。先将差值归一到 $[-\pi, \pi)$ ，再把它作为误差。函数 `wrapAngle` 使用弧度；输入 90 表示 90 rad，不是 90 度。

2.5 练习：自己写一次逆变换

1. **推导：**为何逆变换中不能只把 t 取负？用 $\psi = 90^\circ, t = (1, 2)$ 给一个反例。
2. **代码：**补完 `exercises/ch02/starter/inverse_point.hpp` 的标记函数。它复用正式库的 `Pose2D`，检查包括两个旋转方向。
3. **实验：**把示例朝向换成 -90° ，预测地图点，再运行核对。还可以将激光外参设为机体前方 0.5 m，手算其地图位置。

独立编译命令、分级提示和参考答案位于同章习题 README。数学库的自动测试另外覆盖手算变换、传感器偏移、200 次固定种子的往返和角度边界。

2.6 仿真时间与真实等待时间

假设每个仿真步长是 $\Delta t = 0.005$ s。第 k 步的仿真时间为

$$t_k = k\Delta t. \quad (2.4)$$

即使电脑偶尔调度得慢，状态仍然只前进一个固定步长。运行速度可能变慢，但数学实验的步长没有偷偷改变。实时性能是另一项需要测量的性质。

`SimClock` 用整数 `tick` 和整数纳秒保存时间，避免反复执行浮点加法。暂停时自动 `advance` 返回 `false`，单步只在暂停时允许执行：

```

14 bool SimClock::advance()
15 {
16     if (paused_) {
17         return false;
18     }
19     ++ticks_;
20     return true;
21 }
22
23 bool SimClock::singleStep()
24 {
25     if (!paused_) {
26         return false;
27     }
28     ++ticks_;
29     return true;
30 }
31 //
```

本章由 `frame_demo_node` 唯一发布 `/clock`；RViz2 和节点都设置 `use_sim_time=true`。暂停后仍会重复发送相同时间值，便于新订阅者获得当前时间，但状态和时间戳不再前进。不要把消息数量误认为仿真步数。

2.7 把坐标放到 ROS 的 TF 树中

$\text{map} \rightarrow \text{odom} \rightarrow \text{base_link} \rightarrow \{\text{laser}, \text{imu_link}\}.$

本章 map 到 odom 为单位变换；传感器位于圆心、与机体系对齐，也是单位变换。这三条静态边通过 /tf_static 发布。odom 到 base_link 是带采样时刻的动态边，通过 /tf 发布。

为观察旋转效果，示例固定圆心 (1, 2)，采用解析朝向

$$\psi(t) = \frac{\pi}{2} + 0.2t. \quad (2.5)$$

这是一套坐标演示装置，还没有受力动力学和真实激光模型。图中的蓝色箭头只是 laser 帧中长度为 2 m 的向量。

ROS 使用四元数表示旋转。平面 yaw 对应 $(q_x, q_y, q_z, q_w) = (0, 0, \sin(\psi/2), \cos(\psi/2))$ 。发布时刻直接取仿真时钟，不能换成回调到达的真实时间：

```

126 geometry_msgs::msg::TransformStamped transform;
127 transform.header.stamp = rclcpp::Time(clock_.nanoseconds(), RCL_ROS_TIME);
128 transform.header.frame_id = "odom";
129 transform.child_frame_id = "base_link";
130 transform.transform.translation.x = x_;
131 transform.transform.translation.y = y_;
132 const double angle = wrapAngle(yaw_ + yaw_rate_ * clock_.seconds());
133 transform.transform.rotation.z = std::sin(angle / 2.0);
134 transform.transform.rotation.w = std::cos(angle / 2.0);
135 broadcaster_.sendTransform(transform);
136 //

```

2.8 暂停、单步与重置实验

停止第 1 章程序，重新构建并加载环境后启动：

```
ros2 launch motion2d_bringup ch02.launch.py
```

另一个终端同样加载环境后调用：

```

ros2 service call /sim/pause std_srvs/srv/SetBool "{data: true}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"

```

暂停时圆盘朝向和箭头冻结；每次单步前进 5 ms；reset 回到时间零、初始朝向，并保持暂停。运行中请求单步会明确失败，防止把手动步数混入自动积分。

在工作空间运行 /usr/bin/python3 ../scripts/check_ch02.py 可检查唯一 TF 父帧、解析朝向、冻结时钟、5 ms 单步、reset 和恢复运行。它会改变演示状态，仅用于本章实验。

重置会使仿真时间向后跳，TF 监听器会清空缓存，这是新试验的正常边界。本节点每隔 1 s 墙钟时间重发静态外参，暂停时同时重发当前动态边，使监听器能在 reset 后恢复。状态并未因此前进。后续 reset 还须清理估计和传感器历史。

时间练习

先暂停，再连续调用 `step` 十次，预测仿真时间增量和朝向增量，之后查看 `/clock` 和 TF 核对。标准答案是 0.05 s 与 0.01 rad。改变 YAML 中的 `dt` 后要重新推导；不应继续照搬默认值的检查脚本。

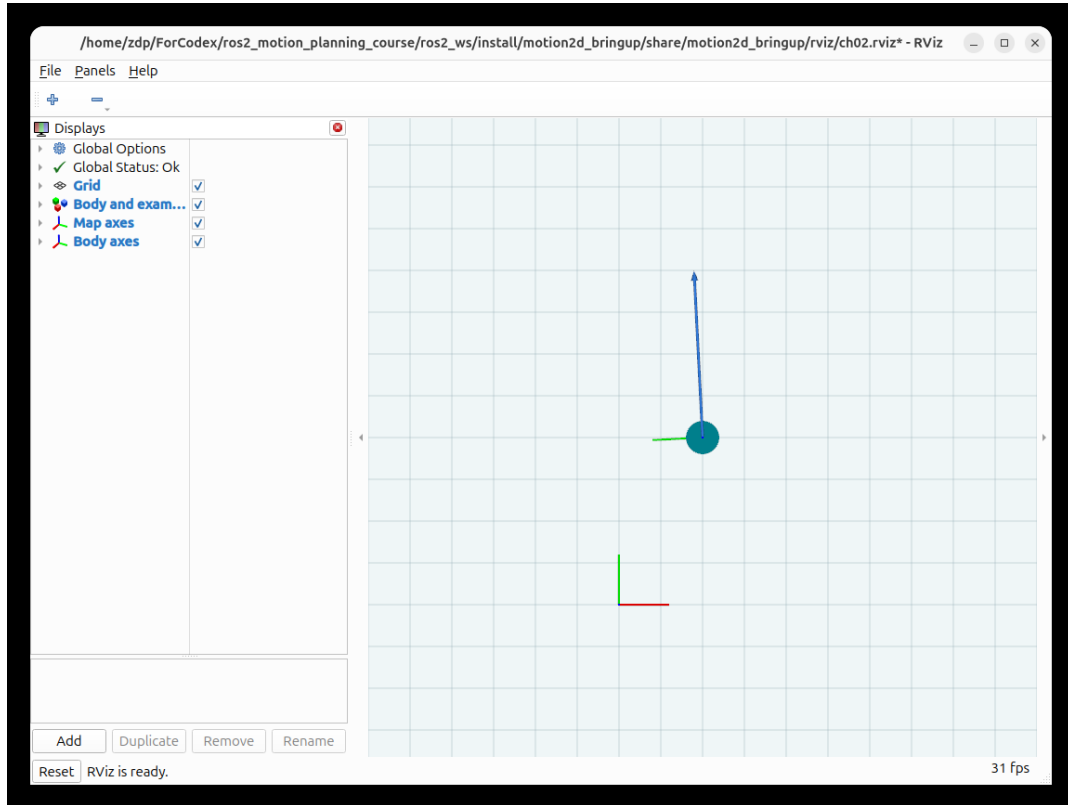


图 2.1: 重置并恢复运行后的主机 RViz2。圆盘与机体轴向量正确显示，TF 状态正常。

第三章 一个可复现的二维世界

目标 生成随机圆与凸多边形，显示起终点，检查有半径的机器人能否放入世界。

先修 前两章；向量点积。凸包与距离的必要直觉在本章说明。

3.1 先看世界，再谈算法

停止之前的演示，重新构建并加载工作空间后运行：

```
ros2 launch motion2d_bringup ch03.launch.py
```

默认世界宽、高都是 20 m，边界以原点为中心。 $\text{seed}=42$ 时生成 8 个圆和 8 个凸多边形；机器人在 $(-8, -8)$ ，目标在 $(8, 8)$ 。世界是静态的，机器人尚未运动。

此处显示的是仿真掌握的完整几何。后续激光只能看见无遮挡且在量程内的表面，建图不能直接读取这些障碍物顶点。理解这个数据边界，才能公平比较真值模式和 SLAM 模式。

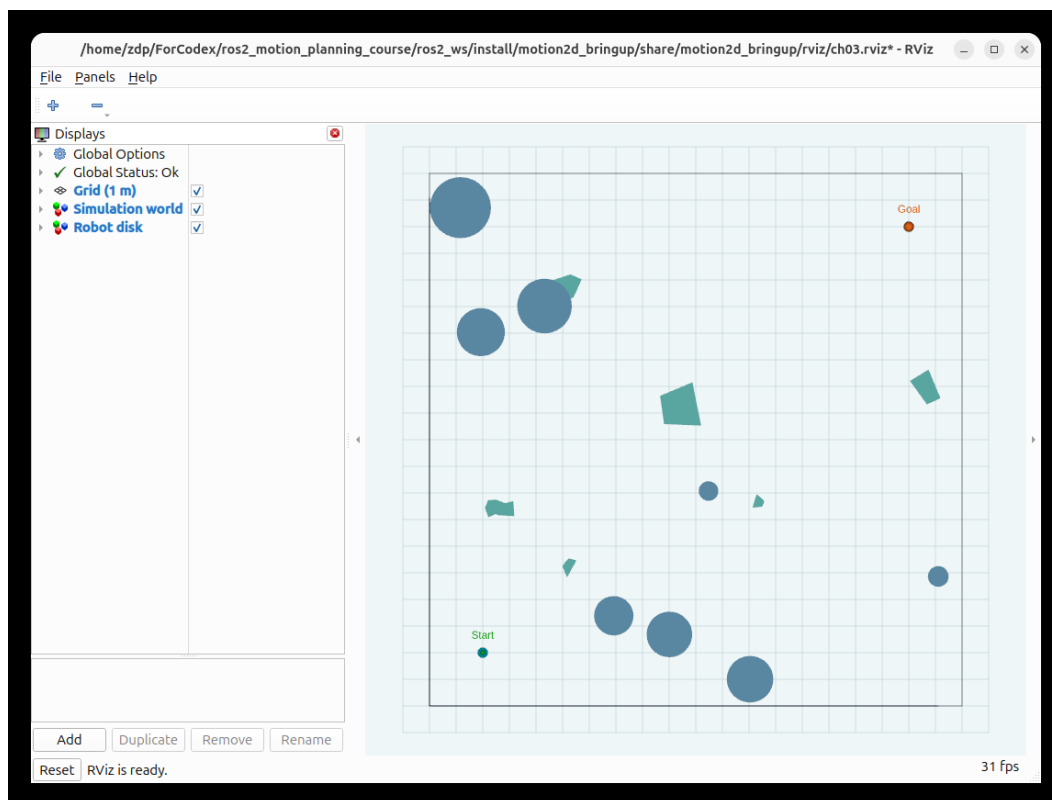


图 3.1: $\text{seed}=42$ 的实际世界。圆和凸多边形可以重叠；Start 与 Goal 分别标出起终点。

3.2 圆心不碰撞，不代表圆盘不碰撞

半径为 r 的圆盘中心位于 p ，圆形障碍中心为 c 、半径为 R 。两者相切时，中心距为 $r + R$ ，本课程把接触计为碰撞：

$$\|p - c\| > r + R \iff \text{与该障碍无碰撞.} \quad (3.1)$$

定义单个障碍的有符号距离 $d(p)$ ：外部为正、内部为负。自由区域中，圆盘需要 $d(p) > r$ 。如要额外留出安全裕量 δ ，使用 $d(p) > r + \delta$ ；物理半径与规划裕量是两个不同参数。

世界边界也限制可用空间。核心函数在自由域中取边界和所有障碍的最小距离，再比较半径：

```

11 double clearance(const World2D & world, const Eigen::Vector2d & center)
12 {
13     double distance = std::min(
14         world.width / 2.0 - std::abs(center.x()),
15         world.height / 2.0 - std::abs(center.y()));
16     for (const auto & circle : world.circles) {
17         distance = std::min(distance, signedDistance(circle, center));
18     }
19     for (const auto & polygon : world.polygons) {
20         distance = std::min(distance, signedDistance(polygon, center));
21     }
22     return distance;
23 }
24
25 bool isFree(const World2D & world, const Eigen::Vector2d & center, double radius)
26 {
27     return clearance(world, center) > radius;
28 }
29 //

```

占据区或越界时，这个函数用于判断符号，不承诺给出并集内部的精确 ESDF；第 12 章会专门构建距离场。

3.3 点到线段的距离

多边形边界由线段组成。设边为 $a \rightarrow b$ ，把点 p 投影到直线上：

$$t^* = \frac{(p - a)^\top (b - a)}{\|b - a\|^2}, \quad t = \min(1, \max(0, t^*)), \quad q = a + t(b - a). \quad (3.2)$$

q 是线段上离 p 最近的点，距离是 $\|p - q\|$ 。限制 t 很重要：离线段延长线很近，不代表离有限线段很近。若 $a=b$ ，则线段退化为一个点。

```

17 double pointSegmentDistance(
18     const Eigen::Vector2d & point, const Eigen::Vector2d & a, const Eigen::Vector2d & b)
19 {
20     const Eigen::Vector2d edge = b - a;
21     if (edge.squaredNorm() == 0.0) {
22         return (point - a).norm();
23     }
24     const double t = std::clamp((point - a).dot(edge) / edge.squaredNorm(), 0.0, 1.0);
25     return (point - (a + t * edge)).norm();
26 }
27 //

```

对逆时针凸多边形，内部点位于每条有向边的左侧。若点在内部，距离取负；外部则取到全部边的最小距离。

3.4 随机多边形为什么从凸包开始？

随意把随机点连起来可能产生自交。我们的步骤是：在一个采样圆盘内生成点，再取凸包。可把凸包想成绕所有点拉紧的橡皮筋。实现先按 x/y 排序，再维护上、下两条只向同一方向拐弯的链。

若 u, v 独立服从 $[0, 1)$ 均匀分布，使用

$$\theta = 2\pi u, \quad \rho = R\sqrt{v}, \quad p = c + \rho(\cos \theta, \sin \theta) \quad (3.3)$$

让采样点在圆盘面积上均匀。因为半径不超过 ρ 的面积比例是 ρ^2/R^2 ，所以半径要取平方根。

代码中 `polygon_samples=8` 表示采样八个点，凸包顶点通常更少。校验要求顶点有限、严格凸、逆时针；对每条边检查其余顶点都位于左侧，也能排除自交星形。

不同障碍可以重叠，这仍是合法障碍并集。世界生成器只拒绝无效形状、越界和侵入起终点保护圆的候选。每个形状最多尝试 1000 次，失败明确报告，不偷偷换种子。

3.5 先检查可达，再讲最短路径

起点、终点各自无碰撞，并不意味着中间一定连通。我们先用粗网格和四邻接 flood fill 检查连通性：从起点所在格入队，反复访问相邻自由格，能到终点格即通过。它不寻找最短路径，也不替代第 11 章的 A^* 。

不能仅检查格中心。对于宽 Δx 、高 Δy 的单元，中心到最远角点的距离为

$$\epsilon_{\text{cell}} = \frac{1}{2} \sqrt{\Delta x^2 + \Delta y^2}. \quad (3.4)$$

中心只有在净空大于 $r + \delta + \epsilon_{\text{cell}}$ 时才标为自由。这保证整个单元能容纳圆盘，代价是窄通道可能被保守拒绝。

默认检查分辨率 0.25 m。若失败，可以明确减小分辨率或关闭 `require_connected` 观察场景；同一个 seed 不会自动被替换成“更好看”的世界。记录失败案例也是学习的一部分。

3.6 参数、练习与验收

编辑 `ros2_ws/src/motion2d_bringup/config/ch03.yaml` 后重启。公共 `x/y/radius` 同时提供给坐标节点和世界节点；圆盘显示与碰撞不应使用两套不同半径。

1. **理解：** 给一个圆心在自由空间、圆盘却碰撞的例子。解释相切和安全裕量。
2. **代码：** 实现 `exercises/ch03/starter/segment_distance.hpp` 的点线段距离。独立检查覆盖内点、端点外、线段上和退化线段；提示与参考解单独存放。
3. **实验：** 相同配置和 `seed=42` 启动两次，比较地图。然后改为 43，再把两类障碍各增至 20 个，记录可达性和起终点净空。

代码练习从仓库根目录单独编译，不依赖 ROS：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iexercises/ch03/starter \
exercises/ch03/check.cpp -o /tmp/ch03_exercise
/tmp/ch03_exercise
```

将 `starter` 替换为 `solutions` 可以检验参考解。先运行未完成的练习观察失败位置，再修改投影公式；不要通过删掉检查来得到“通过”。

在同一标准库和参数下，固定种子复现相同世界；跨编译器标准库版本不保证分布函数的逐位一致。输出状态说明是否执行了连通性检查，避免把“没有检查”当成“已经通过”。

运行 `colcon test` 检查几何、碰撞和可达性；演示运行时，在工作空间执行以下命令核对默认消息：

```
/usr/bin/python3 ../scripts/check_ch03.py
```

RViz2 展示全部障碍、边界和起终点，图形并非激光建图结果。

常见问题

看不见障碍时，先确认 Fixed Frame 为 `map`、World 的话题为 `/visualization/world`，再检查节点有没有因无效参数退出。改参数后要重启；本章尚不提供运行中重建世界。若可达性检查失败，保留 `seed` 和配置，再研究分辨率与通道宽度。

主机观察： 本机 RViz2 曾在一次较长演示的关闭阶段崩溃，运行显示正常，后续复测未再出现，原因尚未确定。启动文件已避免重复关闭事件；若需要分开观察进程，用 `rviz:=false` 启动节点，再按本章 README 的命令独立打开 RViz2，先关闭 RViz2，再停止节点。

第四章 让圆盘运动：位置、速度与力

目标 区分位置、速度和力输入；理解质量、阻尼、碰撞与制动。

先修 前三章、速度与加速度的含义。先运行，再推导。

4.1 圆心状态与圆盘外形

机器人用圆心描述状态，圆盘描述占据的空间：

$$x = (p, v, \psi, \omega), \quad p = (p_x, p_y), \quad v = (v_x, v_y). \quad (4.1)$$

位置单位 m，速度 m/s，yaw 为 rad，角速度 rad/s。位置和速度在 odom 系表达；本章的 map 与 odom 重合。半径 r 单独参与碰撞，不把“质点状态”误解为“没有体积”。若 $r=0$ ，才是几何质点；屏幕上的小点仅用于看清位置。

控制量不同，模型的物理意义就不同。理想到点输入目标位姿；速度模型输入平面速度；惯性模型输入力和偏航力矩。它们分别使用不同话题，不能把同一个数值悄悄解释成另一种单位。下面从最容易运行的理想到点出发，再运行速度模型与受力惯性模型。

4.2 先让机器人到一个点

停止之前的场景，重新构建并加载 install 后运行：

```
ros2 launch motion2d_bringup ch04.launch.py model:=ideal
```

在 RViz2 用 2D Goal Pose 工具点击、拖动，指定目标位置和 yaw。工具向 /command/pose 发布消息。也可在第二个已加载 ROS 环境的终端发送：

```
ros2 topic pub --once /command/pose geometry_msgs/msg/PoseStamped \
  "{header: {frame_id: odom}, pose: {position: {x: -7.0, y: -8.0}, orientation: {w: 1.0}}}"
```

默认从 $(-8, -8)$ 移到 $(-7, -8)$ ，图中留下接受过的运动轨迹。目标在下一 tick 才执行；暂停时发目标，圆盘保持不动，单步后才到达目标。

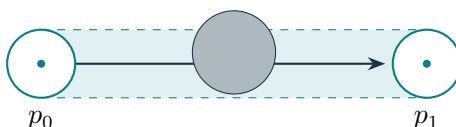
4.3 扫过的空间也要安全

起终点安全并不足够。半径 r 的圆盘沿线段 $[p_0, p_1]$ 移动，扫过的区域是胶囊形。对于圆心 c 、半径 R 的障碍，无碰撞要求

$$\text{dist}(c, [p_0, p_1]) > R + r. \quad (4.2)$$

这里直接复用第 03 章的点线段距离。对凸多边形，检查起终点和运动线段到每条边的距离；对矩形边界，内缩后的矩形是凸集，检查两个端点即可保证整条直线留在其中。

端点自由，整段被挡住



```

32 bool sweptDiskIsFree(const World2D & world, const Eigen::Vector2d & from,
33   const Eigen::Vector2d & to, double radius)
34 {
35   // The inset rectangle is convex, so valid endpoints imply a valid segment.
36   if (!isFree(world, from, radius) || !isFree(world, to, radius)) {return false;}
37   for (const auto & circle : world.circles) {
38     if (pointSegmentDistance(circle.center, from, to) <= radius + circle.radius) {
39       return false;
40     }
41   }
42   for (const auto & polygon : world.polygons) {
43     const auto & v = polygon.vertices;
44     for (std::size_t i = 0; i < v.size(); ++i) {
45       if (segmentDistance(from, to, v[i], v[(i + 1) % v.size()]) <= radius) {
46         return false;
47       }
48     }
49   }
50   return true;
51 }
52 //

```

代码没有按固定间距抽样，因此不会因为障碍很薄、目标很远，就从障碍另一侧“穿过去”。这是连续直线扫掠的几何检查，浮点实现仍受数值精度限制。

4.4 理想到点不产生真实 IMU

通过几何检查后，理想模型直接赋值目标位姿：

```
6 State2D idealPose(const Pose2D & target)
7 {
8     State2D result;
9     result.pose = target;
10    result.pose.yaw = wrapAngle(target.yaw);
11    return result; // Zero derivatives mean "unused", not an IMU measurement.
12 }
13 //
```

理想到点没有有限的运动速度。这里的零导数表示未使用，不能用位置跳变的差分模拟 IMU。需要连续运动时，使用本章后面的解析圆参考。

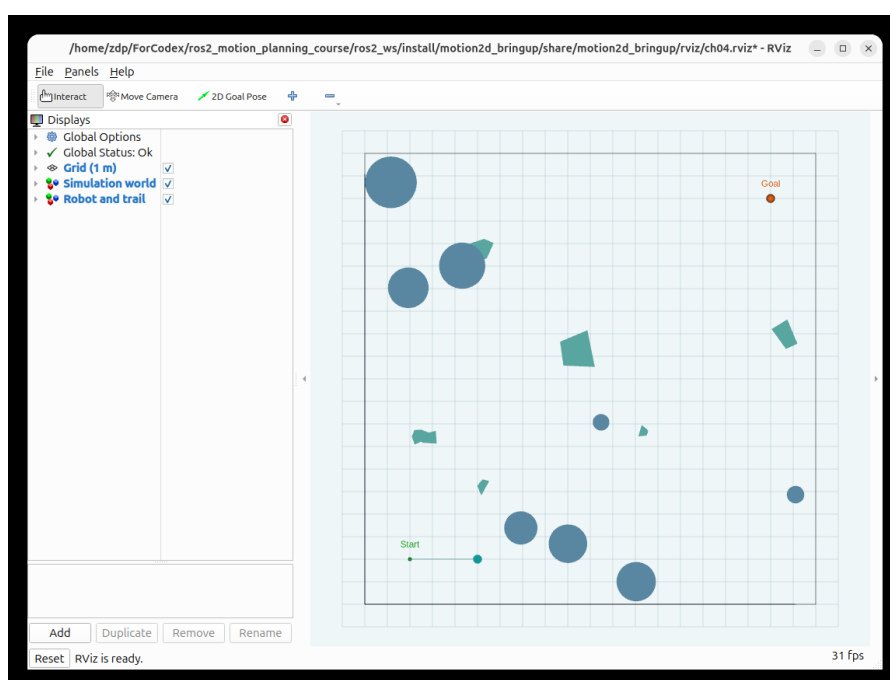


图 4.1: 实际 RViz2 画面：世界、圆盘与已接受的移动轨迹。

4.5 冻结试验与重新开始

扫描检查失败时，程序不接受候选状态，位置、速度和时间均保持在上一个安全的 tick。此时圆盘变红，`/sim/status` 显示 `collision_predicted`。这是提前终止试验，尚未模拟接触力、反弹或摩擦。它没有把撞入障碍的状态推回自由空间。需要调用 `reset` 才能继续。

```
ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"
```

`reset` 开始新试验：回初始状态、清空待执行命令与轨迹、仿真时间归零并暂停。单步推进一个 `dt`；恢复运行不改变 `dt`。墙钟定时器决定运行快慢，整数 `tick` 决定模拟时间。

4.6 给速度：先分清它在哪个坐标系

重启场景，选用速度模型：

```
ros2 launch motion2d_bringup ch04.launch.py model:=velocity
```

此时只订阅 `/command/velocity`，RViz2 的位姿目标不参与驱动。在另一个已加载环境的终端连续发送机体系速度：

```
ros2 topic pub -r 20 /command/velocity geometry_msgs/msg/TwistStamped \
  "{header: {frame_id: base_link}, twist: {linear: {x: 0.5}, angular: {z: 0.3}}}"
```

`odom` 系速度方向固定；`base_link` 系速度会随机器人转动。同一个 $(0.5, 0)$ ，前者表示一直向 `odom` 的 x 正方向走，后者表示一直朝圆盘的“前方”走。全向模型也允许 y 方向输入，这和不能横移的差速车不同。

若一个时间步 h 内保持 `odom` 速度 u 与角速度 ω ，则

$$p_{k+1} = p_k + hu, \quad \psi_{k+1} = \text{wrap}(\psi_k + h\omega). \quad (4.3)$$

若 u 在机体系，积分 $\dot{p} = R(\psi_k + \omega t)u$ 可得

$$p_{k+1} = p_k + h \text{sinc}(\beta) R(\psi_k + \beta)u, \quad \beta = \frac{\omega h}{2}. \quad (4.4)$$

其中 $\text{sinc}(\beta) = \sin \beta / \beta$ ，在零处取 1。小角度用 $1 - \beta^2/6$ 避免除以很小的数。这个式子覆盖圆弧与直线极限：

```
16 State2D stepVelocity(const State2D & state, const VelocityCommand & command, double dt)
17 {
18     State2D next = state;
19     const double half_turn = 0.5 * command.yaw_rate * dt;
20     next.pose.yaw = wrapAngle(state.pose.yaw + 2.0 * half_turn);
21     next.yaw_rate = command.yaw_rate;
22     next.yaw_acceleration = 0.0; // Only within the held-command interval.
23     next.acceleration.setZero();
24     if (command.body_frame) {
25         const double sinc = std::abs(half_turn) < 1e-6 ?
26             1.0 - half_turn * half_turn / 6.0 : std::sin(half_turn) / half_turn;
27         next.pose.position += dt * sinc * rotation(state.pose.yaw + half_turn) * command.velocity;
28         next.velocity = rotation(next.pose.yaw) * command.velocity;
29         next.acceleration = command.yaw_rate * Eigen::Vector2d{-next.velocity.y(), next.velocity.x()};
30     } else {
31         next.pose.position += dt * command.velocity;
32         next.velocity = command.velocity;
33     }
34     return next;
35 }
36 //
```

4.7 限速、超时与圆弧安全

默认平移速度模长限制为 1 m/s, 角速度绝对值限制为 1 rad/s。超出平移限制时等比例缩短向量, 保留方向; 不能分别限每个分量后, 仍声称限制了模长。配置项是 `speed_max` 和 `yaw_rate_max`。

命令默认在模拟时间中有效 0.5 s, 到期后在 `tick` 边界切为零速度, 状态为 `command_timeout`。暂停不会消耗有效期。零时间戳从接收时刻起算, 非零时间戳从消息时刻起算。改变 `dt` 时, 过期检测最多量化到一个 `tick`。由于可以瞬间改变速度, 该模型依然不用于 IMU 融合; 后续惯性模型需要真实制动。

圆弧的中间部分不一定贴着起终点连线。设加速度模长在一个步长内不超过 $a_{\max}$, 曲线与线性插值的偏差满足

$$\|p(t) - p_{\text{chord}}(t)\| \leq \frac{a_{\max}}{2}t(h-t) \leq \frac{a_{\max}h^2}{8}. \quad (4.5)$$

可把它理解为线性插值的二阶误差界。机体系恒速度转弯时 $a_{\max} = |\omega| \|u\|$, `odom` 恒速度时为零。因此把检查半径暂时改为 $r + a_{\max}h^2/8$, 再复用直线扫掠检查, 就把整段曲线包在胶囊内。这是保守的充分检查, 可能拒绝狭窄但实际上可通过的路径。

为什么不能只检查两个状态? 薄墙可能位于两个 `tick` 之间; 转弯时障碍也可能位于圆弧上、却不在端点连线上。测试专门构造了后一种场景, 验证“只检查弦”会漏检, 加上曲率引起的余量后能拒绝。

代码练习 ch04-2: 补完 `exercises/ch04/starter/velocity_step.hpp` 的 `odom` 系速度积分, 并归一化 `yaw`。用 `check_velocity.cpp` 检查负速度、时间步长和跨越 `pi` 的角度。编译时沿用本章习题 README 的命令。

实验: 在转向后分别发送 `odom` 与 `base_link` 的同一速度, 比较轨迹; 停止发送, 验证超时; 暂停两秒再单步, 解释为什么暂停不消耗模拟时间。运行下面的实际通信检查:

```
/usr/bin/python3 ../scripts/check_ch04.py --model velocity
```

4.8 给力：质量必须真正起作用

速度命令能立即停下，真实惯性不能。全向平面模型把平移和偏航分开：

$$\dot{p} = v, \quad m\dot{v} = F - c_v v, \quad (4.6)$$

$$\dot{\psi} = \omega, \quad I_z \dot{\omega} = \tau - c_\omega \omega. \quad (4.7)$$

F 为 odom 系平面力，单位 N； τ 为偏航力矩，单位 N·m。 m 是质量， I_z 是转动惯量。 c_v, c_ω 分别为平移和转动阻尼，不能把 kg/s 的阻尼系数与没有单位的“衰减比例”混用。

推荐继续使用这个全向模型：控制器给力，可以直接观察质量、限幅和制动，再学习线性 MPC。扫地机器人的差速驱动不能横移，适合在附录加上非完整约束。本模型也不等价于四旋翼完整的姿态与推力动力学。

重启场景后，在第二个终端持续施加力与力矩：

```
ros2 launch motion2d_bringup ch04.launch.py model:=inertial
# second terminal
ros2 topic pub -r 20 /command/wrench geometry_msgs/msg/WrenchStamped \
"{header: {frame_id: odom}, wrench: {force: {x: 1.0}, torque: {z: 0.02}}}"
```

本模式只订阅 /command/wrench，只接受 odom 系的平面分量。力始终指向 odom 的 x 正方向，圆盘转向也不改变这个力的方向。停发消息后，默认 0.5 s 超时，撤去的是力，不是速度。

配置项	默认值	单位与意义
mass	1.0	kg，必须为正
inertia_z	-1.0	自动取 $mr^2/2$ ；可显式给正惯量
linear_drag	0.0	kg/s，非负
angular_drag	0.0	kg·m ² /s，非负
force_max	2.0	N，每个平面轴分别限幅
torque_max	0.2	N·m，偏航力矩绝对值上限

参数在启动时读取，改 YAML 后重启。若 $r=0$ ，必须显式提供正的转动惯量；几何质点不意味着可以除以零。速度模式的两个速度上限不用于惯性模式，因为直接剪掉速度会改变物理方程。

4.9 从恒加速度到带阻尼的离散模型

先设阻尼为零。一个步长 h 内恒力意味着恒加速度 $a = F/m$:

$$p_{k+1} = p_k + hv_k + \frac{1}{2}h^2a, \quad v_{k+1} = v_k + ha. \quad (4.8)$$

位置更新使用旧速度。若先更新速度再用新速度加上同一个二次项，就会多走一段。偏航用同样的关系，把 a 换成 τ/I_z 。

有线性阻尼时，每个分量都是 $\dot{v} = u - \lambda v$ ，其中 $u = F/m, \lambda = c_v/m$ 。步内输入保持不变称为**零阶保持**。积分这个一阶方程可以得到

$$A = e^{-\lambda h}, \quad B = \frac{1 - e^{-\lambda h}}{\lambda}, \quad C = \frac{h - B}{\lambda}, \quad (4.9)$$

$$v_{k+1} = Av_k + Bu, \quad p_{k+1} = p_k + Bv_k + Cu. \quad (4.10)$$

在 $\lambda = 0$ 时取连续极限： $A = 1, B = h, C = h^2/2$ 。实现对很小的 λh 用泰勒展开，避免两个相近浮点数相减损失精度。偏航使用 $\lambda = c_\omega/I_z, u = \tau/I_z$ ，复用相同的积分系数。

```

54 State2D stepInertial(const State2D & state, const Wrench2D & command,
55     const InertialParameters & p, double dt)
56 {
57     const auto applied = limitWrench(command, p);
58     const auto xy = gains(p.linear_drag / p.mass, dt);
59     const auto yaw = gains(p.angular_drag / p.inertia_z, dt);
60     const Eigen::Vector2d u = applied.force / p.mass;
61     const double alpha = applied.torque / p.inertia_z;
62     State2D next;
63     next.pose.position = state.pose.position + xy.velocity * state.velocity + xy.position * u;
64     next.velocity = xy.decay * state.velocity + xy.velocity * u;
65     next.acceleration = (applied.force - p.linear_drag * next.velocity) / p.mass;
66     next.pose.yaw = wrapAngle(state.pose.yaw + yaw.velocity * state.yaw_rate + yaw.position * alpha);
67     next.yaw_rate = yaw.decay * state.yaw_rate + yaw.velocity * alpha;
68     next.yaw_acceleration = (applied.torque - p.angular_drag * next.yaw_rate) / p.inertia_z;
69     return next;
70 }
71 //

```

先限幅，再积分。步末加速度由施加的力和步末速度重新计算，后续 IMU 会使用这个运动真值。它不是对有噪声的位置做差分。命令在 `tick` 切换，位置和速度连续，加速度可以跳变。

4.10 惯性、超时和碰撞如何一起工作

无阻尼时，零力意味着速度不变；有阻尼、零力时，

$$v(t) = v(0)e^{-c_v t/m}. \quad (4.11)$$

所以命令超时不能让惯性模型“瞬间刹住”。超时只切换为零输入，同时报告 `command_timeout`。后续控制器要用反向力制动，并考虑剩余空间；如果一直保持反向力，越过零速度后会向反方向加速。

碰撞策略仍然是提前终止试验。对于步内恒定力和非负线性阻尼，

$$a(t) = a(0)e^{-c_v t/m}, \quad a(0) = \frac{F - c_v v_k}{m}. \quad (4.12)$$

因此本步加速度模长不超过 $\|a(0)\|$ 。继续使用前一节的插值误差界，把 $\|a(0)\|h^2/8$ 加到检查半径上，可以保守覆盖整个曲线。碰撞或数值异常时都不发布失败候选，而是冻结上一个有效状态。这不是接触力学，也不是任意浮点误差下的形式化无碰撞证明。

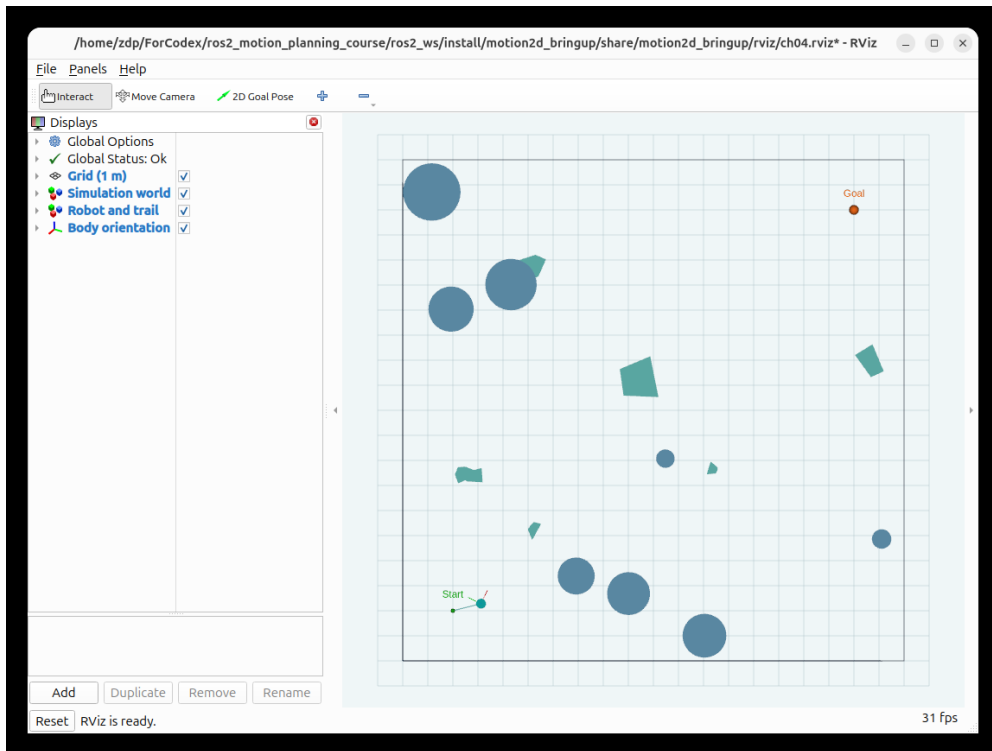


图 4.2: 受力后的实际圆盘、运动轨迹与机体坐标系。平移与偏航可以分别驱动。

默认配置启动后，执行真实通信实验：

```
/usr/bin/python3 ../scripts/check_ch04.py --model inertial
```

脚本检查力与力矩限幅、自动惯量、暂停不推进、超时滑行、反向力制动和重置。质量改为 2 kg 时，可追加 `--mass 2`；其余仍使用默认零阻尼配置。

4.11 实验：质量、步长和制动距离

从静止出发，1 N 力持续 1 s，1 kg 的机器人移动 0.50 m、达到 1 m/s；2 kg 的机器人移动 0.25 m、达到 0.5 m/s。再从相同初速 $v_0 = 1$ m/s 出发，用大小 $F_b = 1$ N 的反向力制动，在无阻力时

$$t_{\text{stop}} = \frac{mv_0}{F_b}, \quad d_{\text{stop}} = \frac{mv_0^2}{2F_b}. \quad (4.13)$$

因此质量翻倍，同速制动距离也翻倍。比较制动时必须明确初速是否相同。

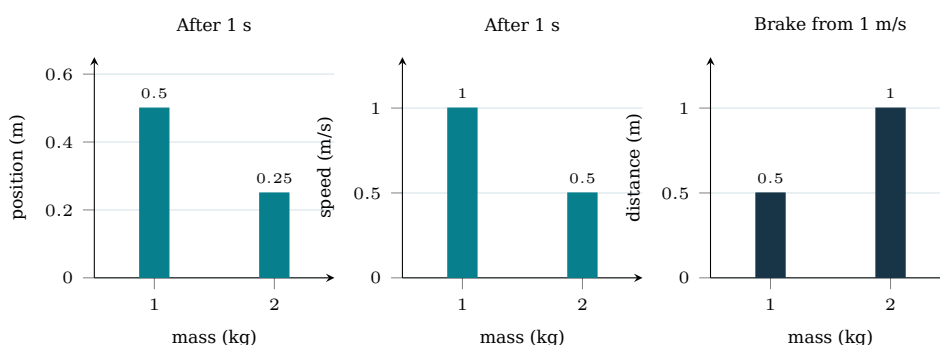


图 4.3: 直接读取 C++ 实验 CSV 的结果，dt=0.005 s、阻尼为零，力大小为 1 N。

核心实验不依赖 ROS 回调时延，直接运行：

```
ros2 run motion2d inertia_demo
```

m / kg	dt / s	1 s 位移 / m	1 s 末速 / (m/s)	同速制动距离 / m
1	0.020	0.50	1.00	0.50
1	0.005	0.50	1.00	0.50
2	0.020	0.25	0.50	1.00
2	0.005	0.25	0.50	1.00

这里改变 dt 仍得到相同解析结果，因为力恒定、切换时刻恰好位于步边界，并使用零阶保持的解析离散。这不代表 dt 永远无关：输入刷新、事件检测、未来控制闭环都会受影响；曲线碰撞余量也随 dt 的平方变化。

代码练习 ch04-3: 补完 exercises/ch04/starter/force_step.hpp。先做无阻尼恒力一步更新，输入已经限幅；平移和偏航都要正确。独立的 check_force.cpp 覆盖非零初速度、不同质量与转动惯量，运行命令见习题 README。

拓展实验: 把 linear_drag 改为 0.5 kg/s，从有速度的状态撤去力，画出速度随时间的衰减，与 $v_0 e^{-c_v t/m}$ 比较。不要用瞬时速度模式代替这个试验。

4.12 连续理想参考：给后续传感器一条平滑轨迹

还有一种常用实验：先给定一条连续轨迹，让机器人完美执行，单独研究传感器和定位。这里从解析圆轨迹开始，不引入尚未需要的轨迹消息和优化器：

```
ros2 launch motion2d_bringup ch04.launch.py model:=reference
```

`reference` 自动运行，不订阅外部位置、速度或力命令。它是理想参考执行，不验证力限幅可行性；任何碰撞仍然要检查。轨迹半径 `reference_radius` 默认 1 m，与机器人碰撞半径 `radius` 是两个参数；角频率 `reference_omega` 默认 0.4 rad/s。

令初始位姿为 (p_0, ψ_0) , $R_0 = R(\psi_0)$ 、 $\theta = \Omega t$, 选择

$$p(t) = p_0 + R_0 \begin{bmatrix} r_c \sin \theta \\ r_c(1 - \cos \theta) \end{bmatrix}, \quad (4.14)$$

$$v(t) = R_0 r_c \Omega \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}, \quad a(t) = R_0 r_c \Omega^2 \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}, \quad (4.15)$$

$$\psi(t) = \text{wrap}(\psi_0 + \theta), \quad \omega(t) = \Omega. \quad (4.16)$$

因此速度模长是 $r_c|\Omega|$ ，加速度模长是 $r_c\Omega^2$ 。三个平移量来自同一个解析参考，不需要通过差分位置猜测加速度：

```
7 State2D sampleCircle(const Pose2D & initial, double radius, double omega, double time)
8 {
9     const double theta = omega * time;
10    const auto r0 = rotation(initial.yaw);
11    State2D reference;
12    reference.pose.position = initial.position +
13        r0 * Eigen::Vector2d{radius * std::sin(theta), radius * (1.0 - std::cos(theta))};
14    reference.velocity = r0 * Eigen::Vector2d{
15        radius * omega * std::cos(theta), radius * omega * std::sin(theta)};
16    reference.acceleration = r0 * Eigen::Vector2d{
17        -radius * omega * omega * std::sin(theta), radius * omega * omega * std::cos(theta)};
18    reference.pose.yaw = wrapAngle(initial.yaw + theta);
19    reference.yaw_rate = omega;
20    return reference;
21 }
22 //
```

4.13 运动中的初始条件与导数核对

这个圆参考在 $t=0$ 就有非零速度。默认起点仍为 $(-8, -8)$ ，初速度是 0.4 m/s ，加速度模长为 0.16 m/s^2 。reset 恢复的是这个运动中的初始条件，开始新试验；它没有模拟“从静止瞬间加速到 0.4 m/s ”。后续 IMU 积分实验也要明确初始速度、偏航等先验，不能把这些已知条件隐藏起来。

碰撞余量取 $r_c \Omega^2 h^2 / 8$ 。默认场景可以走完一圈；换大轨迹半径或换 seed 后，若参考穿过障碍，仍会冻结试验。理想执行并不授予穿越障碍的能力。

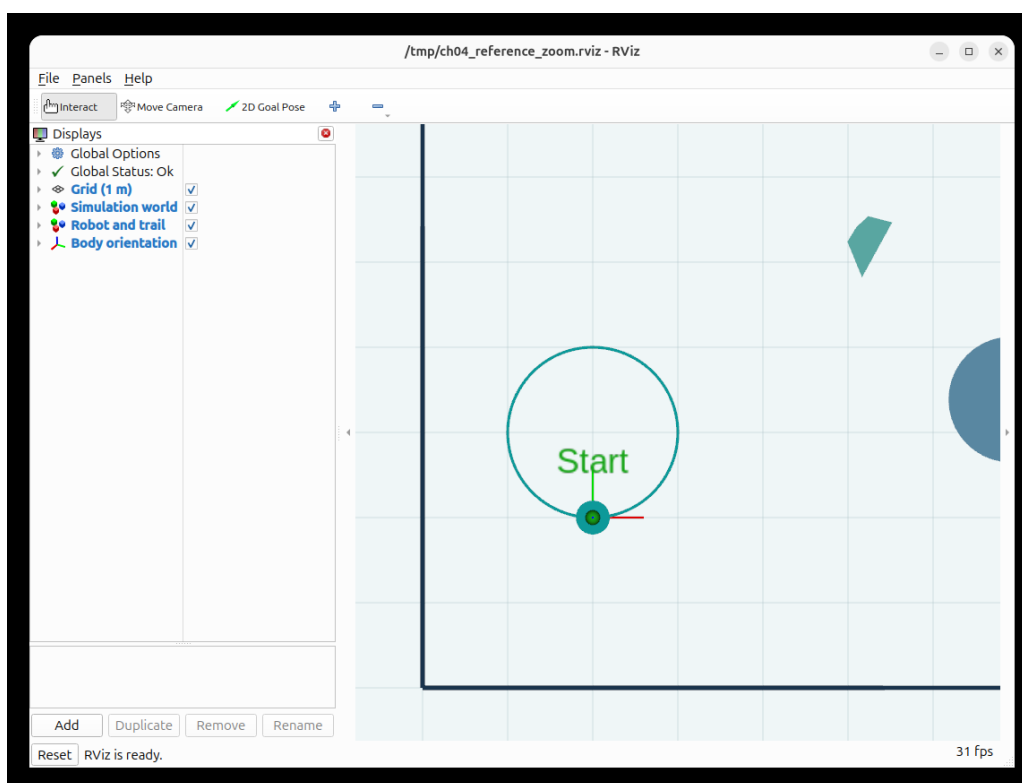


图 4.4: 默认场景中完整圆参考的局部放大。圆盘与朝向随参考运动，轨迹不由规划器计算。

如何验证导数？ 除了检查四分之一圈的手算值，测试还使用中心差分

$$v(t) \approx \frac{p(t + \epsilon) - p(t - \epsilon)}{2\epsilon}, \quad a(t) \approx \frac{v(t + \epsilon) - v(t - \epsilon)}{2\epsilon}. \quad (4.17)$$

用它核对解析函数，而不拿差分结果驱动 IMU。测试还比较整圈前后的 $p/v/a$ 。实际离散 tick 未必恰好落在周期终点，应与相同采样时刻的解析值比较。

代码练习 ch04-4: 补完 `exercises/ch04/starter/reference_acceleration.hpp`，对速度求时间导数，注意链式法则中的第二个 Ω 。再把角频率翻倍，验证速度翻倍、加速度变为四倍。命令与验收入口：

```
/usr/bin/python3 ../scripts/check_ch04.py --model reference
```

4.14 接口、练习与验收

本章默认 $dt=0.005$ s。位姿输入接受 `odom/map`；`map` 可用是因为本章二者重合。姿态必须是平面单位四元数，不能把四个分量全填零。时间戳为零表示下一 `tick`；非零时间戳只接受当前模拟时刻附近的命令，过旧或过远的未来命令被拒绝。

`/sim/pose`、`/sim/velocity`、`/sim/acceleration` 都是真值调试输出，平移分量在 `odom` 系。第 07 章才引入规范的里程计消息与选源，后续 SLAM 不得读取这些调试话题。本章没有激光或 IMU。

1. **理解：** 给一个起终点都自由但中间碰撞的例子。为什么瞬时位置赋值不适合 IMU？
2. **代码 ch04-1：** 完成 `exercises/ch04/starter/sweep_circle.hpp`。复用点线段距离，判定圆盘与圆障碍的整段扫掠；相切必须判为碰撞。
3. **实验：** 暂停后发目标并单步；改变半径重复同一条贴近障碍的移动。记录 `seed`、半径、起终点和结果。不要把显示尺寸当作碰撞半径。

从仓库根目录编译练习：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iros2_ws/src/motion2d/include -Iexercises/ch04/starter \
exercises/ch04/check_sweep.cpp \
ros2_ws/src/motion2d/src/geometry/obstacles.cpp \
-o /tmp/ch04_sweep
/tmp/ch04_sweep
```

提示和参考解分开存放；将 `starter` 替换为 `solutions` 可以验证参考解。演示运行时，从工作空间执行：

```
/usr/bin/python3 ../scripts/check_ch04.py
```

脚本会改变场景状态，检查暂停命令、单步、错误帧、碰撞冻结与 `reset`。算法测试另外覆盖薄墙和零半径质点。

没有移动时先检查 是否暂停、是否需要 `reset`、模型是否与输入话题对应，再检查 `frame` 和四元数。只运行一个课程 `launch`，避免多个时钟或 TF 发布者。

第五章 一束光到一圈扫描：CPU 激光雷达

目标 用可手算的几何求交构建激光雷达，理解遮挡、角分辨率与测距误差。

先修 二维向量、圆方程，以及前四章的坐标、世界与机器人模型。

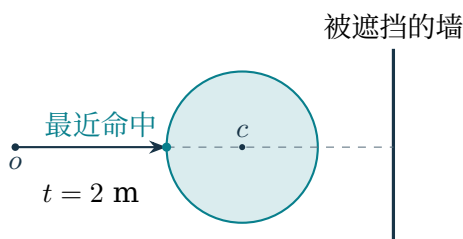
5.1 先让一束光碰到障碍

雷达安装在圆盘中心，发出一条没有宽度的射线。机器人位置与 `yaw` 决定射线的起点和方向。雷达不检测机器人自身，否则从圆心发出的光总会先碰到自己的外壳。机器人半径仍参与运动碰撞。

射线用起点 o 、单位方向 d 和非负距离 t 表示：

$$p(t) = o + td, \quad \|d\| = 1, \quad t \geq 0. \quad (5.1)$$

只有方向已归一化时， t 才直接表示米。测距求的是最近表面，不是障碍中心的距离。沿同一方向有多个交点时，较近表面会挡住较远表面。



重新构建工作空间并加载 `install`，先运行不依赖 ROS 回调的几何演示：

```
ros2 run motion2d raycast_demo
```

三个距离应依次为 2、4、2 m。前两个分别是圆和墙的单独求交；最后一个在世界中取最近值。代码位于 `src/examples/raycast_demo.cpp`，这里的墙和圆都可以手算。

5.2 射线与圆：别忘了第二个根

把射线代入圆心 c 、半径 R 的圆方程，令 $q = c - o$ 、 $h = q^\top d$ ：

$$\|o + td - c\|^2 = R^2 \implies t^2 - 2ht + \|q\|^2 - R^2 = 0. \quad (5.2)$$

二维叉积 $a \times b = a_x b_y - a_y b_x$ 是一个标量。单位方向下， $|d \times q|$ 是圆心到射线所在直线的距离，因此

$$D = R^2 - (d \times q)^2, \quad t_{\pm} = h \pm \sqrt{D}. \quad (5.3)$$

$D < 0$ 时没有交点； $D = 0$ 时相切，也算命中。先检查较小的根 t_- 是否非负，否则再检查 t_+ 。两个根都负，说明圆在射线背后。圆内起点的小根为负，大根是出口；这有助于测试函数，但课程中的机器人应始终位于自由空间。

```

37 double rayCircle(const Eigen::Vector2d & origin, const Eigen::Vector2d & direction,
38   const Circle & circle)
39 {
40   const Eigen::Vector2d offset = circle.center - origin;
41   const double h = offset.dot(direction);
42   const double perpendicular = cross(direction, offset);
43   const double discriminant = circle.radius * circle.radius - perpendicular * perpendicular;
44   if (discriminant < 0.0) {return kMiss;}
45   const double root = std::sqrt(discriminant);
46   if (h - root >= 0.0) {return h - root;}
47   return h + root >= 0.0 ? h + root : kMiss;
48 }
49 //

```

没有命中返回正无穷，它与后续所有距离取最小值时自然不起作用。此处先计算真实几何，量程与测量噪声留给扫描层处理；不要在这个函数里混入 ROS 消息。

代码练习 ch05-1：在 `exercises/ch05/starter/forward_root.hpp` 补完根的选择。从仓库根目录运行：

```

g++ -std=c++17 -Iexercises/ch05/starter \
  exercises/ch05/check_root.cpp -o /tmp/ch05_root
/tmp/ch05_root

```

先手算圆心 (3,0)、半径 1、起点 (0,0)、方向 (1,0) 的两个根；再把起点放到圆心。这两个例子分别检查最近回波和第二个根。

5.3 射线与线段：平行不一定没交点

令线段端点为 a 、 b ， $e = b - a$ 。联立 $o + td = a + ue$ ，分别取叉积可得

$$t = \frac{(a - o) \times e}{d \times e}, \quad u = \frac{(a - o) \times d}{d \times e}. \quad (5.4)$$

只有 $t \geq 0$ 且 $0 \leq u \leq 1$ 才命中线段，而不只是命中它所在的直线。当分母接近零时不能继续做除法：平行但不共线则无交点；共线时将端点投影到射线上，检查投影区间是否与 $[0, +\infty)$ 相交，取重叠部分的最近点。

```

19 double raySegment(const Eigen::Vector2d & origin, const Eigen::Vector2d & direction,
20   const Eigen::Vector2d & a, const Eigen::Vector2d & b)
21 {
22   const Eigen::Vector2d edge = b - a, offset = a - origin;
23   const double denominator = cross(direction, edge);
24   const double tolerance = 1e-12 * std::max(1.0, edge.norm());
25   if (std::abs(denominator) <= tolerance) {
26     if (std::abs(cross(offset, direction)) > tolerance) {return kMiss;}
27     const double t0 = offset.dot(direction), t1 = (b - origin).dot(direction);
28     return std::max(t0, t1) < 0.0 ? kMiss : std::max(0.0, std::min(t0, t1));
29   }
30   const double t = cross(offset, edge) / denominator;
31   const double u = cross(offset, direction) / denominator;
32   return t >= 0.0 && u >= 0.0 && u <= 1.0 ? t : kMiss;
33 }
34 //

```

世界求交 `raycastWorld` 遍历所有圆、所有多边形边，以及矩形边界的四条边，对得到的非负距离取最小值。这样自然处理遮挡，不依赖障碍物存储顺序。算法复杂度随表面数线性增长；先保留这个清楚的 CPU 基准，后续再讨论加速。

验收与实验：运行 `colcon test --packages-select motion2d`。几何测试覆盖圆相切、背向、线段端点、平行与共线、矩形角点、遮挡和刚体变换不变性。把演示的圆移到墙后，预测最近命中；反转射线方向，解释距离为何改变。

这里的浮点容差面向课程的米制小场景。它用于避免接近平行时除以极小数，不是任意坐标尺度下的精确几何谓词。无效几何应在世界生成边界拒绝。

5.4 均匀角度采样：一圈不要重复起点

在雷达坐标系中，第 i 束的角度为 $\alpha_i = \alpha_0 + i\Delta\alpha$ 。初始方向 α_0 与 yaw 是不同的量：前者决定扫描扇区，后者描述机器人朝向。世界方向与圆心起点是

$$d_i = \begin{bmatrix} \cos(\psi + \alpha_i) \\ \sin(\psi + \alpha_i) \end{bmatrix}, \quad o = p_{\text{robot}}. \quad (5.5)$$

完整 360 度使用 $\Delta\alpha = 2\pi/N$ ， i 从 0 到 $N-1$ 。若同时采 $-\pi$ 与 π ，两束实际上指向同一方向。部分视场 F 则包含两个端点，使用 $\Delta\alpha = F/(N-1)$ ，并要求 $N \geq 2$ 。不要用“束数减一”的同一个公式处理这两种情况。

```

29 double beamIncrement(const LidarConfig & config)
30 {
31     const bool full_turn = std::abs(config.fov - kTwoPi) <= 1e-12;
32     return full_turn ? kTwoPi / config.beams : config.fov / (config.beams - 1);
33 }
34
35 std::vector<float> scanCpu(const World2D & world, const Pose2D & pose,
36     const LidarConfig & config)
37 {
38     std::vector<float> ranges(config.beams);
39     const double increment = beamIncrement(config);
40     for (int i = 0; i < config.beams; ++i) {
41         const double angle = pose.yaw + config.angle_min + i * increment;
42         const Eigen::Vector2d direction{std::cos(angle), std::sin(angle)};
43         const double distance = raycastWorld(world, pose.position, direction);
44         if (distance < config.range_min) {
45             ranges[i] = std::numeric_limits<float>::quiet_NaN();
46         } else if (distance > config.range_max) {
47             ranges[i] = std::numeric_limits<float>::infinity();
48         } else {
49             ranges[i] = static_cast<float>(distance);
50         }
51     }
52     return ranges;
53 }
54 //

```

核心输出只是一组 `float` 距离，便于与 ROS 分离测试。机器人状态与几何使用 `double`；`Laser-Scan` 本身使用 `float32`，因此通信检查允许相应的舍入误差。当前扫描逐束遍历全部表面，复杂度为 $O(N(C + E))$ ， C 是圆数， E 是所有边的数量（含世界四边）。

5.5 量程、无效读数与采样时间

先求最近真实表面，再判断量程。近处有盲区障碍时，不能略过它后返回更远的墙。默认最小量程 0.05 m、最大量程 10 m，两个端点都包含。

情形	ranges 中的值	后续建图含义
量程内命中	有限距离	光线清空、终点占据
量程内没有回波	$+\infty$	只清空量程内光线
真实命中近于盲区	NaN	整束跳过

NaN 的大小比较均为假。有效读数应检查有限性和上下界，不能只写“若小于最小值或大于最大值，则无效”的反向判断。量程边界处也不能把无回波伪装成恰好命中最大量程，二者的建图终点含义不同。

与设备规范的区别 REP-117 用 $-\infty$ 表示过近。课程沿用已批准的约定，把近距盲区统一归入 NaN 无效读数；适配实际硬件时需保留设备自己的语义，不把这两种编码说成相同。

snapshot 是一张快照。 所有束使用同一时刻的机器人位姿，因此 `time_increment=0`。默认 10 Hz 的帧间周期为 0.1 s，所以 `scan_time=0.1`。header 记录采样时刻，`frame` 为 `laser`；没有反射强度模型时 `intensities` 为空。这些字段依据 [LaserScan 定义](#) 及主机安装的同名消息核对。

模拟器默认每 0.005 s 推进一步，雷达每 20 步采样一次。`t=0` 有首帧，暂停期间不重复发布新扫描；碰撞冻结也不继续采样。`reset` 清理采样时序，在 `t=0` 生成新试验首帧。本章先学习周期为 `dt` 整数倍的情形，辅助函数 `lidarPeriodTicks` 会拒绝 7 Hz。当前模拟器已接入第六章的采样调度，因此运行时也支持 7 Hz，原理见第六章。

代码练习 ch05-2：补全 `exercises/ch05/starter/beam_angle.hpp`，用 `check_angles.cpp` 检查 90/360/720 束与部分视场。理解题：如果盲区被写成 $+\infty$ ，后续建图会把哪里错误标为空闲？

5.6 在 RViz2 同时观察光束和命中点

停止上一章场景，构建并加载 install 后运行：

```
ros2 launch motion2d_bringup ch05.launch.py
# second terminal, from ros2_ws with the same ROS environment
/usr/bin/python3 ../scripts/check_ch05.py
```

默认沿第 04 章的解析圆参考运动。也可选择 `model:=ideal`，用 2D Goal Pose 手动移动，或选择 `velocity/inertial` 并发送相应控制量。修改 `motion2d_bringup/config/ch05.yaml` 后重启；早期章节默认不启用雷达。

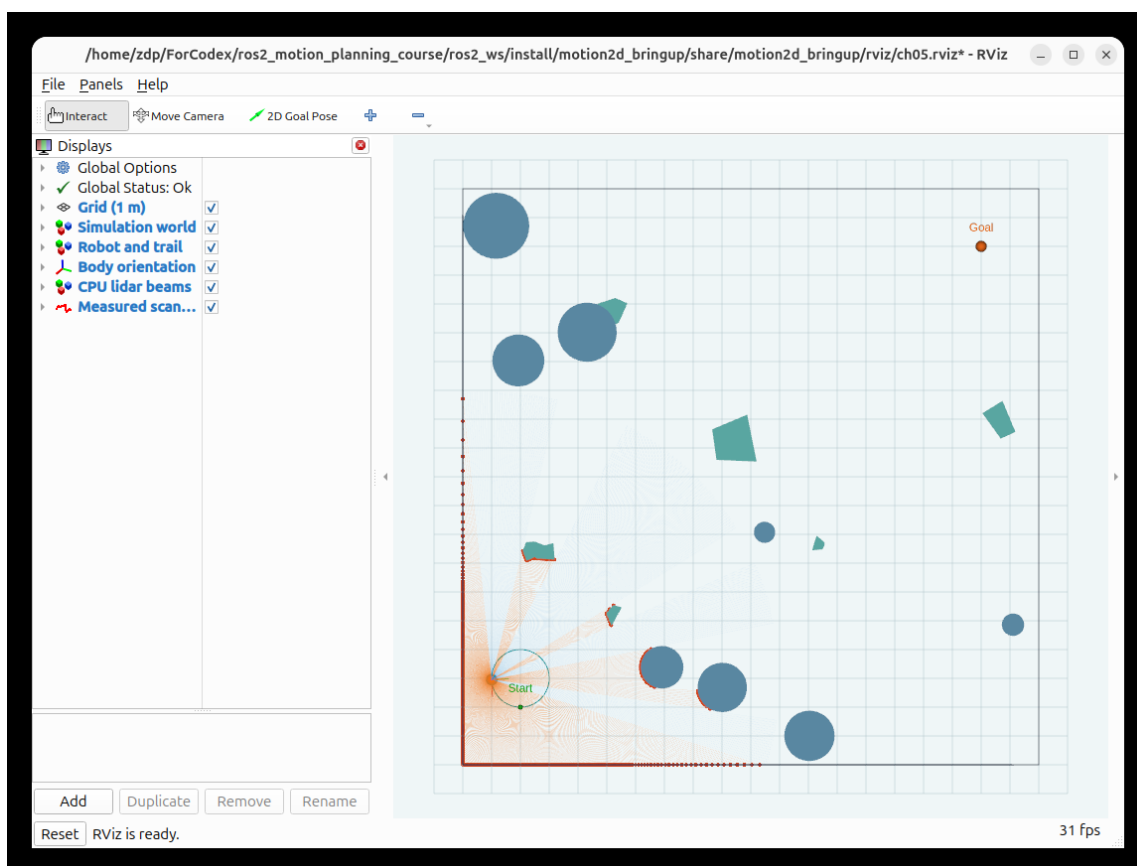


图 5.1: 关闭测距噪声的主机基准画面。橙色线是有效测距射线，红点是 LaserScan 命中点；淡蓝线只表示量程内无回波，障碍后的空间仍不可见。

/scan 使用 SensorDataQoS (Best Effort、Volatile)，RViz 的 LaserScan 显示也要匹配。显示辅助光束发布在 /visualization/lidar，按采样位姿固定在 odom 中，不会在两帧之间跟随当前机器人漂移。辅助显示含真值对齐信息，SLAM 不得读取。

验收实验：检查脚本会暂停、重置、逐步推进并核对 TF、角度、QoS 与光束数量。默认 19 步内没有第二帧，第 20 步的采样时间为 0.1 s；尝试改为 20 Hz 再手动核对间隔。暂停时晚加入的 LaserScan 订阅者不会收到历史帧，恢复运行后等待下一帧即可。

5.7 高斯测距噪声：先有回波，再加误差

对真实量程内的有效回波 r_i ，加入独立的零均值高斯误差：

$$\tilde{r}_i = r_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_r^2). \quad (5.6)$$

σ_r 是每束、每次采样的标准差，单位 **m**，不是方差，也不是噪声密度。默认 `lidar.range_stddev=0.01 m`；设为零即可回到精确几何基准。课程暂不模拟反射率、入射角、距离相关噪声或多径。

无回波的 $+\infty$ 和盲区 `NaN` 保持原值，不凭空生成障碍。有效距离加噪后若越出量程，记为 `NaN`；既不夹到最大量程形成假墙，也不改为 $+\infty$ 暗示自由空间。

```

71 float perturbRange(float range, double error, const LidarConfig & config)
72 {
73     if (!std::isfinite(range)) {return range;}
74     const double measured = static_cast<double>(range) + error;
75     if (measured < config.range_min || measured > config.range_max) {
76         return std::numeric_limits<float>::quiet_NaN();
77     }
78     return static_cast<float>(measured);
79 }
80
81 void addRangeNoise(std::vector<float> & ranges, const LidarConfig & config, std::mt19937 & random)
82 {
83     if (config.range_stddev == 0.0) {return;}
84     std::normal_distribution<double> gaussian(0.0, config.range_stddev);
85     for (auto & range : ranges) {
86         if (std::isfinite(range)) {range = perturbRange(range, gaussian(random), config);}
87     }
88 }
89 //

```

几何函数与加噪函数分开，CPU 和未来 CUDA 可先比较无噪声距离，再用同一组误差比较输出。`perturbRange` 接受给定误差，便于这种成对实验。雷达使用独立随机流，`lidar.noise_seed=4242` 不会影响世界种子。`reset` 重置这个随机流；同一配置、轨迹、采样顺序和标准库下可重放。改变束数会改变雷达的随机数消费顺序，但不会重新生成障碍物。

代码练习 ch05-3：补完 `exercises/ch05/starter/noisy_range.hpp`，正确处理给定误差与特殊值。检查入口为 `check_noise.cpp`，编译命令见本章习题 `README`。

5.8 噪声验收：看统计量，也看真实消息

一两个噪声样本不能说明分布正确。先让所有真实距离固定在 5 m，远离 0.05 m 和 10 m 的量程边界，取 50000 个样本：

```
ros2 run motion2d lidar_noise_demo
```

误差 $e_j = \tilde{r}_j - r_j$ 的样本均值与总体式样本标准差为

$$\bar{e} = \frac{1}{M} \sum_{j=1}^M e_j, \quad \hat{\sigma} = \sqrt{\frac{1}{M} \sum_{j=1}^M (e_j - \bar{e})^2}. \quad (5.7)$$

下面直接读取同一 C++ 演示生成的 CSV (seed=4242)：

样本数	设定 σ/m	实测均值/m	实测标准差/m
50,000	0.010	0.00004150	0.00995074

均值应接近零，误差容差应随样本数变化。测试要求 $|\bar{e}| < 5\sigma/\sqrt{M}$ ，方差与 σ^2 的相对偏差不超过 3%。这是可复现的统计检查，不是要求有限样本均值恰好为零。靠近量程边界时，越界样本会被丢弃，保留下来的误差分布可能偏斜，不能照搬这组检验。

还要确认 ROS 发布链确实使用了噪声。使用无障碍的 20 m 方形房间，把机器人静止放在中心；每束真实墙距为

$$r(\alpha) = \frac{10}{\max(|\cos \alpha|, |\sin \alpha|)}. \quad (5.8)$$

停止上一场景，从 ros2_ws 运行下列实验：

```
ros2 launch motion2d_bringup ch05.launch.py model:=ideal rviz:=false \
  config:=src/motion2d_bringup/config/ch05_noise_lab.yaml
# second terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch05_noise.py
```

实验使用 360 束、50 Hz、最大量程 20 m。脚本重置后检查 t=0 到 3.98 s 的 200 帧、72000 个实际残差；缺帧会明确失败。这里远处墙面也在量程内，不会因无回波而混入另一种分布。

理解与实验：把标准差翻倍，预测方差变成几倍；把采样频率翻倍，解释为什么单个样本的设定标准差不变。修改后应同步调整验收脚本中的预期参数，不要把默认配置的断言直接用于所有实验。

5.9 束数实验：看不见不等于没有障碍

相邻光束在相同距离 r 处的端点形成一个等腰三角形，端点间距为

$$s = 2r \sin \frac{\Delta\alpha}{2} \approx r\Delta\alpha. \quad (5.9)$$

距离越远，相同角分辨率对应的间距越大。以 5 m 处为例，90、360、720 束的整周扫描分别对应约 34.90、8.73、4.36 cm 的端点间距。增加束数能让边界更密，但有限束数仍可能漏掉细小物体。

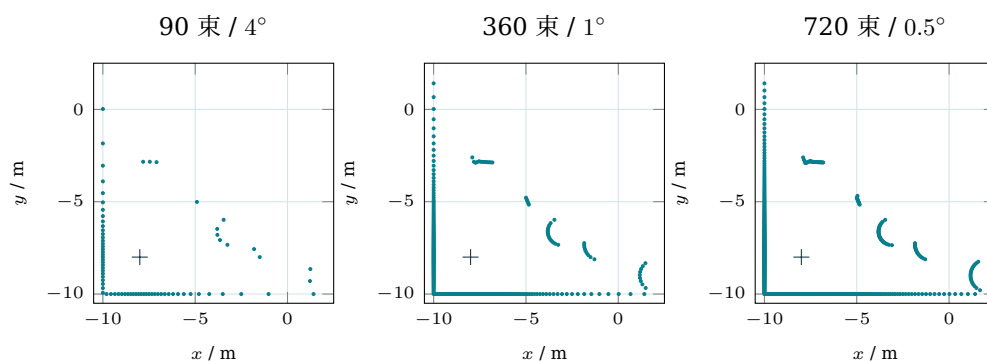


图 5.2: 同一 $\text{seed}=42$ 世界、同一圆心 $(-8, -8)$ 、 $\text{yaw}=0$ 、关闭噪声的有限回波。十字为雷达；图中空白不等于已经观测到的自由空间。

再单独构造一个直径 0.10 m 的小圆：圆心距离雷达 5 m，方向为 2° 。雷达位于原点，扫描从 $-\pi$ 开始。90 束的方向间隔是 4° ，恰好从小圆两侧穿过；360 束有一束命中，720 束有三束命中。这个确定性例子已经写成测试，不依赖肉眼判断。

束数	角间隔/度	小圆命中束数
90	4	0
360	1	1
720	0.5	3

代码练习 ch05-4: 补完 `exercises/ch05/starter/beam_gap.hpp`，把角分辨率换算为米制端点间距。然后改变小圆的方向、尺寸或距离，比较是否命中。表中结果只对应这个特定场景，不是“720 束必定能看见所有小障碍”的保证。

实际 ROS 对照: 复制 `ch05.yaml`，分别将 `lidar.beams` 改为 90、360、720，保持其余参数和世界 `seed` 相同。启动各配置后运行 `check_ch05.py --beams 90`（其余束数类推），核对消息与光束数量。

5.10 性能实验：先说清楚计时范围

从 `ros2_ws` 运行 CPU 基准：

```
ros2 run motion2d lidar_benchmark > ../textbook/data/ch05_benchmark.csv
ros2 run motion2d lidar_benchmark --points > ../textbook/data/ch05_hits.csv
# repository root
make -C textbook
```

程序使用 `seed=42` 的默认 8 圆、8 多边形世界和固定起点。每种束数先预热 100 帧，再测 1000 次 `scanCpu` 调用，用单调时钟统计中位数和 P95。P95 表示按耗时排序后，约 95% 的调用不超过这个数；它不是最坏情况上界。

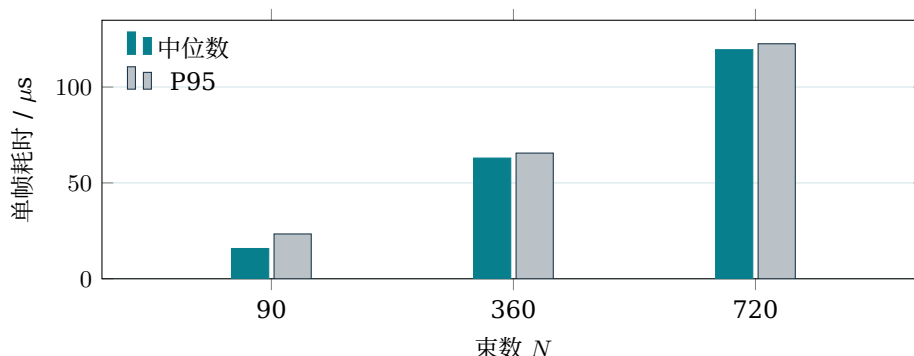


图 5.3: 本机 AMD Ryzen 7 9700X、GCC 15.2、RelWithDebInfo 下的一次测量。仅 CPU 几何扫描，噪声关闭；柱图直接读取实验 CSV。

束数	中位数/ μ s	P95/ μ s	有限回波数
90	15.79	23.38	75
360	62.95	65.55	301
720	119.48	122.57	600

计时包含方向计算、全部表面求交、量程编码和结果数组分配。不包含世界生成、加噪、ROS 消息转换、DDS 传输、光束 Marker 或 RViz 渲染，所以不能把表中数字称为端到端传感器时延或整机实时性保证。实验未绑定 CPU 核，调频与系统负载会使复测数字变化。

为 CUDA 留下基准：未来使用同一几何、姿态、束数和输出编码，先关噪声比较距离，再分别测计算内核、主机/设备传输和完整消息发布。不能把 GPU 内核时间与 CPU 完整流程直接比较，也不能在未测量前承诺加速比。本章没有建图或规划求解器，求解失败率不适用。

第六章 机器人感觉到什么：IMU 与传感器时间

目标 从运动真值产生原始 IMU，区分比力、白噪声和采样时间。

先修 第二章的坐标变换、第四章的连续运动，以及第五章的传感器消息。

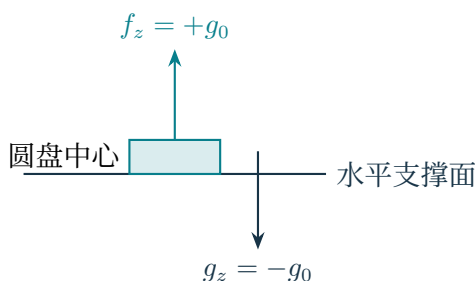
6.1 静止的加速度计为什么不为零

把一个加速度计平放在桌上，它的 z 轴向上时会测到约 $+9.81 \text{ m/s}^2$ 。这不是机器人正在加速上升，而是桌面的支持力产生了比力。自由落体才接近零比力；本课程机器人受平面约束，不模拟自由落体。

设 R 将机体系向量转到世界系， a 是圆心的世界运动加速度， g 是世界重力：

$$f = R^\top(a - g), \quad g = (0, 0, -g_0)^\top, \quad g_0 = 9.81 \text{ m/s}^2. \quad (6.1)$$

后续估计器若要恢复运动加速度，需要 $a = Rf + g$ ，并且必须使用自己的姿态估计。仿真器内部可用真值 yaw 生成测量，估计器不能把它当作 IMU 给出的姿态。



本章 IMU 在圆心，与 `base_link` 同向。机器人只做水平平移和 `yaw` 转动，因此 $a_z = 0$ 、`roll=pitch=0`。陀螺仪理想输出 $(0, 0, \dot{\psi})$ ，单位 rad/s ，加速度计理想输出的单位是 m/s^2 ，不是 g 的倍数。这一约定参考 [REP-145 的 IMU 报告约定（草案）](#)。

6.2 用三行公式读懂理想 IMU

二维部分只需要一个旋转转置：

$$\begin{bmatrix} f_x \\ f_y \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} a_x \\ a_y \end{bmatrix}, \quad f_z = g_0, \quad \omega_z = \dot{\psi}. \quad (6.2)$$

例如 yaw 为 90°、世界加速度为 (2, 0) 时，世界的 +x 方向正好是机体的 -y，故 $f = (0, -2, 9.81)$ 。

```

8 ImuSample idealImu(const State2D & state, double gravity)
9 {
10     ImuSample sample;
11     sample.angular_velocity.z() = state.yaw_rate;
12     sample.specific_force.head<2>() =
13         rotation(state.pose.yaw).transpose() * state.acceleration;
14     sample.specific_force.z() = gravity;
15     return sample;
16 }
17 //

```

ImuSample 只保存角速度与比力两个三维向量，算法不依赖 ROS。输入的 acceleration 成员由动力学或解析轨迹提供；不要用含噪位置二阶差分代替它。

运动模型的选择。第四章的 reference 提供一致的解析位置、速度和加速度；inertial 从力、质量和阻力计算加速度。二者适合本章。ideal 瞬移没有物理导数，velocity 的速度突变需要脉冲加速度，都不能据此伪造可融合的 IMU。

安装位置的意义。圆心原地自转时，圆心不产生向心加速度。圆心本身沿圆轨迹移动时则有。若以后把 IMU 安装到偏离圆心的位置，需要加入角速度和角加速度引起的杆臂项；本章不暗中模拟这些项。

加速度计不能区分静止和匀速直线运动；两者的运动加速度都是零。原始 IMU 也不直接测量位置、速度或绝对 yaw。

6.3 先手算，再运行五个场景

重新构建工作空间、加载 install 后运行：

```
ros2 run motion2d imu_demo
colcon test --packages-select motion2d
colcon test-result --verbose
```

演示的 CSV 输出可与下表核对。角速度单位 rad/s，比力单位 m/s^2 。

场景	ω_z	f_x	f_y	f_z
静止	0	0	0	9.81
匀速 $(2, -1) \text{ m/s}$	0	0	0	9.81
$a = (2, 0)$, $\text{yaw} = \pi/2$	0	0	-2	9.81
圆心原地自转	-0.3	0	0	9.81
$r = 1$, $\omega = 0.4$ 的圆参考	0.4	0	0.16	9.81

圆参考的机体 x 轴沿切向， y 轴朝路径圆心，故 $f_y = r\omega^2$ 。这与圆盘碰撞半径无关；不要把路径圆半径 r 与机器人半径混淆。

代码练习 ch06-1。在 `exercises/ch06/starter/specific_force.hpp` 补全世界加速度到机体比力的变换，从仓库根目录编译：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iexercises/ch06/starter \
exercises/ch06/check_specific_force.cpp -o /tmp/ch06_force
/tmp/ch06_force
```

理解题：静止、匀速和原地自转有何区别？实验题：将路径 r 翻倍，再单独将 ω 翻倍，预测和记录陀螺仪、加速度的变化。验收包括重力符号、正负 90° 旋转、 180° 旋转；提示与参考解单独保存在练习目录。

常见错误。直接把世界加速度写入机体系消息；把 g_z 符号写反；将陀螺仪单位写成度每秒；认为“运动很快”就意味着加速度很大。

6.4 让每个采样点带上一白噪声

最小测量模型将理想值与独立的高斯误差相加：

$$\omega_m = \omega + b_g + n_g, \quad f_m = f + b_a + n_a, \quad b_g = b_a = 0. \quad (6.3)$$

每个轴的每次采样独立， $n_i \sim \mathcal{N}(0, \sigma_i^2)$ 。本章先不模拟偏置、温漂或随机游走，也不把噪声裁剪到几个标准差内。同一采样的六个误差各抽一次；不能为整段运动只抽一个常数误差。

```

31 ImuSample addImuNoise(ImuSample sample, const ImuNoise & noise, std::mt19937 & random)
32 {
33     std::normal_distribution<double> normal(0.0, 1.0);
34     for (int axis = 0; axis < 3; ++axis) {
35         if (noise.gyro_stddev[axis] > 0) {
36             sample.angular_velocity[axis] += noise.gyro_stddev[axis] * normal(random);
37         }
38         if (noise.accel_stddev[axis] > 0) {
39             sample.specific_force[axis] += noise.accel_stddev[axis] * normal(random);
40         }
41     }
42     return sample;
43 }
44 //

```

默认陀螺仪三轴标准差为 (.002, .003, .004) rad/s，加速度计为 (.04, .05, .06) m/s²。这些是**每采样标准差**，不同轴可以不同。世界、雷达、IMU 各自拥有随机引擎；改变雷达束数不会推进 IMU 随机流。`reset` 重新设置 IMU 种子 6060，没有跨帧保留的高斯分布缓存。同配置、同轨迹、同采样顺序和同一标准库下可逐值重放，不承诺跨标准库字节一致。

标准差不等于噪声密度。必须先说明频谱约定、滤波带宽与采样方式才能换算。若连续白噪声强度为 q ，且每个读数是长度 Δt 的独立区间均值，则 $\text{Var}(n_k) = q/\Delta t$ 。这只是一个明确假设下的关系；本章直接配置每采样 σ ，改频率时不会自动改变它。

6.5 原始消息不提供航向答案

话题 `/imu/data_raw` 使用 `sensor_msgs/Imu`, 帧为 `imu_link`。 `linear_acceleration` 字段装比力, 不能仅凭字段名认为它已经去重力。姿态四元数用单位值占位, 协方差首项 -1 明确表示没有姿态估计。这遵循主机定义及[官方 Imu 消息约定](#)。

```

9  sensor_msgs::msg::Imu toImuMessage(const ImuSample & sample, const ImuNoise & noise,
10     const builtin_interfaces::msg::Time & stamp)
11  {
12     sensor_msgs::msg::Imu message;
13     message.header.stamp = stamp;
14     message.header.frame_id = "imu_link";
15     message.orientation.w = 1.0; // Placeholder, never a heading measurement.
16     message.orientation_covariance[0] = -1.0;
17     message.angular_velocity.x = sample.angular_velocity.x();
18     message.angular_velocity.y = sample.angular_velocity.y();
19     message.angular_velocity.z = sample.angular_velocity.z();
20     message.linear_acceleration.x = sample.specific_force.x();
21     message.linear_acceleration.y = sample.specific_force.y();
22     message.linear_acceleration.z = sample.specific_force.z();
23     for (int axis = 0; axis < 3; ++axis) {
24         message.angular_velocity_covariance[4 * axis] = std::pow(noise.gyro_stddev[axis], 2);
25         message.linear_acceleration_covariance[4 * axis] = std::pow(noise.accel_stddev[axis], 2);
26     }
27     return message;
28 }
29 //

```

独立噪声使 $C = \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_z^2)$; 行主序对角位置是 0、4、8, 单位也是原测量单位的平方。六个标准差均设为零可做理想实验; 但全零测量协方差在 ROS 协议中表示未知, 后续滤波器需要明确配置噪声模型, 不能理解成无限置信度。

代码练习 ch06-2: 补全 `starter/covariance.hpp`, 检验三轴方差和非对角项。例如 $\sigma_z = .004$ rad/s 对应方差 $1.6 \times 10^{-5} \text{ (rad/s)}^2$ 。

6.6 用统计量验收噪声

先在 `ros2_ws` 运行独立实验：

```
ros2 run motion2d imu_noise_demo
ros2 run motion2d imu_noise_demo \
> ../textbook/data/ch06_noise.csv
```

使用 `seed=6060`、每轴 $N = 60000$ 个误差样本。均值和总体标准差分别为

$$\bar{n} = \frac{1}{N} \sum_k n_k, \quad s = \sqrt{\frac{1}{N} \sum_k (n_k - \bar{n})^2}. \quad (6.4)$$

下面直接读取演示生成的 CSV；g 行单位为 rad/s，a 行为 m/s^2 。

轴	配置标准差	实测均值	实测标准差
gx	0.002	-0.000025	0.002007
gy	0.003	-0.000003	0.002990
gz	0.004	0.000002	0.003997
ax	0.04	-0.000328	0.039941
ay	0.05	0.000163	0.050240
az	0.06	-0.000462	0.060063

零均值指分布的期望，不要求有限样本均值恰好为零。均值的标准误为 $\sigma/\sqrt{N}$ ，因此测试使用 $|\bar{n}| < 5\sigma/\sqrt{N}$ ，标准差相对误差小于 2%。单元测试还检查六轴间和相邻时刻的相关量，防止不小心复用随机误差。

真实 ROS 消息实验。启动默认第 06 章场景后运行 `check_ch06.py`。它收集 200 Hz、 $t = 0$ 到 10 s 的 2001 条圆参考消息。先减掉理想值 (0,0,.4) 和 (0,.16,9.81)，再检验残差；不要对包含运动的原始值直接求“噪声均值”。因样本较少，标准差容差为 8%，均值仍用五个标准误。脚本也检查无缺帧、协方差、QoS、暂停和 41 帧 `reset` 重放；缺帧会明确失败。

6.7 零均值误差也会积成漂移

暂时假设姿态已知且固定，只研究一个水平加速度轴和 yaw 角速度的噪声。这排除了姿态误差与重力耦合，便于手算，不是惯性定位算法。把每个噪声样本看作一个时长 h 内的常值：

$$\delta\psi_{k+1} = \delta\psi_k + n_{g,k}h, \quad \delta p_{k+1} = \delta p_k + \delta v_k h + \frac{1}{2}n_{a,k}h^2, \quad \delta v_{k+1} = \delta v_k + n_{a,k}h. \quad (6.5)$$

独立噪声的方差相加。经历 N 个区间后，

$$\text{Var}(\delta\psi_N) = \sigma_g^2 N h^2, \quad (6.6)$$

$$\text{Var}(\delta p_N) = \sigma_a^2 h^4 \sum_{j=0}^{N-1} (j + \frac{1}{2})^2 = \sigma_a^2 h^4 \frac{N(4N^2 - 1)}{12}. \quad (6.7)$$

当 $T = Nh$ 固定且 N 较大，位置方差近似 $\sigma_a^2 h T^3 / 3$ 。固定每采样 σ 与固定连续白噪声强度，是两个不同的频率对比实验。

```
30     yaw += sample.angular_velocity.z() * dt;
31     position += velocity * dt + .5 * sample.specific_force.x() * dt * dt;
32     velocity += sample.specific_force.x() * dt;
33     //
```

```
ros2 run motion2d imu_noise_demo --drift
```

实验固定 $h = .005$ s、 $T = 10$ s、1000 次试验、seed=6060。两组复用同一随机序列，只把 σ 倍率从 1 改为 2。表中为位置误差 RMS 与解析标准差，单位 m：

σ 倍率	实测位置 RMS	解析位置标准差
1	0.053555	0.05164
2	0.10711	0.10328

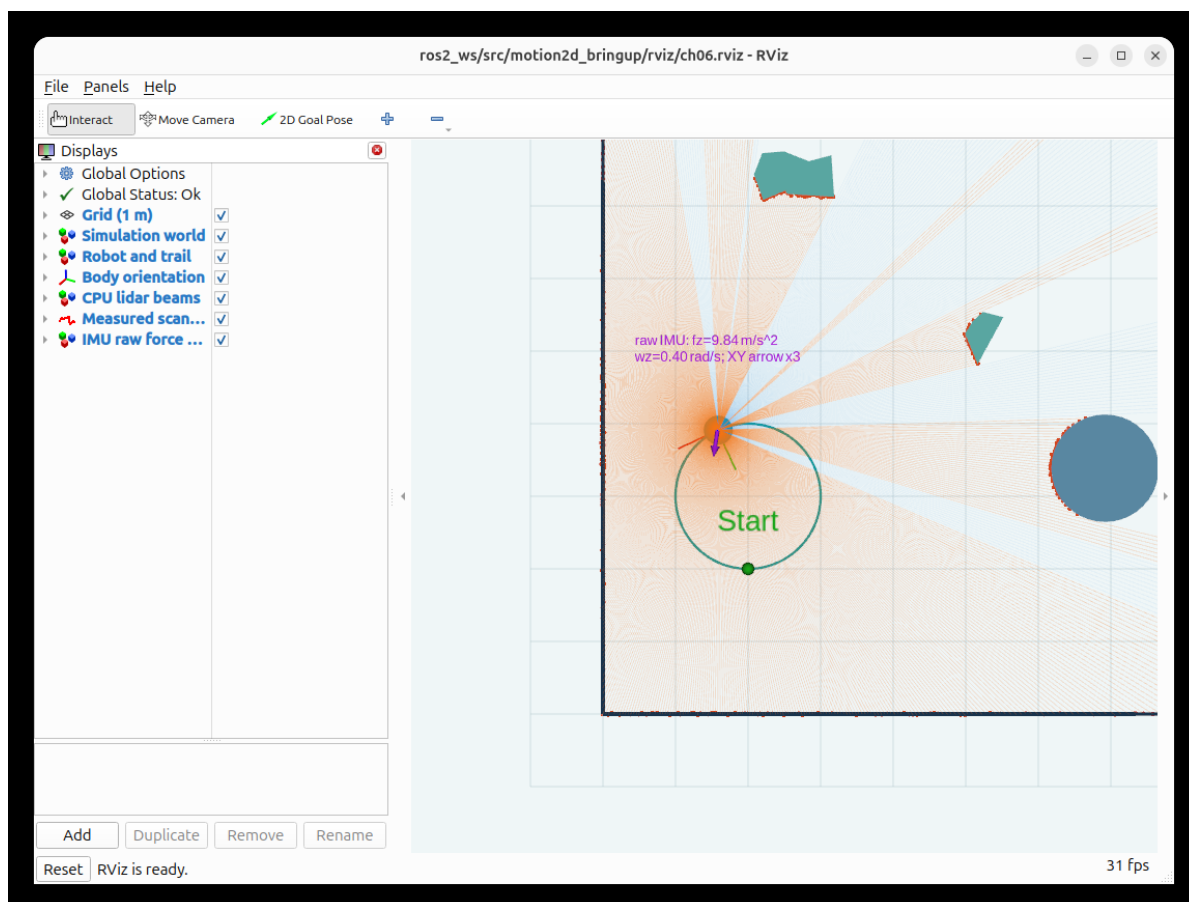
练习与验收：解释配对实验为何恰好翻倍；改变试验时长验证增长趋势。统计上的 RMS 增长不意味着单次误差的绝对值每步都增大。

6.8 在 RViz 中同时观察激光与 IMU

从 `ros2_ws` 构建、加载 `install` 后运行，默认选择连续的圆参考：

```
ros2 launch motion2d_bringup ch06.launch.py
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch06.py
```

参数在 `motion2d_bringup/config/ch06.yaml`，改后重启。`imu.rate` 默认 200 Hz, `imu.noise_seed` 为 6060; `imu.gyro_stddev`、`imu.accel_stddev` 均为长度 3 的数组。当前比力依赖连续运动，因此启用 IMU 时只接受 `reference` 或 `inertial`。要做静止实验，可选 `inertial` 且不发力；不可切换到瞬移模型生成伪 IMU。



紫色箭头显示实测水平比力，长度按 $1 \text{ m/s}^2 \mapsto 3 \text{ m}$ 放大；标签显示竖直比力和 z 角速度。噪声会让箭头轻微抖动。Marker 最多 20 Hz，原始 IMU 默认 200 Hz，两者频率不同。这些 Marker 使用真值采样姿态定位，只能辅助显示，不能交给 SLAM。

常见问题：只收到激光而无 IMU，先检查本章配置及模型；暂停后新订阅者收不到旧 IMU 属于 Volatile QoS 行为，恢复运行再观察。检查程序会 `reset` 并最终暂停。重置会让时间回退，已有 TF 缓存需要进入新试验；必要时恢复运行或重新启动 RViz，不把旧缓存警告当作新运动数据。

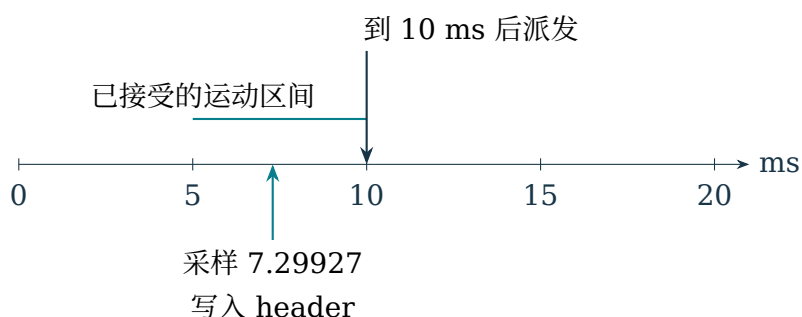
6.9 采样时间不是回调到达时间

一个 137 Hz 传感器的周期约为 7.29927 ms，无法写成 5 ms 仿真步长的整数倍。若强行每一个 tick 采样，它会变成 200 Hz；每两个 tick 则变成 100 Hz。本章为每个传感器保留独立的采样序号 k ，共享同一条模拟时间轴：

$$t_k = \text{round}\left(\frac{10^9 k}{f_s}\right) \text{ ns}, \quad k = 0, 1, 2, \dots \quad (6.8)$$

每次从 k 计算，舍入误差不会像反复累加一个已舍入的周期那样增长。纳秒网格的量化误差至多半个纳秒，具体纳秒间隔可能相差 1。

```
15 std::int64_t SensorScheduler::nextStamp() const
16 {
17     return std::llround(static_cast<long double>(index_) * 1000000000.0L / rate_);
18 }
19 //
```



采样、模拟器派发、订阅端墙钟到达，是三个不同的时间。本实现等一个 tick 通过运动与碰撞检查后才派发，因此模拟派发时间与采样时间相差小于 dt 。这个上界不包含墙钟执行、DDS 传输或排队时间。主机忙时，模拟时间也可能走得更慢。估计器应按 header 时间计算积分间隔，不能拿两次回调的墙钟差代替它。

代码练习 ch06-3：补全 `starter/sample_stamp.hpp`。检查 200、7、137.5 Hz 以及一小时后的采样点。单位测试保留第五章的整数 tick 辅助函数作对照；实际模拟器已使用本章调度器，启动 7 Hz 不再报错。

6.10 先接受运动区间，再产生观测

设刚接受的运动区间为 $[t_a, t_b]$ 。传感器时刻 t_s 落在其中时，只用已经确定的区间输入计算 $x(t_s)$ ，然后生成测量。**reference** 直接计算绝对时刻的圆轨迹；**inertial** 和 **velocity** 从 $x(t_a)$ 出发，用本区间保持的命令积分 $t_s - t_a$ 。两者复用第四章解析解，不是分别线性插值位置、速度和加速度。以后接入数值动力学时才需重新设计一致的区间插值。

```

148 void publishSensors()
149 {
150     const auto now = clock_.nanoseconds();
151     std::int64_t last_transform = -1;
152     while (true) {
153         const auto never = std::numeric_limits<std::int64_t>::max();
154         const auto laser_time = lidar_schedule_ ? lidar_schedule_->nextStamp() : never;
155         const auto imu_time = imu_schedule_ ? imu_schedule_->nextStamp() : never;
156         const auto stamp = std::min(laser_time, imu_time);
157         if (stamp > now) {break;}
158         const auto sampled = acquisitionState(stamp);
159         publishTruth(sampled, stamp);
160         last_transform = stamp;
161         if (stamp == laser_time) {publishLidar(sampled, stamp); lidar_schedule_->advance();}
162         if (stamp == imu_time) {publishImu(sampled, stamp); imu_schedule_->advance();}
163     }
164     if (last_transform != now) {publishTruth(state_, now);}
165 }
166 //

```

代码先选两个采样网格中较早的时刻；同一时刻可同时生成 IMU 和激光。每个采样位姿先发布对应的真值 TF，最后补齐 tick 末端的 TF。这样按采样 stamp 查询得到精确的模拟位姿，而不是误用最新一帧 TF。暂停时仍重复当前 TF 供晚加入者显示，但不重采样传感器。

边界约定。恰好在 tick 末端的 IMU 使用刚结束区间的加速度；晚到命令不能重写过去的测量。 $t=0$ 是初始状态，之后施力影响之后的区间。瞬移模型只在 tick 末端切换位置，不虚构中间轨迹，也不允许启用 IMU。碰撞拒绝整个候选区间，状态、时间、传感器序号均停在上次接受的位置。

6.11 不对齐频率也要逐帧可验证

停止旧场景，用无噪声空房间配置运行：

```
ros2 launch motion2d_bringup ch06.launch.py rviz:=false \
  config:=src/motion2d_bringup/config/ch06_time_lab.yaml
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch06_time.py
```

配置为 200 Hz 物理、137 Hz IMU、7 Hz 雷达，90 束扫描、20 m 正方形房间。启动时暂停，检查脚本走 200 个 tick 到 1 s。包含 $t=0$ ，应有 138 条 IMU 与 8 帧激光。IMU 第一个非零 stamp 为 7299270 ns，雷达为 142857143 ns；它们分别在 10 ms 和 145 ms 的 tick 结束后派发。

解析检查。reference 用圆轨迹核对比力、墙距和 TF。惯性实验的启动参数为 `model:=inertial`，检查脚本需加 `--model inertial`。其质量 2 kg、线性阻力.4 kg/s、转动惯量.04 kg m²、角阻力.02 kg m²/s。先静止.1 s，然后施加 (1, -0.3) N 与.01 N m。脚本按独立公式求出不断变化的运动，逐条比较比力、90 束墙距及采样位姿 TF。随后另起一次试验向墙运动，检查碰撞冻结。

reset 是新试验。它重置状态、命令、轨迹历史、采样序号与两个传感器的随机引擎。服务无法撤回已在 DDS 队列中的旧消息；后续估计器必须在模拟时间回退时清理积分和历史，或在新试验开始后重新建立消费者。验收脚本在 reset 屏障后建立新的 TF 监听器，并不表示修复了外部 RViz 的缓存实现。

主机 RViz 在 reset 附近曾出现静态 TF 缓存异常，关闭时也曾异常退出。自动检查建议先用 `rviz:=false`。显示时恢复模拟，再单独启动 RViz：

```
ros2 service call /sim/pause std_srvs/srv/SetBool '{data: false}'
ros2 run rviz2 rviz2 -d src/motion2d_bringup/rviz/ch06.rviz \
  --ros-args -p use_sim_time:=true
```

本章验收：会区分比力与运动加速度、标准差与方差、采样与到达时间；完成三道独立代码练习，能复现白噪声、积分漂移和不对齐频率实验。原始 IMU 仍不是位置、姿态或里程计答案；下一章再明确接入真值里程计与观测建图。

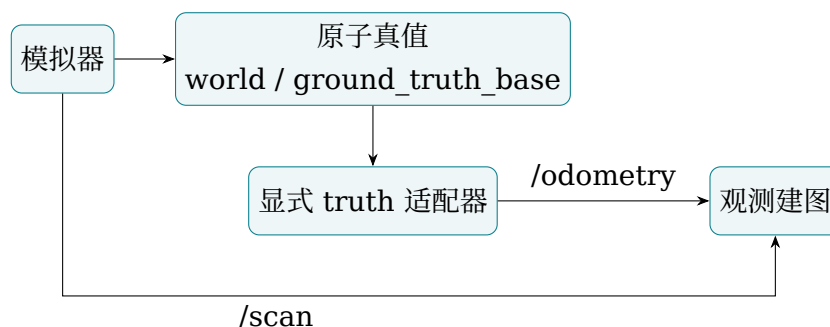
第七章 位姿已知时，如何从观测建图

目标 将定位与建图分开，用已知位姿解释激光观测，逐步得到规划可用的地图。

先修 SE(2)、连续运动、激光编码，以及第六章的采样时间。

7.1 先给建图一个已知的位姿

在同时学习定位和建图之前，先假设位姿完全已知。这能把“点为什么落错地方”拆成两个问题：是位姿不准，还是扫描投影和地图更新出了错？本章使用仿真真值做定位基线，仍只靠激光观测建图。障碍物列表属于模拟器和独立评估，不交给建图节点。



`/ground_truth/odometry` 保留独立的真值帧标识，不直接建立对应 TF 边。适配器输出统一入口 `/odometry`，使用 `odom / base_link`。本章 `world`、`map`、`odom` 的原点与轴向相同，因此转换只需明确改写帧标识；初始位置 `(-8,-8)` 不会偷偷变为 `(0,0)`。这是一项**已知的坐标对齐约定**，不是任意两个坐标系都能通过改字符串实现变换。

TF 边	唯一发布者
<code>map → odom</code>	<code>truth_odometry</code> ，单位变换
<code>odom → base_link</code>	<code>truth_odometry</code> ，随采样状态变化
<code>base_link → laser / imu_link</code>	<code>simulator</code> ，固定圆心外参

后续换成激光里程计或 SLAM 时，会替换这个真值入口。不能继续保留一条隐藏的真值 TF，让估计器从中读取答案；每条 TF 边也不能同时由两个节点发布。

7.2 同一条里程计消息里的两种坐标系

依据主机与[官方 Odometry 定义](#)，pose 在 header.frame_id，twist 在 child_frame_id。本章内部保存世界速度 v_W ，写入消息前需要计算

$$v_B = R(\psi)^\top v_W, \quad \omega_B = (0, 0, \dot{\psi}). \quad (7.1)$$

例如 $\psi = \pi/2$ 、 $v_W = (2, 1)$ ，则 $v_B = (1, -2)$ m/s。速度只转轴，不加位移；把位置 p 加到速度上不仅物理意义错误，单位也不一致。

```

7  nav_msgs::msg::Odometry truthOdometry(
8      const State2D & state, const builtin_interfaces::msg::Time & stamp)
9  {
10     nav_msgs::msg::Odometry message;
11     message.header.stamp = stamp;
12     message.header.frame_id = "world";
13     message.child_frame_id = "ground_truth_base";
14     message.pose.pose.position.x = state.pose.position.x();
15     message.pose.pose.position.y = state.pose.position.y();
16     message.pose.pose.orientation.z = std::sin(state.pose.yaw / 2);
17     message.pose.pose.orientation.w = std::cos(state.pose.yaw / 2);
18     const Eigen::Vector2d body_velocity =
19         rotation(state.pose.yaw).transpose() * state.velocity;
20     message.twist.twist.linear.x = body_velocity.x();
21     message.twist.twist.linear.y = body_velocity.y();
22     message.twist.twist.angular.z = state.yaw_rate;
23     return message;
24 }
25 //

```

位姿、速度、时间都来自同一个 State2D。若分别订阅位置与速度，再按“最新一条”拼接，可能混合不同采样时刻，尤其在转弯时不一致。本章原子消息避免了这种额外误差。零协方差表示仿真给出的精确状态，不代表真实定位系统能达到零误差。

练习 ch07-1：补全 exercises/ch07/starter/body_velocity.hpp。先手算正 90° 和 180° 旋转，再运行检查。思考圆参考中世界速度为何变化，机体前向速度却始终为 $r\omega$ 。

7.3 运行真值适配器并核对 TF

从 `ros2_ws` 构建、加载 `install` 后运行：

```
ros2 launch motion2d_bringup ch07.launch.py rviz:=false
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch07_odometry.py
```

默认圆参考、200 Hz 物理。真值还覆盖非整数频率传感器的精确采样时刻。暂停期间里程计和 TF 可重复当前时间，方便晚加入的显示节点；重复时间戳不能算作新观测。检查脚本核对原始/选源消息、机体速度、TF 位姿、唯一时钟与唯一运动 TF 发布者。

```
31     auto selected = truth;
32     selected.header.frame_id = "odom";
33     selected.child_frame_id = "base_link";
34     geometry_msgs::msg::TransformStamped transform;
35     transform.header = selected.header;
36     transform.child_frame_id = selected.child_frame_id;
37     transform.transform.translation.x = selected.pose.pose.position.x;
38     transform.transform.translation.y = selected.pose.pose.position.y;
39     transform.transform.rotation = selected.pose.pose.orientation;
40     dynamic_->sendTransform(transform);
41     output_->publish(selected);
42     //
```

第七章 `launch` 强制关闭模拟器的 `publish_truth_tf`，把运动 TF 交给适配器。第 04-06 章仍默认开启旧显示方式；不要同时启动两个章节。去掉 `rviz:=false` 可观察里程计箭头、机器人与扫描。本机 RViz 的重置缓存/关闭异常仍按第六章记录，自动 `reset` 检查建议先不开 GUI。

```
# Repository root: standalone exercise
g++ -std=c++17 -I/usr/include/eigen3 -Iexercises/ch07/starter \
    exercises/ch07/check_velocity.cpp -o /tmp/ch07_velocity
/tmp/ch07_velocity
```

常见错误：世界速度直接写入 `body twist`；把起点重定位为零却没同步地图；同时打开模拟器与适配器的运动 TF；把本章真值基线称为 SLAM。

7.4 从距离到二维点云

单束距离 r_i 与方向 α_i 给出雷达系点：

$$p_i^L = \begin{bmatrix} r_i \cos \alpha_i \\ r_i \sin \alpha_i \end{bmatrix}, \quad p_i^O = R(\psi(t_s))p_i^L + p(t_s), \quad \alpha_i = \alpha_{\min} + i\Delta\alpha. \quad (7.2)$$

t_s 是整帧 snapshot 扫描的采集时间。圆心外参为单位变换，因此 base_link 与 laser 共用位姿；偏置安装时应改用 $T_{OL} = T_{OB}T_{BL}$ ，不能只改 frame 字符串。例如 $r = 2$ m、 $\alpha = 0$ 、 $p = (3, 4)$ m、 $\psi = \pi/2$ ，变换结果为 $(3, 6)$ m。

```

20 std::vector<Eigen::Vector2d> projectScan(const Scan2D & scan)
21 {
22     std::vector<Eigen::Vector2d> points;
23     points.reserve(scan.ranges.size());
24     for (std::size_t i = 0; i < scan.ranges.size(); ++i) {
25         const double range = scan.ranges[i];
26         if (!std::isfinite(range) || range < scan.range_min || range > scan.range_max) {continue;}
27         const double angle = scan.angle_min + i * scan.angle_increment;
28         points.emplace_back(range * std::cos(angle), range * std::sin(angle));
29     }
30     return points;
31 }
32 //

```

只有有限且量程内的回波才进入点云。 $+\infty$ 表示“在量程内没有回波”，NaN 表示无效；二者都没有表面端点。将无穷替换为最大量程并画成点，会凭空造出一圈障碍物。

输出	坐标系	内容
/cloud/scan	laser	当前帧有效回波
/cloud/registered	odom	用采样位姿变换的同一批点

两种消息都使用 PointCloud2 并保留扫描时间。每点的 x、y、z 各用一个 float32，其中 z=0。消息只有一行 (height=1)，每点占用 12 字节 (point_step=12)。is_dense=true 表示输出不含无效点。这里的 registered 仅表示已按位姿转入统一坐标系，尚未进行扫描匹配。本章不累积 /map/cloud；后续全局位姿修正需要保留关键帧才能重建点云。

练习 ch07-2：完成 starter/scan_point.hpp，用 check_projection.cpp 核对手算坐标。进一步说明：“没有点的地方”为什么不能直接标为空闲？点云只记录命中表面，射线才携带沿途自由信息。

7.5 时间对齐比点云画出来更重要

同一条话题内部按采样顺序发布，不代表不同话题的回调先后相同。`mapping` 使用两个小缓存，按整数纳秒完全相等匹配扫描与位姿。模拟器已在每个扫描时刻输出真值状态，所以本章无需插值。真实传感器若没有这一条件，必须设计插值或同步策略，不能直接沿用此假设。

```
118     const auto points = motion2d::projectScan(scan->second.scan);
119     laser_cloud_->publish(motion2d::toPointCloud(points, scan->second.header));
120     auto header = scan->second.header;
121     header.frame_id = "odom";
122     registered_cloud_->publish(motion2d::toPointCloud(
123         motion2d::registerPoints(points, pose->second), header));
124     //
```

队列最多保留 1000 个不同时间的位姿、20 帧待处理扫描，缺少匹配时等待；扫描队列满后告警并丢弃最旧帧。不能用最近一次位姿伪装匹配成功。正时间的重复扫描不重复处理； $t=0$ 扫描是本模拟器的启动/重置标记，即使已暂停在零时刻，也重新建立首帧。暂停心跳只刷新位姿，不产生新点云。未来接入更低频的估计里程计时，应输出同一帧扫描的估计位姿，或明确加入插值。

```
# ch07.launch.py rviz:=false is running
/usr/bin/python3 ../scripts/check_ch07_cloud.py
# Separate terminals/domain: mapping node only
ROS_DOMAIN_ID=47 ros2 run motion2d mapping_node
ROS_DOMAIN_ID=47 /usr/bin/python3 ../scripts/check_ch07_join.py
```

后一个脚本先发旧位姿，再发新扫描，确认无输出；补发同时间位姿后才产生点云。它也覆盖位姿先到、扫描先到、重复帧和两种重置到达顺序。默认场景脚本逐点核对真实 `LaserScan`、`PointCloud2` 与里程计，误差容差为 3×10^{-6} m，容纳消息 `float32` 舍入。

重置边界：本章 `reset` 会回到固定的 $t=0$ 初态并暂停。扫描时间回退时清除扫描队列与旧的正时间位姿，里程计回退时清除位姿历史。先等待新试验首帧和里程计到齐，再恢复；外部 DDS 旧消息不能被服务远程清空。不支持任意乱序回放或跨试验无屏障混流，必要时重启消费者。

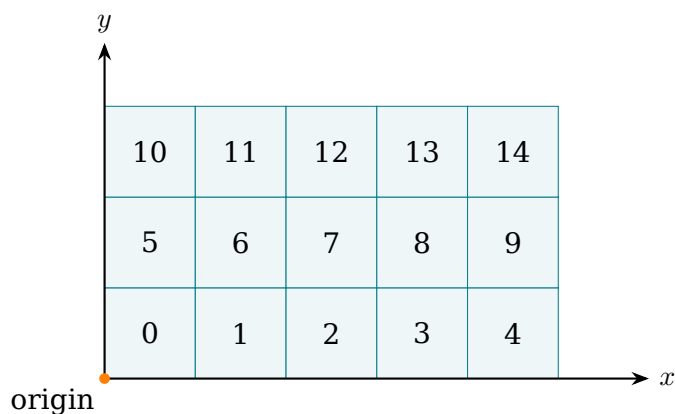
RViz 的 `Registered scan` 显示 `/cloud/registered`，橙色点应落在可见障碍表面附近。世界几何只是显示对照，可在 `Displays` 中关闭；`mapping` 的订阅中没有世界列表、真值调试话题或 TF。

7.6 格子有面积，未知也是一种状态

地图将连续空间切成边长 ρ 的正方形， ρ 的单位是 m/格。origin 是左下角格子的左下角，不是格心。对于地图内的位置 $p = (x, y)$,

$$i_x = \left\lfloor \frac{x - o_x}{\rho} \right\rfloor, \quad i_y = \left\lfloor \frac{y - o_y}{\rho} \right\rfloor, \quad j = i_y W + i_x. \quad (7.3)$$

W, H 是格数， x 索引变化最快。索引 j 的格心是 $o + \rho(i_x + \frac{1}{2}, i_y + \frac{1}{2})$ 。边界采用半开区间 $[o_x, o_x + W\rho) \times [o_y, o_y + H\rho)$ ；先检查边界，再取 floor，避免负数直接转整数时错误地朝零截断。



$W = 5$: 存储索引不是米制坐标

默认地图是独立配置的 220×220 格、 $\rho = 0.1$ m、 $\text{origin} = (-11, -11)$ m。已知地图存储范围不等于已知障碍物；初始所有格子都未知。参数使用 `mapping.width_cells`、`height_cells`、`resolution` 与 `origin_x/y`。改变分辨率时若要保持物理范围，必须同时调整格数。

三种含义 `unknown`: 还没有射线信息；`free`: 多次观测支持空闲；`occupied`: 回波支持占据。一片没有点云的区域，可能空旷，也可能被墙遮挡，不能把它全部刷白。

本章使用独立的 `observed` 掩码：尚未观察时输出 -1；已观察但概率恰为 0.5 时输出 50。二者不相等。地图固定大小；雷达中心在地图外时拒绝本帧地图更新并告警，不静默扩图。

7.7 DDA：沿射线依次走过哪些格子

将线段写成 $p(t) = s + t(e - s)$, $0 \leq t \leq 1$ 。记 $\Delta = e - s$ 。下一条竖格线和横格线的参数分别为 t_x, t_y ；穿过一格后，增量为

$$\delta t_x = \frac{\rho}{|\Delta_x|}, \quad \delta t_y = \frac{\rho}{|\Delta_y|}. \quad (7.4)$$

比较两个下一次穿线时刻，较小的一侧先前进一步；轴向差为零时对应值取 $+\infty$ 。两者恰相等表示经过角点，本章同时推进两轴，不计只触到角点的旁边格子。这是观测射线遍历，不能拿来证明圆盘机器人无碰撞。

```

81 while (x != end_x || y != end_y) {
82     // A negative-direction endpoint can lie exactly on a grid line.
83     // Do not step an axis past the endpoint's half-open cell.
84     if (x == end_x) {next_x = inf;}
85     if (y == end_y) {next_y = inf;}
86     if (next_x < next_y) {
87         x += step_x;
88         next_x += dt_x;
89     } else if (next_y < next_x) {
90         y += step_y;
91         next_y += dt_y;
92     } else { // Exact corner: advance both axes.
93         x += step_x; y += step_y;
94         next_x += dt_x; next_y += dt_y;
95     }
96     cells.push_back(y * config_.width + x);
97 }
98 //

```

起点必须在地图内，终点可以在外部。先将线段裁到地图范围，再遍历起点格到裁剪后的终点格。外部回波只提供图内的自由信息；裁剪边界不是测到的表面，不能在那里标占据。负方向终点恰好在格线上时，其所属格子由半开规则决定；已经到达的轴不再前进。

练习 ch07-3：补完 `starter/grid_traversal.hpp` 的循环。练习已准备好下一格线的参数，只考虑单位格长、原点为零、两个端点在图内；工程核心另外包含边界裁剪。使用 `check_traversal.cpp` 验证水平、竖直、正反对角线、同格和格线终点。先画格子手算访问次序，再运行参考解。

7.8 把射线变成概率证据

设某格占据概率为 p ，对数几率为

$$L = \log \frac{p}{1-p}, \quad p = \frac{1}{1 + \exp(-L)}. \quad (7.5)$$

先验 $p_0 = 0.5$ ，所以 $L_0 = 0$ 。使用简化的逆传感器模型：命中格给 $p_{\text{hit}} = 0.7$ ，穿过格给 $p_{\text{miss}} = 0.4$ 。在独立观测近似下，将贝叶斯乘法改写为加法：

$$L_k = \text{clip} \left(L_{k-1} + \log \frac{p_{\text{evidence}}}{1 - p_{\text{evidence}}} - L_0, -4, 4 \right). \quad (7.6)$$

命中增加约 0.847，穿过减少约 0.405。截断避免旧证据无限累积，也让后续矛盾观测有机会改变地图；默认输出概率约在 2% 到 98% 之间。

```

123 for (std::size_t i = 0; i < evidence.size(); ++i) {
124     if (evidence[i] == 0) {continue;}
125     const double increment = (evidence[i] & 2) ? hit_increment_ : miss_increment_;
126     log_odds_[i] = static_cast<float>(std::clamp(log_odds_[i] + increment,
127         -config_.logodds_limit, config_.logodds_limit));
128     observed_[i] = 1;
129 }
130 //

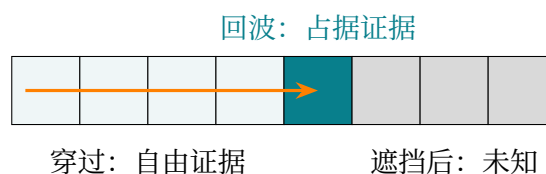
```

`evidence` 的两个标志分别表示本帧有射线穿过、有回波落入。同一帧同一格最多更新一次，命中优先。否则几百束激光共享的起点格会在一帧内被推到极限，细障碍的回波格也可能被相邻射线反复“清掉”。相邻束与相邻帧实际相关，因此这些数值是教学用置信度模型，不应声称得到了严格校准的独立贝叶斯后验。

消息取值	本课程含义	本章评估阈值
-1	尚未观测	unknown
0-100	占据概率 ×100 取整	≤ 35 free; ≥ 65 occupied
36-64	已观测但证据不足	uncertain

这些数值与阈值是课程约定，不是 ROS 强制的占据概率模型。未来规划默认把 `unknown` 与 `uncertain` 当作阻塞；占据概率也不是 ESDF 的米制距离。第一次穿过仅得到 40，第二次约为 31；不要要求单帧就将空闲格写成零。

7.9 有限回波、无回波和无效读数



```

107 for (std::size_t i = 0; i < scan.ranges.size(); ++i) {
108     const double range = scan.ranges[i];
109     const bool no_return = std::isinf(range) && range > 0;
110     const bool hit = std::isfinite(range) && range >= scan.range_min && range <= scan.range_max;
111     if (!hit && !no_return) {continue;}
112     const double angle = pose.yaw + scan.angle_min + i * scan.angle_increment;
113     const double length = no_return ? scan.range_max : range;
114     const Eigen::Vector2d end = pose.position +
115         length * Eigen::Vector2d(std::cos(angle), std::sin(angle));
116     const auto hit_cell = hit ? cellIndex(end) : std::nullopt;
117     for (const int cell : rayCells(pose.position, end)) {
118         evidence[cell] |= (hit_cell && cell == *hit_cell) ? 2 : 1;
119     }
120 }
121 //

```

有效有限距离：终点格为占据，之前的格子为空闲证据，不越过回波继续更新。 $+\infty$ ：沿方向清到 `range_max` 所在格，**不产生占据端点**。NaN、负无穷或量程外有限值：整条射线跳过，不清自由区。这保留了第五章近距遮挡与噪声越界的约定。清空从圆心开始，适用于本课程“过近命中记 NaN”的模拟器；真实设备的盲区与异常编码需由适配层重新确认。

一条射线只约束它穿过的格子；噪声也可能让端点前后摆动，所以障碍边界会变厚。`unknown` 不能被完整世界几何“补全”。教学演示中的完整世界只供可选对照显示与独立评估。

`/map` 在每帧成功更新后发布，使用 `map` 帧、扫描采样时间、可靠且 `transient-local` 的深度 1 队列。晚打开 RViz 也能读到暂停时的最后地图。`reset` 的首帧先清空概率与 `observed` 掩码，再插入 `t=0` 观测；`map_load_time` 记录本轮首次插入的扫描时间。

7.10 在 RViz 中检查“看见了什么”

```
ros2 launch motion2d_bringup ch07.launch.py
# For automated resets, launch with rviz:=false instead.
/usr/bin/python3 ../scripts/check_ch07_map.py
```

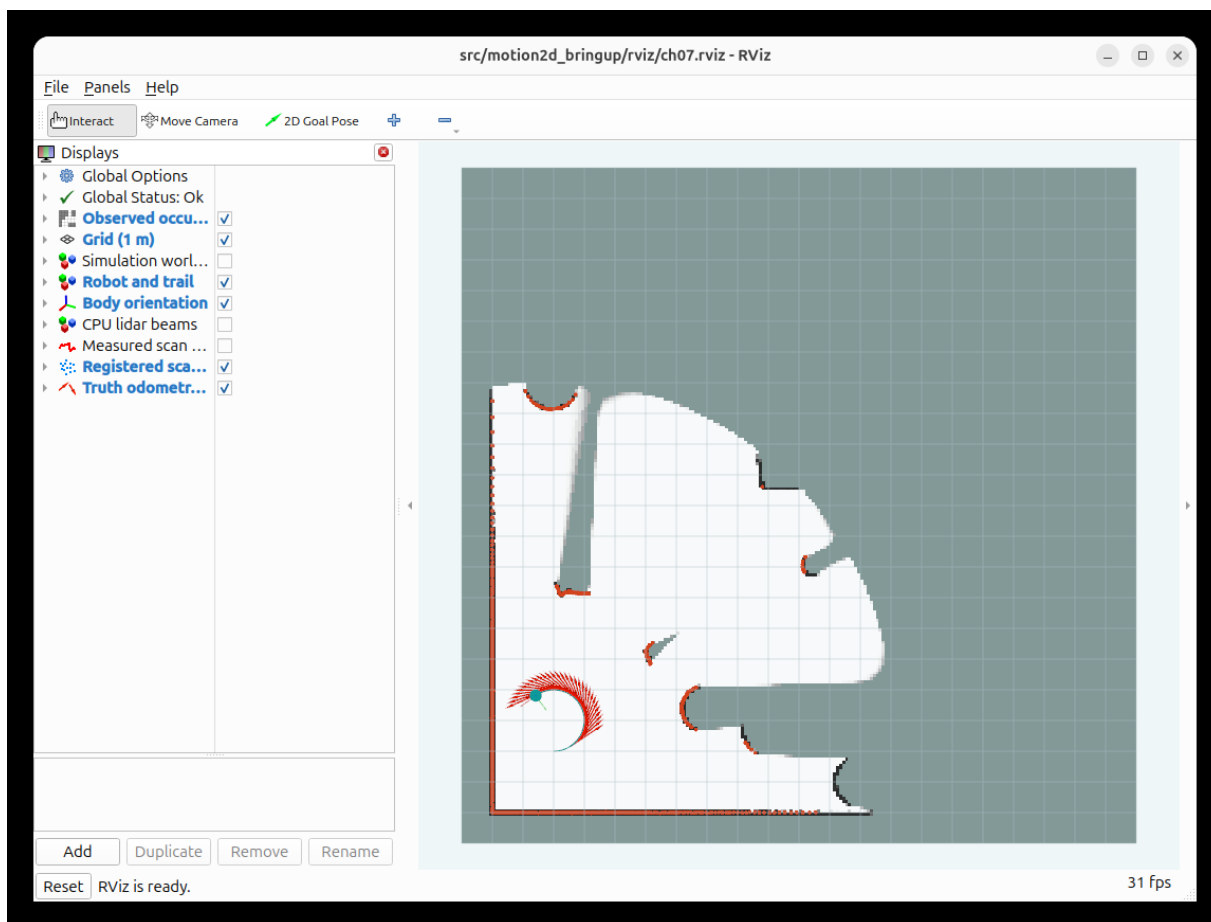


图 7.1: 本机 RViz2 实测。完整世界显示关闭；浅色是已观测自由域，深色轮廓是占据证据，灰色是未知。橙色点为当前回波，箭头为显式真值里程计。

一米半径的圆参考只覆盖起点附近视野，不是全图探索。右上方目标周围仍未知；这正是观测地图与完整真值世界的区别。打开 Simulation world 可对照形状，但该显示开关不影响 mapping 的输入。

本增量验收：82 个 GoogleTest 用例通过；DDA 另用 1000 条固定种子线段，与逐格线段-方格求交独立比对。ROS 默认场景 $t=0$ 到 1 s，已观测格数从 9579 增至 10154；晚加入接收、暂停、首帧重播和连续零时刻 reset 均通过。扫描配对与真值适配器检查也已回归。本机 RViz 的既有 reset/退出问题仍未修复；截图在自动检查后重新运行 GUI 获取。

7.11 独立真值栅格：只能用来评估

“用真值位姿建图”与“直接把真值地图交给建图器”是两件事。本章 `mapping_demo` 先生成扫描, 再用扫描和位姿更新观测图；更新结束后才调用 `rasterizeTruth` 生成对照。运行中的 `mapping_node` 不调用这个评估函数。

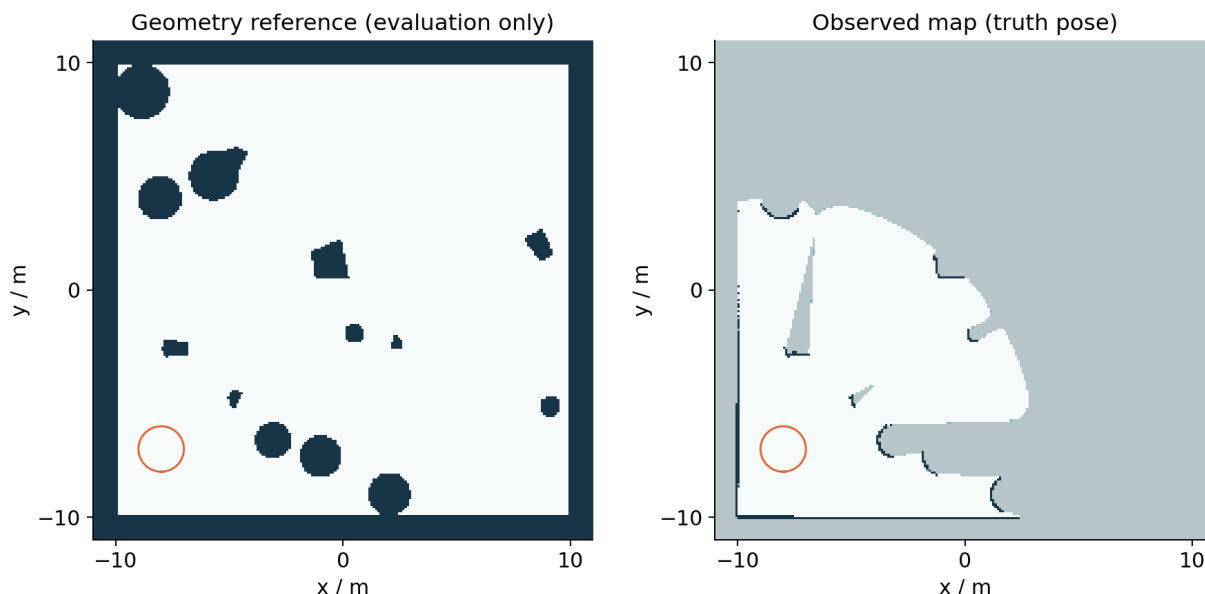


图 7.2: 相同 0.1 m 分辨率, 左侧为独立几何参考, 右侧为 160 帧无噪声观测。灰色未知没有被补全；橙线是采样轨迹。参考图不作为建图输入。

参考格子只要与实体或房间边界接触就记占据。圆用“圆心到方格最近点的距离”与半径比较；凸多边形用分离轴检验。它遍历完整几何，**不调用射线模拟器或地图更新函数**，从而避免用同一个建图结果检验自己。这仍是数值仿真的交叉验证，不是外部实测真值。

评估先记录 `observed` 比例，再排除 `unknown` 与 `uncertain`，仅对有明确判断的格子统计 TP、FP、FN、TN。TP 是正确的占据预测，FP 是把参考自由格判为占据，FN 是把参考占据格判为自由，TN 是正确的自由预测。

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall}_{\text{evaluated}} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (7.7)$$

分母为零时记 NaN，而不是填 100%。这里的 `recall` 只针对已评估格子，**不是整张世界地图的召回率**；`unknown` 也不能当作预测正确的自由格。

整格接触真障碍的参考规则比单射线“穿过了一部分格子”更保守。因此无噪声也可能有 FN；它揭示栅格边界与观测模型的差异，不能直接解读为米制定位误差或连续碰撞安全证明。

7.12 实验一：分辨率、噪声与评价指标

固定世界 seed=42、8 圆/8 多边形，沿一米圆参考运动，在 $t = 0, 0.1, \dots, 15.9$ s 采集 160 帧、每帧 720 束、最大量程 10 m。噪声 seed=4242, $\sigma = 0/0.05$ m。每组噪声的扫描只生成一次，在三种分辨率上复用；角度和量程采用与 LaserScan 一致的 float32 元数据。

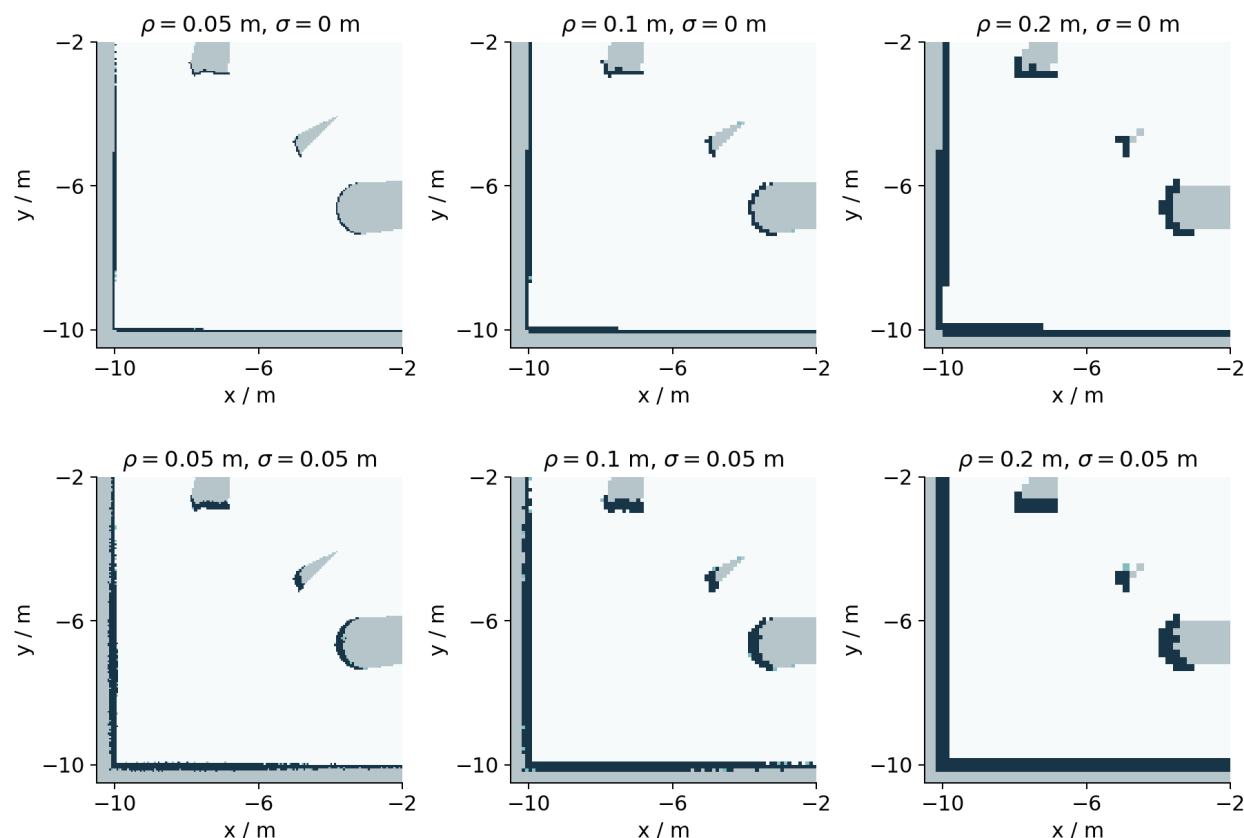


图 7.3: 相同位置的局部放大。白色自由、青色不确定、深色占据、灰色未知。上排无噪声, 下排 $\sigma = 0.05$ m; 列方向分辨率逐渐变粗。

地图始终覆盖 22×22 m; 格数依次为 $440^2, 220^2, 110^2$ 。分辨率减半会使二维格数变为四倍, 但不会增加雷达原本没有测到的信息。图中边界随着分辨率和噪声变厚, 细格也会保留角采样造成的间隙。

ρ/m	σ/m	观测比例	precision	局部 recall
0.05	0	0.275	1	0.63
0.1	0	0.281	1	0.667
0.2	0	0.29	1	0.715
0.05	0.05	0.281	0.977	0.807
0.1	0.05	0.287	0.983	0.823
0.2	0.05	0.295	0.988	0.846

完整 CSV 同时保存不确定格数与四项计数。不要只挑更高的 recall: 噪声让部分障碍内部也被“看到”, 把占据带画厚, 可能提高此指标; 观测比例增大也可能来自越过真实表面的噪声, 不能称为探索能力变强。

7.13 实验二：墙到底变厚了多少米

用空房间隔离形状与遮挡因素：右墙位于 $x = 4.03$ m，雷达静止在原点，161 束覆盖 $[-0.4, 0.4]$ rad、量程 6 m。每组 160 帧、seed=4242，比较 $\sigma = 0, 0.05, 0.10$ m。4.03 避开格线，减少“真实墙恰落在两个格子之间”的量化混淆。

在格心 $y \in [-1, 1]$ m 的每一行，取 $x \in [3, 5]$ m 内被判占据的首末格，计算跨度，再对有占据的行求平均：

$$w_y = (i_{x,\max} - i_{x,\min} + 1)\rho, \quad \bar{w} = \frac{1}{N_{\text{rows}}} \sum_y w_y. \quad (7.8)$$

跨度包括中间间隙，并非逐格相加的实体面积；没有占据的行不参与均值，必须同时报告行数，防止漏检被平均值掩盖。

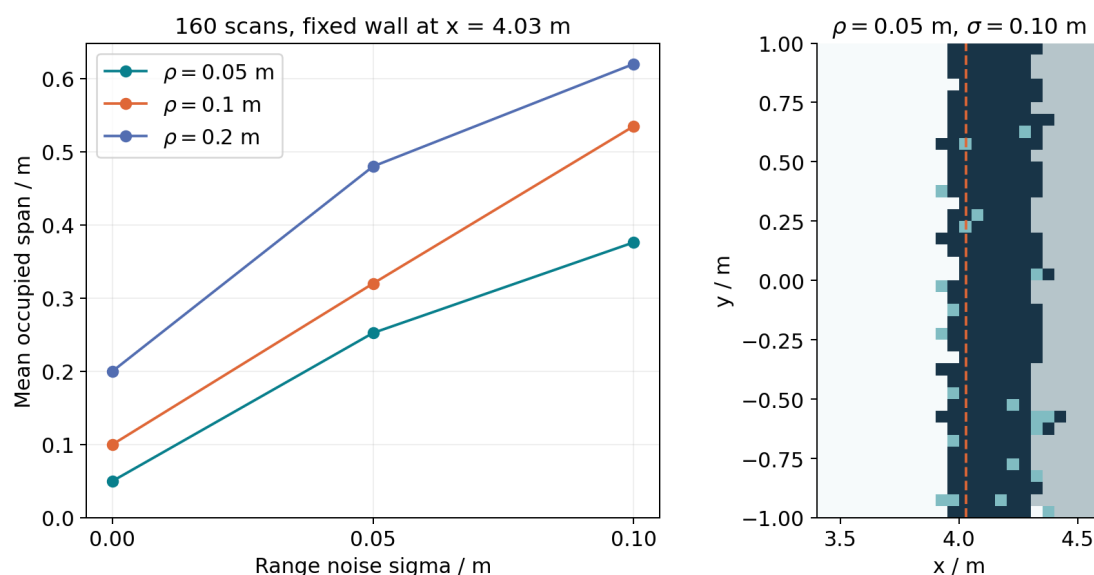


图 7.4: 左：实测平均占据跨度。右：细格、大噪声的局部地图，虚线为真实墙面。简化逆传感器模型会生成偏厚、偏移的占据带。

ρ/m	σ/m	平均跨度/m	有效行数
0.05	0	0.05	40
0.1	0	0.1	20
0.2	0	0.2	10
0.05	0.05	0.253	40
0.1	0.05	0.32	20
0.2	0.05	0.48	10
0.05	0.1	0.376	40
0.1	0.1	0.535	20
0.2	0.1	0.62	10

无噪声时，本实验占据带是一格厚；0.1 m 分辨率下，噪声从 0 增至 0.05 m 时，平均跨度由 0.10 增至 0.32 m。这不是障碍真正变厚，而是有限格长、端点噪声和证据更新共同造成的结果。降低分辨率数值不能自动修复测距误差；更合理的逆传感器模型留作进阶。

7.14 复现实验与本章练习

```
# Repository root, after sourcing the ROS install environment
ros2 run motion2d mapping_demo tmp/ch07_eval
/usr/bin/python3 scripts/plot_ch07.py tmp/ch07_eval textbook/figures
# Exercise ch07-4 (replace starter with solutions for reference)
g++ -std=c++17 -Iexercises/ch07/starter \
    exercises/ch07/check_metrics.cpp -o /tmp/ch07_metrics
/tmp/ch07_metrics
```

绘图使用主机 Matplotlib/NumPy。CSV 保存在实验目录；教材表格读取 textbook/data/ch07_mapping.csv 与 ch07_wall.csv 的已验收副本。PGM 文件是紧凑实验数据，255 表示未知，其余为概率百分比，绘图按物理 y 轴向上显示。同机同标准库的两次运行，全部 18 个输出文件逐字节一致。

```
77 for (std::size_t i = 0; i < observed.size(); ++i) {
78     const int value = observed[i];
79     if (value < -1 || value > 100 || truth[i] > 1) {
80         throw std::invalid_argument("invalid occupancy encoding");
81     }
82     if (value == -1) {continue;} // Unknown is not a correct free prediction.
83     ++metrics.observed;
84     if (value > 35 && value < 65) {++metrics.uncertain; continue;}
85     if (value >= 65) {
86         if (truth[i]) {++metrics.true_positive;}
87         else {++metrics.false_positive;}
88     } else {
89         if (truth[i]) {++metrics.false_negative;}
90         else {++metrics.true_negative;}
91     }
92 }
93 //
```

1. **理解：**为什么已知定位仍会得到不完整地图？为什么更大的观测比例或 recall 不一定代表更好？
2. **代码 ch07-4：**完成 starter/observed_metrics.hpp。必须先排除 -1 与 36-64，再统计四项计数；35/65 是包含端点的阈值。
3. **实验：**保持世界、轨迹、量程、束数与扫描序列相同，仅改变分辨率。报告格数、观测比例、四项计数、不确定数量与米制墙厚。再改变噪声；说明表面变厚如何影响精度与召回率。

完成边界：本章具备已知位姿下的观测建图，四道代码练习与参考解同步，不是 SLAM。下一章先研究如何从扫描本身估计运动；在得到可验证的估计位姿之前，不把真值 TF 混入估计器。建图接口仍是扫描加位姿。

第八章 让两帧扫描对齐：激光里程计

目标 从最近邻和最小二乘理解二维配准，再把相对运动积累成里程计。

先修 第 01-07 章。先实现只接收两组点的配准内核。

8.1 先对齐两组已知形状的点

在 `ros2_ws` 加载环境后运行：

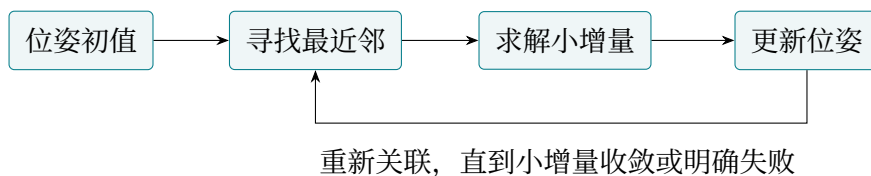
```
ros2 run motion2d scan_matcher_demo
```

演示先造两面互相垂直的墙，再用已知位姿 $t = (0.12, -0.08)$ m、 $\psi = 0.04$ rad 生成当前帧。配准函数只收到两组点和初值，不收到这个答案。输出包括两种残差的状态、位姿、RMSE、关联数与迭代次数。真实雷达点还会受遮挡、噪声和视角变化影响。实验同时使用零初值，以及 $(0.11, -0.075, 0.038)$ 的邻近初值。零初值下，点到点结果约为 $(0.1030, -0.0865, 0.0474)$ ：虽然报告收敛，仍有约 1.8 cm 的位置误差。邻近初值能恢复本例答案，点到线则在两个初值下都成功。这个例子说明规则采样也可能形成错误的局部极小值。

设当前帧点为 p_i ，参考帧点为 q_j 。我们求的是 $T_{\text{reference} \leftarrow \text{current}}$ ：

$$e_i = R(\psi)p_i + t - q_{j(i)}, \quad j(i) = \arg \min_j \|R(\psi)p_i + t - q_j\|. \quad (8.1)$$

先用当前猜测寻找最近邻，固定这些对应关系解一个小问题，再重新寻找对应关系。这就是 ICP 的交替过程。对应关系也在变化，所以它不是一次线性求解，也不保证找到全局最优。



超过关联距离的点不参与本轮优化。默认门限为 0.5 m，最多 40 次迭代，至少 20 个有效关联。门限不是越大越好：太大容易认错墙，太小则运动稍快就没有足够的点。

8.2 点到点残差的两行雅可比

本章使用加法更新 (t_x, t_y, ψ) 。记 $u = R(\psi)p_i$ ，则

$$J_i = \frac{\partial e_i}{\partial(t_x, t_y, \psi)} = \begin{bmatrix} 1 & 0 & -u_y \\ 0 & 1 & u_x \end{bmatrix}. \quad (8.2)$$

旋转列不包含平移。这里的增量约定不是 SE(2) 左乘或右乘指数映射；换一种更新方式，雅可比也要随之改变。

```
101 Eigen::Matrix<double, 2, 3> j;
102 j << 1, 0, -rotated.y(),
103     0, 1, rotated.x();
104 return j;
105 //
```

线性化后 $e_i(\theta + \delta) \approx e_i + J_i \delta$ 。求导令零得到

$$H = \sum_i w_i J_i^\top J_i, \quad g = \sum_i w_i J_i^\top e_i, \quad H \delta = -g. \quad (8.3)$$

不要显式求 H^{-1} ；用分解求解线性方程。默认 Huber 尺度为 0.08 m：

$$\rho(s) = \begin{cases} \frac{1}{2}s^2, & |s| \leq a, \\ a|s| - \frac{1}{2}a^2, & |s| > a, \end{cases} \quad w = \min(1, a/|s|). \quad (8.4)$$

零残差时直接取 $w = 1$ 。点到点使用残差模长，点到线使用有符号距离。鲁棒权重减弱大残差的影响，不能凭空识别所有错误对应。

代码练习 ch08-1：补完 `starter/point_jacobian.hpp`。检查程序在三个朝向对独立投影公式做中心差分。先手算，再运行；不要照抄一个与自己实现完全相同的公式充当测试。

8.3 点到线：沿墙滑动和穿墙不同

参考表面的单位法向量为 n_i 时，

$$r_i = n_i^\top (Rp_i + t - q_i), \quad J_i^{\text{line}} = n_i^\top J_i. \quad (8.5)$$

它主要惩罚穿过表面的误差，降低沿表面离散采样位置不一致的影响。本实现对参考点半径 0.4 m 内的邻域做 PCA；至少 5 点，且小特征值小于大特征值的 0.15 倍，才认为局部像一条线。法向量正负不影响平方残差；孤立点和不可靠的角点不提供法向约束。

8.4 配准失败必须有名字

只有一面水平墙时，点到线误差不包含沿墙的 t_x ， H 秩不足。检查最小与最大特征值之比，能识别这类局部退化。点到点的错误最近邻、重复房间却可能仍给出满秩矩阵，因此满秩不等于正确定位。

```

145     const Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> eigen(system.h);
146     if (eigen.eigenvalues()[0] <= c.min_eigen_ratio * eigen.eigenvalues()[2]) {
147         result.status = MatchStatus::Degenerate;
148         return result;
149     }
150     if (small_step) {
151         result.status = result.rmse <= c.max_rmse ?
152             MatchStatus::Converged : MatchStatus::HighResidual;
153         return result;
154     }
155     if (k == c.max_iterations) {break;}
156     Eigen::Vector3d delta = -system.h.ldlt().solve(system.g);
157     // Keep each local update moderate; this does not rescue a bad initial guess.
158     const double scale = std::max({1.0, delta.head<2>().norm() / .3, std::abs(delta.z()) / .2});
159     delta /= scale;
160     result.pose.position += delta.head<2>();
161     result.pose.yaw = wrapAngle(result.pose.yaw + delta.z());
162     ++result.iterations;
163     small_step = delta.head<2>().norm() < c.translation_tolerance &&
164         std::abs(delta.z()) < c.yaw_tolerance;
165     //

```

结果分为 `converged`、`too_few_pairs`、`degenerate`、`iteration_limit` 和 `high_residual`。只有增量小于容差、重算后的关联数和秩合格、RMSE 不超过门限，才接受位姿。返回的加权法方程矩阵只是局部信息指标，不能直接宣称是经过标定的真实协方差。

1. **理解**：为什么增大门限可能让残差下降，却得到错误的位姿？
2. **实验**：改变已知变换和初值，比较点到点与点到线；删掉一面墙观察退化。
3. **验收**：参考解通过中心差分；刚体点集恢复位姿；空点集、无重叠、单墙退化和迭代耗尽均报告失败。两种 RMSE 的含义不同，不能只凭数值大小比较精度。

配准内核只处理一次点云对齐；下面由激光里程计管理时间历史、参考点云和失败帧。

8.5 把成功扫描留在一个小窗口中

每一帧都与上一帧匹配会连续累积误差。我们把最近几个关键帧转换到 `odom` 系，作为局部参考表面。默认最多保留五帧；累计平移达到 `0.2 m` 或转角达到 `0.15 rad` 时才插入。用边长 `0.06 m` 的方格做体素降采样，每格保留质心，避免同一面墙被重复点过度加权。它仍然是点云窗口，不是第七章的占据栅格。

设前两次成功估计的时刻为 t_{k-1}, t_k ，下次扫描在 t 到达。用割线速度提供局部初值：

$$\hat{v}_k = \frac{p_k - p_{k-1}}{t_k - t_{k-1}}, \quad \hat{\omega}_k = \frac{\text{wrap}(\psi_k - \psi_{k-1})}{t_k - t_{k-1}}, \quad p_{\text{init}} = p_k + (t - t_k)\hat{v}_k. \quad (8.6)$$

航向同样外推并 `wrap`。这里使用 `odom` 平移速度；匀速只是给 ICP 一个较近的起点，不能替代本帧观测。一次配准失败后，分母必须包含跨过的时间，不能仍除以固定扫描周期。**练习 ch08-2** 的代码入口为 `constant_velocity.hpp`，检查缺帧和跨越 $\pm\pi$ 的情况。

```
84 result.match = matchClouds(source, target_, initial, config_.match);
85 result.status = matchStatusName(result.match.status);
86 if (!result.match.accepted()) {return result;}
87 velocity_ = (result.match.pose.position - pose_.position) / dt;
88 yaw_rate_ = wrapAngle(result.match.pose.yaw - pose_.yaw) / dt;
89 pose_ = result.match.pose;
90 last_accepted_ = stamp;
91 if ((pose_.position - keyframe_pose_.position).norm() >= config_.keyframe_distance ||
92     std::abs(wrapAngle(pose_.yaw - keyframe_pose_.yaw)) >= config_.keyframe_angle) {
93     addKeyframe(source, pose_);
94 }
95 //
```

拒绝帧不更新位姿、速度、成功时刻和局部地图。下一帧仍可尝试恢复；连续超过 `1 s` 没有成功观测则报告 `tracking_lost`，等待明确 `reset`。窗口限制了内存和匹配规模，但不能消除长期漂移。重复几何仍可能让错误配准通过局部检查。

8.6 用估计里程计替换真值入口

重新构建并加载工作区后，运行：

```
1 ros2 launch motion2d_bringup ch08.launch.py
2 # For automatic reset checks, launch with rviz:=false.
3 # Second terminal, from ros2_ws with the same ROS environment:
4 /usr/bin/python3 ../scripts/check_ch08.py
```

激光节点只订阅 `/scan`，发布 `/odometry/estimated`；显式 `estimated` 适配器将其转为统一 `/odometry`，并独占 `odom` 到 `base_link` 的 TF。模拟器仍发布独立真值供评估，但 `launch` 关闭了模拟器运动 TF，也没有运行 `truth` 适配器。传感器静态外参保持不变。

第一帧定义 `odom` 原点与朝向；现在地图中的起点是 `(0,0)`。真实世界起点仍是 `(-8,-8)`，两者不能不经变换直接叠加。RViz 默认只显示观测栅格、注册扫描、估计轨迹和机体轴；真值世界与真值机器人显示关闭。`map` 暂与 `odom` 重合。

有状态就必须讨论失败 `/estimation/status` 给出每次配准状态。未收敛时不发布该采样时刻的里程计，建图等待其他有精确配对的扫描，再丢弃未配对旧帧。不会复制上次位姿并伪造一个新时间戳。恢复不等于全局重定位。

里程计 `twist` 是成功扫描间的平均速度，在机体系表达。`pose` 协方差暂用固定测量尺度：平移标准差 0.03 m、航向 0.01 rad；尚未经过统计标定，也不是从配准信息矩阵直接求逆得到。第九章会解释如何把这些尺度放进滤波器，以及过度自信的后果。`reset` 清空首帧原点、关键帧、速度和轨迹。零时刻重新采样；正时间重复帧被忽略。

常见错误：把真值 TF 当初值；把失败帧加入局部地图；拿当前最新位姿变换旧扫描；直接比较不同原点的轨迹。主机 RViz 在时间回拨时仍可能发生 TF 缓存异常；自动 `reset` 验收使用无界面模式，恢复运动后重新启动 RViz。图形窗口重启不负责重置算法。

8.7 实验：束数、噪声与被拒绝的帧

在仓库根目录加载工作区环境，运行：

```
1 ros2 run motion2d lidar_odometry_demo tmp/ch08_results
2 /usr/bin/python3 scripts/plot_ch08.py
```

实验固定 `world seed=42`、`noise seed=4242`、圆参考半径 1 m、角频率 0.4 rad/s, $t = 0, 0.1, \dots, 15.9$ s 共 160 帧。估计器只得到点云；独立评估器用首帧真值定义同一个参考系，再计算位置误差：

$$T_k^{\text{ref}} = (T_0^{\text{truth}})^{-1} T_k^{\text{truth}}, \quad \text{RMSE} = \sqrt{\frac{1}{|\mathcal{A}|} \sum_{k \in \mathcal{A}} \|p_k^{\text{est}} - p_k^{\text{ref}}\|^2}. \quad (8.7)$$

$\mathcal{A}$ 只含成功帧，因此必须同时报告拒绝数。下表直接读取 C++ 实验 CSV。无噪声时也可能因离散扫描、可见性和迭代门限而拒绝某些帧。

束数	σ_r/m	接受	拒绝	位置 RMSE/m
90	0	155	5	0.0041
90	$3 \cdot 10^{-2}$	150	10	0.0257
360	0	158	2	0.0018
360	$3 \cdot 10^{-2}$	154	6	0.0136
720	0	156	4	0.0025
720	$3 \cdot 10^{-2}$	154	6	0.0101

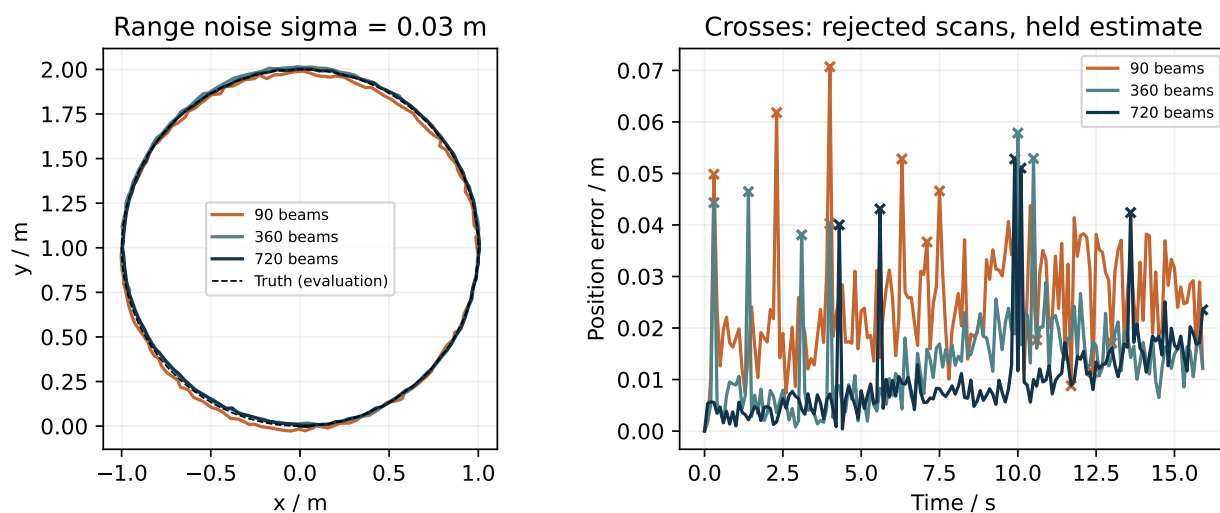


图 8.1: 首帧对齐后的轨迹与位置误差。叉号是拒绝帧, 此时画出保持上次估计的误差; 圆轨迹保持等比例。

默认 ROS 实验的 0 到 3 s 共 31 次扫描中, 30 次成功、1 次拒绝, 成功帧最大位置误差约 0.00214 m。这是一个固定种子小场景, 不能推广为普遍精度承诺。教材的六组回放也没有在配准后用真值修正结果。720 束不保证每项指标都优于 360 束: 法向量、关联和噪声共同影响局部优化。

本章验收: 核心测试覆盖独立差分、恢复、退化、缺帧和有限窗口; 实际 ROS 核对输入隔离、采样时刻、点云/地图、暂停与重播。继续实验时, 一次只改一个参数, 记录成功率、误差及失败状态。下一章使用原始 IMU 预测和激光校正, 在同一时间轴上估计速度与偏置。

第九章 让 IMU 预测，让激光纠偏

目标 在同一时间轴上估计位置、速度、航向与偏置，理解预测和校正的分工。

先修 第 06、08 章；本章逐步解释协方差和卡尔曼增益。

9.1 先单独验证滤波器

重新构建并加载工作区后，从仓库根目录运行：

```
ros2 run motion2d planar_ekf_demo tmp/ch09_filter
/usr/bin/python3 scripts/plot_ch09_filter.py
```

先把两个问题分开：滤波器的矩阵是否正确，真实扫描匹配能否提供可靠观测？这个实验只检查前者。位姿输入是从解析轨迹生成的带噪合成测量，不是激光 *SLAM* 验证。后续在线版本才使用第八章扫描匹配的实际输出。

IMU 给短时间运动变化；它并不知道初始速度或全局位置。位姿观测虽然较慢，却可以修正累积误差，并通过时间关联帮助估计速度与偏置。二者噪声模型不同，不能简单把两个位置取平均。

9.2 八个数描述平面状态

采用状态列向量

$$x = (p_x, p_y, v_x, v_y, \psi, b_{ax}, b_{ay}, b_g)^\top. \quad (9.1)$$

p, v 在 odom 系， ψ 是航向；加计偏置 b_a 在机体系，单位 m/s^2 ；陀螺偏置 b_g 单位 rad/s 。原始 IMU 的 `orientation` 字段不参与计算。因为机器人水平运动且 `roll`、`pitch` 为零，重力没有 x/y 分量：

$$a_k = R(\psi_k)(f_{xy,k} - b_{a,k}), \quad \omega_k = \omega_{m,k} - b_{g,k}. \quad (9.2)$$

对时长 h 的一个小区间，用步首航向旋转并保持加速度，得到

$$\begin{aligned} p_{k+1} &= p_k + h v_k + \frac{1}{2} h^2 a_k, & v_{k+1} &= v_k + h a_k, \\ \psi_{k+1} &= \text{wrap}(\psi_k + h \omega_k), & b_{k+1} &= b_k. \end{aligned} \quad (9.3)$$

这是离散近似：连续转动时，区间内的 R 实际也在变化。缩短步长可以减小该近似误差；不能把公式说成一般旋转运动的精确积分。当前内核接受 $0 < h \leq 0.1 \text{ s}$ ，长时间缺测应报告失效而不是一步越过。

9.3 不确定性如何沿着状态传播

协方差 P 的对角项描述方差，非对角项描述两个误差的关联。例如位置与速度相关后，一个位置校正也能改变速度。令 $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ ， $A_\psi = R(\psi)J(f_{xy} - b_a)$ 。状态转移雅可比中的几个关键块为

$$\begin{aligned} F_{pv} &= hI, & F_{p\psi} &= \frac{1}{2}h^2A_\psi, & F_{v\psi} &= hA_\psi, \\ F_{pb_a} &= -\frac{1}{2}h^2R, & F_{vb_a} &= -hR, & F_{\psi b_g} &= -h. \end{aligned} \quad (9.4)$$

剩余对角块为单位阵。下列真实源码同时构造一步状态、 F 和噪声输入雅可比 G ：

```

20 result.state = state;
21 result.state.head<2>() += dt * state.segment<2>(2) + .5 * dt * dt * a;
22 result.state.segment<2>(2) += dt * a;
23 result.state[4] = wrapAngle(state[4] + dt * (imu.yaw_rate - state[7]));
24 result.f.setIdentity();
25 result.f.block<2, 2>(0, 2) = dt * Eigen::Matrix2d::Identity();
26 result.f.block<2, 1>(0, 4) = .5 * dt * dt * da;
27 result.f.block<2, 1>(2, 4) = dt * da;
28 result.f.block<2, 2>(0, 5) = -.5 * dt * dt * r;
29 result.f.block<2, 2>(2, 5) = -dt * r;
30 result.f(4, 7) = -dt;
31 result.g.setZero();
32 result.g.block<2, 2>(0, 0) = .5 * dt * dt * r;
33 result.g.block<2, 2>(2, 0) = dt * r;
34 result.g(4, 2) = dt;
35 //

```

设本次采样的测量方差为 $\Sigma_u = \text{diag}(\sigma_{ax}^2, \sigma_{ay}^2, \sigma_g^2)$ ，则

$$P_{k+1}^- = F_k P_k^+ F_k^\top + G_k \Sigma_u G_k^\top + Q_b. \quad (9.5)$$

第六章配置的是**每采样**标准差，所以例如速度方差增量为 $h^2\sigma_a^2$ ，位置与速度的互协方差增量为 $\frac{1}{2}h^3\sigma_a^2$ 。不能直接套用连续白噪声密度的 $h\sigma_a^2$ 公式。

偏置固定零时，最后三个状态及其协方差保持零。开启 `estimate_bias` 后，偏置初始标准差默认为 $(.05, .05, .01)$ ；另设随机游走标准差 $\sigma_b = (.0005, .0005, .0001)$ ，它们是每 $\sqrt{s}$ 的量。因此 Q_b 只在偏置对角块添加 $h \text{diag}(\sigma_b^2)$ 。这描述缓慢变化的不确定性，不等于模拟器一定注入了随机游走。

9.4 一次位姿校正为什么也能改变速度

观测向量 $z = (p_x^{\text{obs}}, p_y^{\text{obs}}, \psi^{\text{obs}})$, H 只选择状态中的位置和航向。创新、创新协方差与增益为

$$r = z - Hx^-, \quad S = HP^-H^\top + R_z, \quad K = P^-H^\top S^{-1}. \quad (9.6)$$

r 的航向分量必须用 `wrap` 归一化。例如预测 179° 、观测 -179° ，应校正约 2° ，而非 -358° 。**练习 ch09-1** 补完 `yaw_residual.hpp`；程序检查跨界、多圈及恰好 π 的边界。

代码使用线性方程求解，不显式求逆。先检查 $r^\top S^{-1}r \leq 16.3$ ，不满足则拒绝本次校正。门限只能检查观测与当前模型是否相容，无法判定哪一方一定正确。若模型漏掉了真实偏置，过小的 P 反而会让它拒绝正常观测。

接受时 $x^+ = x^- + Kr$ 。为减少数值误差对协方差的破坏，采用 Joseph 形式：

$$P^+ = (I - KH)P^-(I - KH)^\top + KR_zK^\top. \quad (9.7)$$

```
70  const Eigen::Matrix<double, 8, 3> gain = solve.solve(h * covariance_).transpose();
71  state_ += gain * residual;
72  state_[4] = wrapAngle(state_[4]);
73  const Matrix8d remainder = Matrix8d::Identity() - gain * h;
74  covariance_ = remainder * covariance_ * remainder.transpose() + gain * noise * gain.transpose();
75  //
```

K 的速度、偏置行通常并不为零，因为此前预测已经建立了互协方差。这就是只有位置/航向观测仍能间接校正其他状态的原因。没有运动变化或长期缺少外部观测时，这些量也可能很难区分；算法不能凭空产生可观测性。

教学模型的边界 激光局部地图和当前估计共享历史，观测不是严格独立的。固定 R_z 也只是可解释的测量尺度。这里的 EKF 不宣称严格统计一致，更不能用很小的协方差数字证明定位准确。

9.5 偏置与一秒缺测实验

合成轨迹为 $p = (\sin(0.6t), 0.5[1 - \cos(0.6t)])$ m、 $\psi = 0.6t$ rad。使用 200 Hz IMU、10 Hz 位姿观测、20 s 时长，4 到 5 s 不提供位姿校正。同一组噪声输入同时送给三个滤波器；`seed=9090`。加计偏置为 $(.08, -.04)$ m/s²，陀螺偏置为 $.01$ rad/s，滤波器没有收到这些答案。初始速度先验为零、标准差 1 m/s；实际初速不为零，因此纯 IMU 还包含初速不可知的误差。

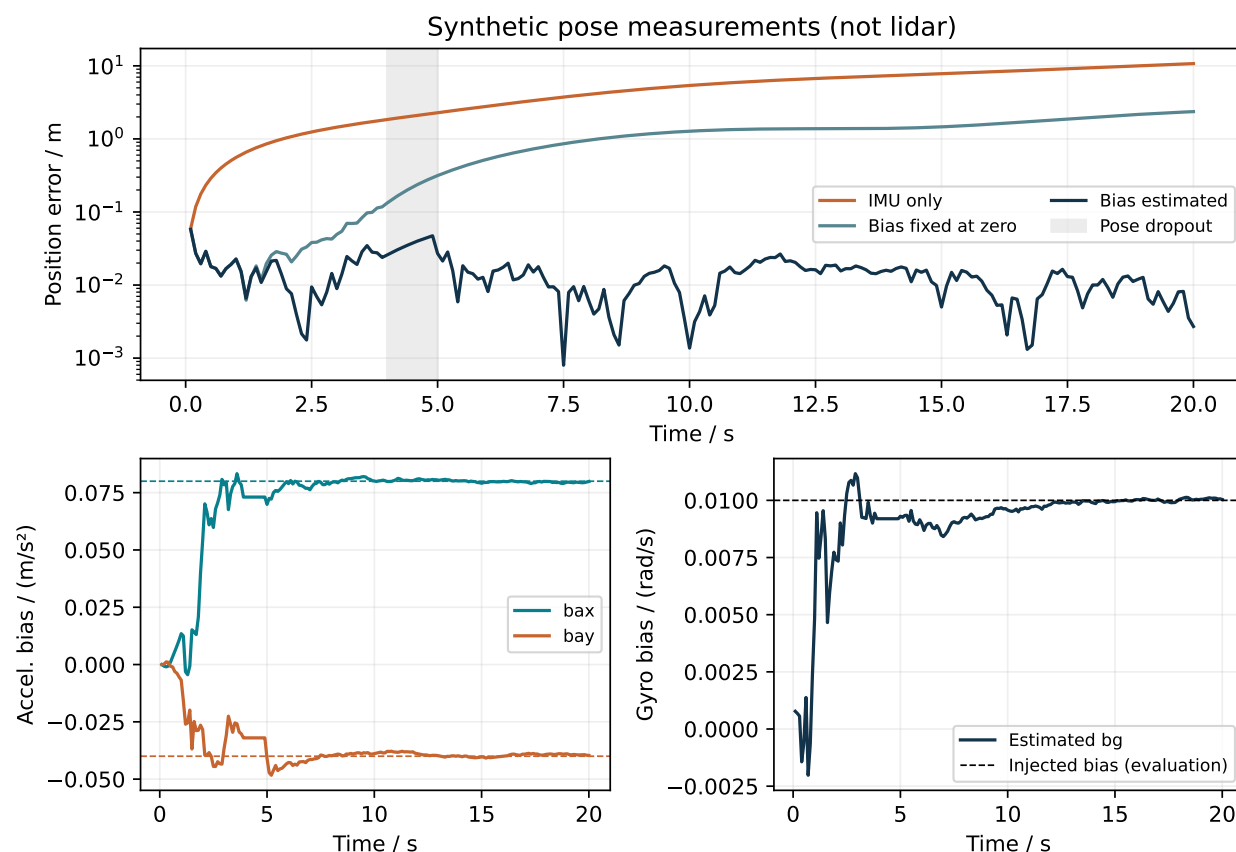


图 9.1: 合成位姿传感器实验。上图纵轴为对数，灰带表示停止位姿校正；下图虚线只用于评估注入的偏置。

模式	全程 RMSE/m	缺测段 RMSE/m	接受	拒绝
仅 IMU	6.06107	2.06224	0	0
偏置固定零	1.25767	0.225992	36	154
估计偏置	0.0183592	0.0381322	189	1

表中误差统计全部 4000 个 IMU 时刻，接受/拒绝计数只针对低频位姿校正。偏置固定零时，模型误差积累并触发大量拒绝；启用偏置估计后，本例全程位置 RMSE 约 0.0184 m。这说明模型与置信度需要匹配，不意味着启用偏置状态在所有场景都更好。

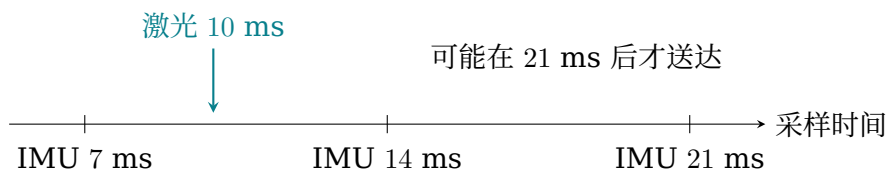
练习与验收：手推 $F_{v\psi}$ ；完成角度残差练习；把注入偏置改为零后复跑，再把观测标准差减半，比较误差及拒绝数。测试还核对常加速度解析解、 F/G 中心差分、互协方差、离群点拒绝以及长时间更新后的协方差对称性与半正定性。下面接入真实扫描匹配输出，单独验证在线时间处理。

9.6 校正发生在采样时刻，不是收到消息时

先运行在线版本：

```
1 ros2 launch motion2d_bringup ch09.launch.py rviz:=false
2 # Another terminal with the same ROS environment, from repository root:
3 /usr/bin/python3 scripts/check_ch09.py
```

默认激光 10 Hz、IMU 200 Hz。配准需要时间，激光位姿送达时，滤波器可能已经处理了更新的 IMU。直接拿旧观测校正当前状态，会把消息延迟误当成运动误差。



例如激光时刻为 10 ms，两侧 IMU 为 7 和 14 ms。先用 7 ms 的样本推进到 10 ms，完成位姿校正，再用同一左侧样本推进到 14 ms，随后使用 14 ms 样本继续。所有区间长度先在整数纳秒内相减，最后乘 10^{-9} 换算秒。

```
41 void FusionTimeline::advance(FilterSnapshot & state, std::int64_t target) const
42 {
43     for (std::size_t i = 0; i + 1 < samples_.size() && state.stamp < target; ++i) {
44         if (samples_[i + 1].stamp <= state.stamp) {continue;}
45         const auto end = std::min(target, samples_[i + 1].stamp);
46         const double dt = (end - state.stamp) * 1e-9;
47         state.filter.predict(samples_[i].sample, dt);
48         state.stamp = end;
49     }
50 }
51 //
```

FusionTimeline 保存一个校正锚点及有限 IMU 队列。激光观测早于最近 IMU 时才可处理；晚于它则由 ROS 节点排队等待。延迟观测校正后，重放到最新 IMU 时刻。默认历史窗口 2 s，超出窗口的旧观测报告 `laser_too_old`，不伪造为“当前测量”。校正必须按激光采样顺序处理，重复帧不重复吸收信息。

近似边界：一步内固定航向；被激光时刻切开的同一 IMU 样本在协方差传播中近似独立。缩小步长能改善均值积分，但不代表噪声模型严格一致。

练习 ch09-2：补完 `held_duration.hpp`，求预测区间与一个 IMU 保持区间的交集。

9.7 高频预测与扫描位姿各走一个出口

消息	时刻与来源	使用者
/odometry/lidar	扫描匹配, 激光采样时刻	融合器
/odometry/estimated	IMU 预测, 最新 IMU 时刻	estimated 适配器
/odometry/scan	激光时刻的校正位姿	观测建图
/odometry	统一选择后的高频估计	跟踪控制与显示

为什么建图不直接订阅高频预测? 同一激光时间的预测可能先到, 而配准校正稍后才到。若把先到预测立即画进地图, 该帧就失去了后来的校正。第九章 launch 将 mapping 的位姿输入重映射到专用的扫描位姿流; 配准或 EKF 拒绝的帧不入图, 后续成功帧仍可继续。

高频输出在每个 IMU 时刻至多发一次; 晚到校正对外体现于下一次 IMU 输出, 不会为同一采样时间反复发布 TF。这里的“连续”表示持续高频传播, 不保证校正时位姿完全无跳变。当前 map 与 odom 重合; 下一章让回环更新 map 到 odom, 而局部 odom 保持自己的运动历史。

失败有明确含义 超过 0.5 s 没有接受激光, 报告 laser_stale, 仍可短时传播 IMU。相邻 IMU 间隔超过 0.1 s, 则报告 imu_gap 并失去初始化, 等待新的激光起点。消息缺失时不能把上一位姿换一个新时间戳冒充已预测; 未来控制器也不能无限使用已失效定位。

第一帧激光定义 odom 原点。初速先验仍为零、标准差 1 m/s, 后续校正逐渐估计它; 不能从首帧 IMU 倒推出真实初始速度。reset 清空偏置、协方差、锚点和历史; 两个输入分别收到新原点时, 保留先到的新试验样本。操作时先暂停, 等待旧 DDS 队列排空。

常见错误: 提前使用右侧 IMU; 只回滚位置而没回滚速度/协方差; 用最新位姿匹配旧扫描; 把激光回环校正直接覆盖底层 odom; 将过小协方差理解成高精度保证。

ROS 验收: 在实际主机管线里检查 3 s 的 601 个 IMU 时刻, 得到 30 个校正扫描/地图配对, 首帧对齐后最大位置误差约 0.0400 m, 主要来自初速尚未估计的初始阶段。输入订阅检查确认融合器和激光里程计没有真值、几何世界或 TF 输入; 暂停不生成新样本, reset 能重播扫描校正值。另一个独立节点实验检查非对齐的 7/10/14/17/21 ms 时刻、两种 reset 顺序、扫描过期和 IMU 中断。

移除 rviz:=false 可显示估计轨迹、扫描点云与观测栅格。本次使用真实 ROS 无界面验收; 由于前述主机 OpenGL 截图及 reset 缓存问题, 没有把 RViz 画面列为已通过。

9.8 真实扫描：转得更快、少扫半秒

```
1 ros2 run motion2d lidar_imu_demo tmp/ch09_fusion
2 /usr/bin/python3 scripts/plot_ch09_fusion.py
```

固定 world seed=42, 距离噪声 seed=4242, IMU seed=6060。360 束、距离噪声 0.01 m、激光 10 Hz、IMU 200 Hz, 圆轨迹半径 1 m, 持续 10 s; 3-3.5 s 不提供扫描, 每次激光结果有 30 ms 交付延迟。使用第六章默认白噪声, 无额外固定偏置。两个转速分别为 0.4 和 1.2 rad/s; 真值只用来生成传感器与独立评估。

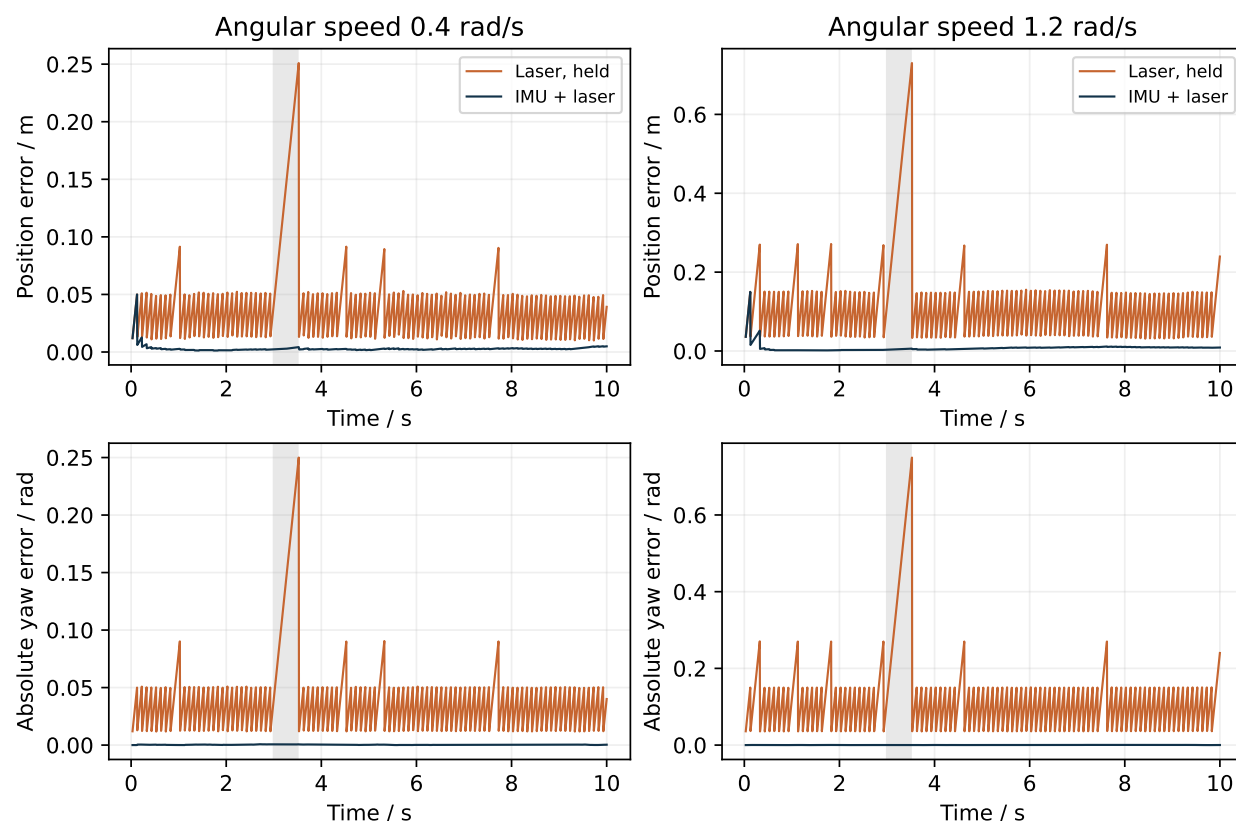


图 9.2: 实际射线与原始 IMU 回放, 灰色区间缺扫。误差在全部可用 IMU 时刻统计, 激光基线保持最新收到的位姿。

$\omega/\text{rad/s}$	模式	位置 RMSE/m	航向 RMSE/rad	缺扫 RMSE/m
0.4	保持激光	0.0505	0.0503	0.1518
0.4	融合	0.0044	0.0003	0.003
1.2	保持激光	0.1522	0.1536	0.4439
1.2	融合	0.0131	0.0003	0.0045

两个转速下激光分别拒绝 4、7 帧, EKF 均无额外拒绝。融合补偿了采样间运动和交付延迟; 这不是声称 ICP 单帧精度被提高, 也不是与带预测的其他激光算法的普遍比较。

本章练习与验收: 完成两道代码练习; 改变转速、延迟或缺扫时长, 连同拒绝数报告误差。检查即时和延迟校正重放后的状态/协方差一致, 历史内存有界, 拒绝观测不修改当前估计。本章使用 snapshot 扫描; 逐束扫描畸变与去畸变保留为可选进阶。

第十章 回到起点之后：关键帧与位姿图

目标 理解相对约束如何共同调整轨迹，以及为什么回环不只是“画一条线”。

先修 第 02、08、09 章；先隔离优化内核，再接地点识别和传感器。

10.1 从一条链变成一张图

运行一个只含相对位姿约束的实验：

```
1 ros2 run motion2d pose_graph_demo tmp/ch10_graph
2 /usr/bin/python3 scripts/plot_ch10_graph.py
```

81 个节点沿 4×3 m 矩形行走；每条顺序边带小误差。不加回环时，把边依次相乘就得到一条逐渐漂移的链。最后加一条“末帧与首帧重合”的测量，优化器让所有节点共同分担矛盾。这只是合成位姿图实验，**尚未验证地点识别或完整 SLAM**。

每个节点是关键帧在 **map** 中的位姿 T_i ；一条边的测量 Z_{ij} 把帧 j 的坐标变到帧 i 。理想情况下 $Z_{ij} = T_i^{-1}T_j$ 。定义

$$E_{ij} = Z_{ij}^{-1}T_i^{-1}T_j, \quad e_{ij} = \begin{bmatrix} p(E_{ij}) \\ \text{wrap}(\psi_j - \psi_i - \psi_{ij}) \end{bmatrix}. \quad (10.1)$$

这里取误差变换的平移和角度，**不是 SE(2) 对数映射**。平移单位 m，角度 rad；不能直接让这三个数不分尺度地竞争。用测量标准差构造

$$D_{ij} = \text{diag}(1/\sigma_x, 1/\sigma_y, 1/\sigma_\psi), \quad \tilde{e}_{ij} = D_{ij}e_{ij}. \quad (10.2)$$

顺序边使用 $\frac{1}{2}\|\tilde{e}\|^2$ ；回环边对范数使用第八章的 Huber 损失。默认阈值为 3，单位是已经白化的残差尺度。

10.2 为什么必须固定第一帧

若对所有位姿左乘同一个固定变换 A ，则

$$(AT_i)^{-1}(AT_j) = T_i^{-1}T_j. \quad (10.3)$$

因此相对约束看不到整张图的全局平移与旋转。固定第一帧的 x, y, ψ ，相当于给地图选坐标原点，不是使用机器人真值。程序直接不把首帧写入优化变量，不靠一个随意放大的先验权重近似“固定”。图若不连通，其他分量仍有自由度，返回 **disconnected**。

10.3 两端位姿各贡献一个雅可比

令 $q = R_i^\top(p_j - p_i)$ 、 $R_z = R(\psi_{ij})$ 、 $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ 。对全局位置与航向做加性扰动，则

$$A_{ij} = \frac{\partial e}{\partial(p_i, \psi_i)} = \begin{bmatrix} -R_z^\top R_i^\top & -R_z^\top Jq \\ 0 & 0 & -1 \end{bmatrix}, \quad B_{ij} = \frac{\partial e}{\partial(p_j, \psi_j)} = \begin{bmatrix} R_z^\top R_i^\top & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (10.4)$$

航向跨越 $\pm\pi$ 时残差必须归一化；差分测试避开恰好不可微的分支切口。

```

10 GraphLinearization linearizeGraphEdge(const Pose2D & from, const Pose2D & to,
11   const Pose2D & measured)
12 {
13   const Eigen::Matrix2d ri = rotation(from.yaw).transpose();
14   const Eigen::Matrix2d rz = rotation(measured.yaw).transpose();
15   const Eigen::Vector2d q = ri * (to.position - from.position);
16   GraphLinearization result;
17   result.residual.head<2>() = rz * (q - measured.position);
18   result.residual.z() = wrapAngle(to.yaw - from.yaw - measured.yaw);
19   result.from.setZero(); result.to.setZero();
20   result.from.topLeftCorner<2, 2>() = -rz * ri;
21   result.from.topRightCorner<2, 1>() = rz * Eigen::Vector2d(q.y(), -q.x());
22   result.from(2, 2) = -1;
23   result.to.topLeftCorner<2, 2>() = rz * ri;
24   result.to(2, 2) = 1;
25   return result;
26 }
27 //

```

把 DA 、 DB 放到对应节点的三列中。每条边只连接两个节点，所以只增加四个 3×3 Hessian 块：

$$H_{ab} += wJ_a^\top J_b, \quad g_a += wJ_a^\top \tilde{e}, \quad H\delta = -g. \quad (10.5)$$

普通边 $w = 1$ ；回环边 $w = \min(1, 3/\|\tilde{e}\|)$ 。Eigen 稀疏 LDLT 求解增量，再回溯缩短步长，直到实际代价充分下降。小步长才报告收敛；迭代耗尽、奇异或线搜索失败都有单独状态。不要把失败的候选图替换当前地图。

鲁棒损失无法证明回环是真的 一个一维反例：正常边测量 $x = 1$ ，错误回环测量 $x = 10$ ，两者 $\sigma = 0.1$ 。普通最小二乘给出 $x = 5.5$ ；只对回环用阈值 3 的 Huber，解仍是 $x = 1.3$ 。错误回环的影响变小，却没有消失。必须先做几何验证，再让约束进入图。

10.4 矩形实验与可独立检验的答案

固定 $\text{seed}=1010$, 沿四条边各走 20 步。每步加独立 0.01 m 平移噪声、 0.002 rad 航向噪声, 并注入每步 0.003 rad 航向偏差。顺序边的测量尺度为 $(.03, .03, .02)$; 闭环观测为单位变换, 尺度 $(.02, .02, .01)$ 。参考轨迹仅用于评估, 不送给优化器。

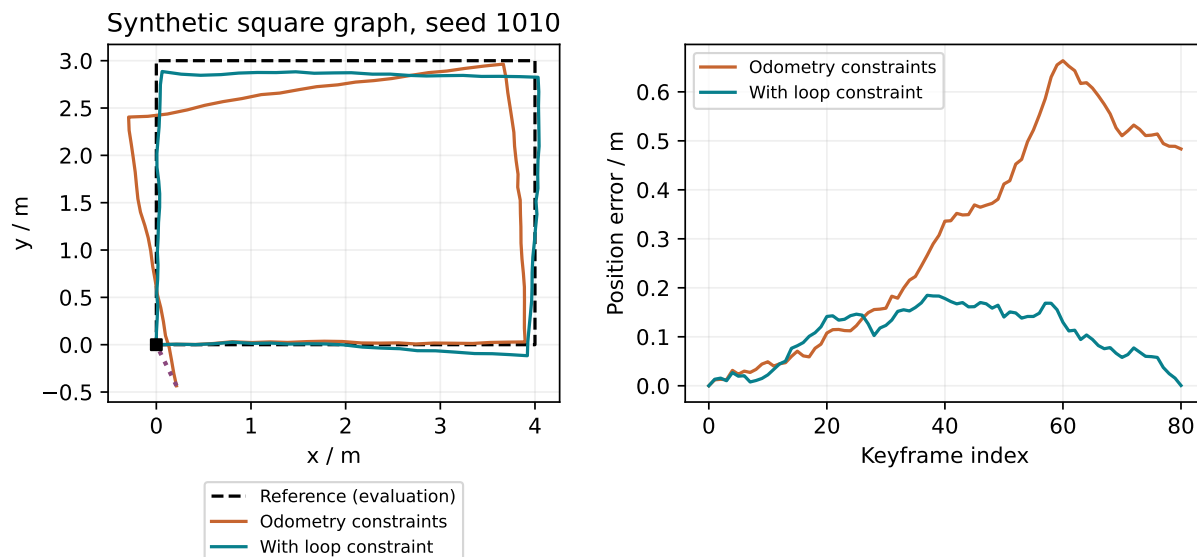


图 10.1: 合成位姿图的优化前后结果。左图等比例, 方块为固定首帧, 紫色虚线指出原始链的闭合缺口。

约束	位置 RMSE/m	闭合误差/m	最终代价	迭代
顺序边	0.3735	0.4835	0	1
加回环	0.1183	0.0006	1.059	5

顺序边代价几乎为零, 因为链本身就是由这些边累乘出来的; 这不表示它没有漂移。加入新约束后边之间有冲突, 最优代价可以大于零, 而参考误差反而下降。不能只比较两个不同图的最终代价来宣称哪个定位更准确。

练习 ch10-1: 补完 `edge_residual.hpp`, 检查旋转坐标和角度跨界。理解题证明左乘不变性; 实验题改变回环尺度, 比较全程 RMSE、闭合误差、代价与求解状态。常见错误包括变换乘法顺序颠倒、只锚定位置却漏掉航向、把错误闭环交给鲁棒核“自动判断”。

本增量验收: 雅可比中心差分、非零首帧固定、已知矩形恢复; 一维两测量的加权解析解; 错误回环影响和不连通图状态。接下来从真实激光/IMU 里程计选择关键帧, 在添加回环前完成几何验证, 并重新构建地图。

10.5 从关键帧中寻找旧地点

完整管线沿用第九章前端；后端只接收扫描及同一采样时刻的校正位姿。平移超过 0.3 m、航向变化超过 0.25 rad，或距上帧超过 2 s，就保存一个关键帧。其中既有原始距离数组，也有机体体系的素点云和局部法向量。原始观测不能被优化后的全局点云替代，否则下一次校正时无法重建。

```
1 ros2 launch motion2d_bringup ch10.launch.py rviz:=false
2 # Another terminal, same ROS environment, repository root:
3 /usr/bin/python3 scripts/check_ch10.py
```

排除最近 20 个关键帧后，只在当前估计位置的 0.8 m 范围内寻找候选，航向差小于 0.6 rad。每次最多检查距离最近的三个候选。这利用局部里程计尚未严重漂移的假设，不能代替全局地点识别。成功回环后至少等待五个新关键帧，避免高频重复加入同一个地点的相关证据。

几何检查	默认条件
双向点到线 ICP	两个方向都收敛，且没有秩退化
有效对应比例	两个方向均至少 55%
残差与初值偏离	RMSE 不超过 0.05 m；校正不超过 0.5 m / 0.25 rad
往返一致性	平移不超过 0.03 m，航向不超过 0.02 rad

若正向变换为 T_{ij} ，反向为 T_{ji} ，则一致时 $T_{ij}T_{ji} \approx I$ 。实际检查代码如下；任何一项失败都不把回环加入图。

```
29 const auto backward = matchClouds(reference.points, current, inverse(result.relative), config.match);
30 if (!backward.accepted() ||
31     static_cast<double>(backward.pairs) / reference.points.size() < config.min_overlap) {
32     result.status = "reverse_rejected"; return result;
33 }
34 const auto cycle = compose(result.relative, backward.pose);
35 if (cycle.position.norm() > config.cycle_translation || std::abs(cycle.yaw) > config.cycle_yaw) {
36     result.status = "inconsistent_cycle"; return result;
37 }
38 result.accepted = true; result.status = "verified";
39 //
```

不能省略的限制：平行长墙可能退化，重复房间可能给出自洽但错误的双向匹配。这些门限减少误接，不能证明地点身份。相同场景下发现了回环，也不能推导出一般环境的成功率。图优化失败时撤掉新回环边；已经成立的局部里程计和顺序边保留。

10.6 回环更新地图坐标，不改写局部里程计

同一机器人有局部位姿 T_{OB} 和图优化后的全局位姿 T_{MB} ，满足

$$T_{MB} = T_{MO}T_{OB}, \quad T_{MO} = T_{MB}T_{OB}^{-1}. \quad (10.6)$$

用最新关键帧的两种位姿计算 map 到 odom 的校正；所有原始 odom 位姿保持不变。**练习 ch10-2** 完成 `map_alignment.hpp`，先手算带 90° 旋转的例子。仅把两个位置相减会漏掉旋转，无法得到正确变换。

```
77   for (std::size_t k = 0; k < frames_.size(); ++k) {frames_[k].map_pose = result.poses[k];}
78   alignment_ = compose(frames_.back().map_pose, inverse(frames_.back().odom_pose));
79   ++loops_; last_loop_ = current;
80   //
```

一条 TF 边只有一个发布者：SLAM 发布 map 到 odom；estimated 适配器发布 odom 到 base_link；模拟器只保留圆心处传感器的静态外参。回环可以让机器人在 map 中跳到更一致的位置，但不会把这个全局位移直接写入局部运动链。第十九章会处理全局目标与局部控制参考的衔接。

输出	含义
/map/cloud 与 /map	按最新图位姿重建的点云与观测栅格
/path/graph_before	固定首帧基准中的原始里程计关键帧
/path/graph_after	同一基准中的优化关键帧
/slam/edges	顺序边与紫色回环边
/slam/status	更新/拒绝状态及关键帧、回环计数

前后路径都用首帧固定的 map 基准比较，不再用当前 map 到 odom 把原始基线变换第二次。高频局部轨迹仍由第九章输出；图更新频率是关键帧频率。

为什么要重建：设旧图在 $x = 3 \text{ m}$ 有一面墙，优化后这次观测应落在 $x = 4 \text{ m}$ 。直接追加新点会留下两面墙。重建时新建空栅格，再遍历所有保留的原始扫描，按照各自优化后的采样位姿重新做射线清空与端点占据；体素点云也重新生成。本章只累积关键帧，未知区域仍是未知，世界真值不参与更新。

当前最多保留 200 个关键帧，达到容量报告 `capacity_reached`；没有悄悄丢弃锚点或改变图语义。全量重建便于初学者理解，却会随着帧数增加变慢。更大场景应学习子图与增量更新。

10.7 实际激光与 IMU 的矩形回放

```
1 ros2 run motion2d slam_demo tmp/ch10_slam
2 /usr/bin/python3 scripts/plot_ch10_slam.py
```

使用 14×12 m 房间，内部有一个圆与一个三角形。半径 0.2 m 的全向圆盘沿 6×4 m 矩形行走，每边 6 s，五次时间规律让它在拐角静止；机体 yaw 保持零。四条边均通过整段扫描检查。360 束、距离噪声 0.03 m、激光 10 Hz、IMU 200 Hz，噪声 seed=4242/6060。

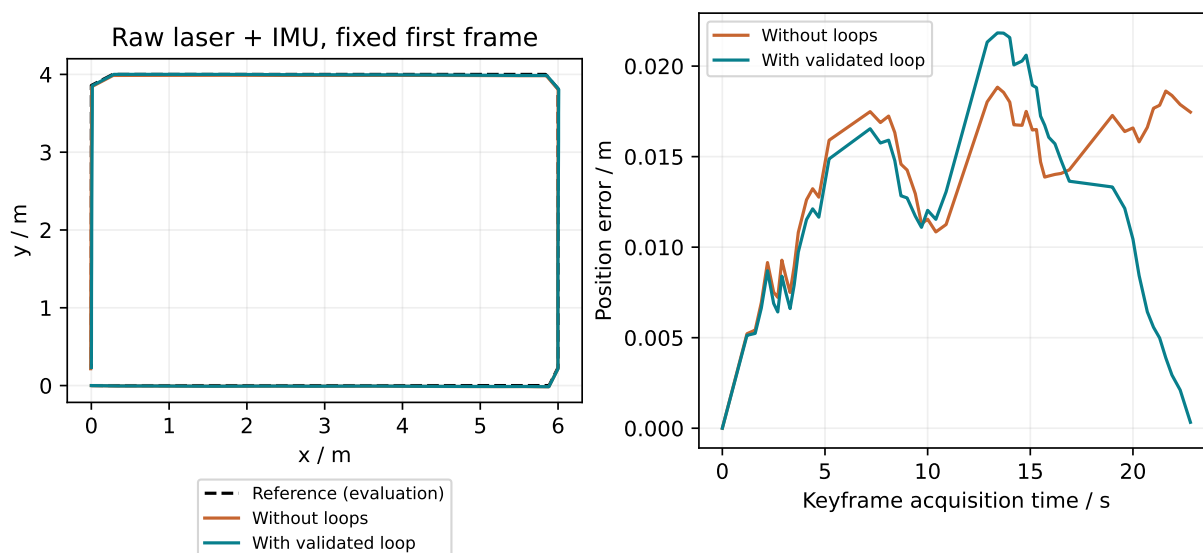


图 10.2: 同一真实传感器前端输出，分别送给回环关闭和开启的图。误差只统计关键帧；参考轨迹仅用于评估。

模式	关键帧	回环	位置 RMSE/m	末关键帧误差/m
关闭回环	53	0	0.0144	0.0175
开启回环	53	1	0.0132	0.0003

本例在 22.3 s 检查一个候选并接受回环；最后关键帧时间为 22.8 s。表中的末帧误差不是 24 s 终点误差，也不是首尾闭合距离。激光拒绝 27 帧，EKF 无额外拒绝；失败扫描没有进入图。

回环显著改善了靠近返回地点的关键帧，但部分中间点误差反而略增。全程位置 RMSE 只从约 1.44 cm 降至 1.32 cm；不能只用末帧约 1.75 cm 到 0.033 cm 的改善宣传全程效果。所有 CSV、原始地图 PGM 与点云 CSV 均可由上述命令重新生成。

10.8 观察重建地图，并检查失败分支

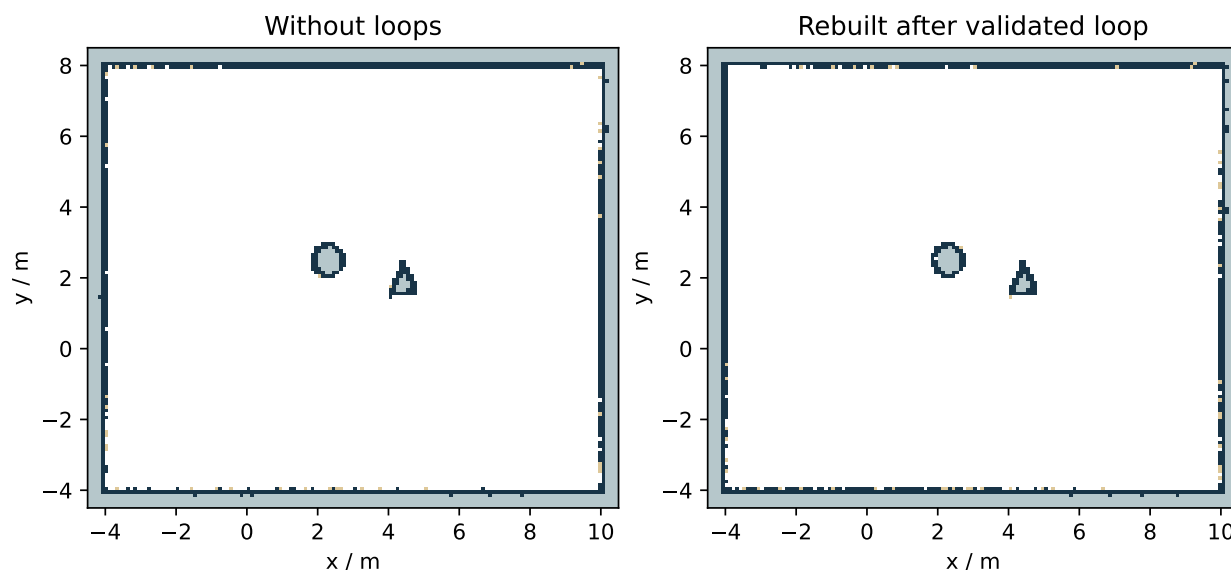


图 10.3: 两图均由关键帧扫描构建，分辨率 0.1 m。白色自由，深色占据，灰色未知，浅褐色为已观测但不确定；纵横尺度相同。

本例原始误差只有厘米量级，小于地图分辨率，因此两张全景图很接近。不能仅凭图片宣称全图精度大幅提高。地图重建的必要性由独立测试验证：将同一观测的墙面移动一个格子后，旧墙位置不再保留占据，新位置正确出现。圆与三角形内部没有被光线穿透，仍然保留未知区域。

启动默认 RViz 配置时，应看到优化关键帧、紫色回环边、重建点云和占据图；原始世界与真值机器人默认隐藏。可修改配置中的 `slam.enable_loops` 为 `false` 对照。本机本次实际 ROS 无界面运行得到 24 个关键帧和一次回环，地图/云/路径时间戳一致。map 到 odom 与 odom 到机体各只有一个发布者；订阅检查没有真值或 TF 输入。局部里程计最大单步位置变化约 0.0403 m，图优化没有覆盖它；reset 后关键帧回到一个、回环边清空。GUI 验收仍受前述主机 OpenGL/时间回拨问题限制，没有用离线图冒充 RViz 截图。

1. **理解：**为什么只有修改 TF、不重建历史点云，会得到坐标正确但几何仍旧扭曲的地图？
2. **代码：**完成两道练习，检查边残差的变换顺序和 map 到 odom 的计算。
3. **实验：**改变候选距离、关键帧间隔或激光噪声，记录候选数、接受数、拒绝帧与误差。
4. **失败：**用单面墙验证退化，用无重叠云验证拒绝；达到图容量时应看到明确状态。

本章边界：已具备静态小场景的激光/IMU 局部定位、几何回环、图优化和观测地图重建。没有实现失锁后的全局重定位、动态物体过滤或长期地图管理。接下来先用真值定位路线学习配置空间与 A*，使规划问题可以独立验证，最终再组合成完整导航。

第十一章 给圆盘找一条路：配置空间与 A*

目标 将圆盘规划化成中心点搜索，理解栅格、未知区域、路径与轨迹的区别。

先修 第 03、07 章。先在给定栅格上检查算法，再接观测地图。

11.1 先运行一张带窄门的地图

```
1 ros2 run motion2d astar_demo tmp/ch11_astar
2 /usr/bin/python3 scripts/plot_ch11_astar.py
```

这次输入是手工给定的 41×31 栅格，分辨率 0.1 m。中间隔墙有宽 0.7 m 的门；起点 (.75, .75) m，终点 (3.25, 2.25) m。改变半径，观察门从可通行变成不可达。该小实验隔离规划算法，不冒充未知环境建图。

11.2 圆盘变成一个中心点

设原始阻塞区域为 $\mathcal{O}$ ，圆盘物理半径为 r ，额外安全裕量为 m 。中心点必须避开膨胀集合

$$\mathcal{O}_{\text{conf}} = \mathcal{O} \oplus B(0, r + m). \quad (11.1)$$

这里 $\oplus$ 表示将集合中的每个点向外扩张一个半径 $r + m$ 的圆。半径与裕量只加一次；后面使用已经膨胀的地图时不能再重复扣除机器人半径。

原始占据栅格表示方格的面积，不是只有格子中心的一个点。本章还要求输出自由格的整个闭区域都安全，方便后续构造走廊。两个边长 ρ 的格子，相对索引差为 $(\Delta i, \Delta j)$ ，其最小距离为

$$d_x = \rho \max(0, |\Delta i| - 1), \quad d_y = \rho \max(0, |\Delta j| - 1), \quad d = \sqrt{d_x^2 + d_y^2}. \quad (11.2)$$

若 $d \leq r + m$ ，候选格子被阻塞。相邻方格共享边或角时距离为零；接触也视为碰撞。

```
47 for (int y = -ny; y <= ny; ++y) {
48     for (int x = -nx; x <= nx; ++x) {
49         const double dx = std::max(0, std::abs(x)-1)*resolution;
50         const double dy = std::max(0, std::abs(y)-1)*resolution;
51         if (std::hypot(dx, dy) <= r + 1e-12) {offsets.emplace_back(x, y);}
52     }
53 }
54 //
```

这种整格安全证书比只检查中心更保守。即使 $r = 0$ ，与障碍接触的相邻格仍会被排除。连续几何可以通过的窄门，粗网格可能判不可达；缩小 ρ 能减少这类保守损失。地图外部也视为阻塞，边界附近格子同样要满足距离要求。

11.3 A* 的三个数与两种邻接

$g(n)$ 是从起点到节点 n 已知的最小代价, $h(n)$ 是到目标的下界, 优先展开

$$f(n) = g(n) + h(n). \quad (11.3)$$

四邻接每步长 ρ , 使用 Manhattan 距离; 八邻接另有长 $\sqrt{2}\rho$ 的对角步, 使用 octile 距离。设到目标的索引差绝对值为 d_x, d_y , 则

$$h_4 = \rho(d_x + d_y), \quad (11.4)$$

$$h_8 = \rho[\max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y)]. \quad (11.5)$$

无障碍时这就是最短距离; 障碍只会让路更长, 因此不会高估。它们还满足一致性, 节点确定后不必反复重新打开。

```

47   for (int dy = -1; dy <= 1; ++dy) {
48       for (int dx = -1; dx <= 1; ++dx) {
49           if ((dx == 0 && dy == 0) || (!diagonal && dx != 0 && dy != 0)) {continue;}
50           const int x = current%width, y = current/width;
51           if (!grid.freeCell(x+dx, y+dy)) {continue;}
52           if (dx != 0 && dy != 0 && (!grid.freeCell(x+dx, y) || !grid.freeCell(x, y+dy))) {continue;}
53           const int next = (y+dy)*width+x+dx;
54           const double candidate = g + grid.geometry.resolution * std::hypot(dx, dy);
55           if (candidate < distance[next]) {
56               distance[next] = candidate; parent[next] = current;
57               open.emplace(candidate+heuristic(next), candidate, next);
58           }
59       }
60   }
61   //

```

关键是中间的对角检查: 终点自由不代表连线安全。例如右邻格被堵、上邻格自由, 即使右上格自由, 也不能沿对角线擦过右邻障碍的角。**练习 ch11-1** 补完 `diagonal_step.hpp`, 分别检查两侧被挡的情况。

程序保留每个节点的父节点, 从目标反向回溯, 再反转得到格子序列。输出折线包含精确请求的起终点; 它们可以不在格子中心。因为起终格的整个面积已保证安全, 格内接到中心的短线仍然安全。两个点在同一格时直接相连, 不必绕到格子中心。

11.4 未知不是免费的捷径

只有已观测且概率编码不大于 35 的格子作为原始自由格；未知 -1、不确定与占据均默认阻塞。这是本课程的规划策略，不是 ROS 强制规定。显式设置 `unknown_blocked=false` 可以做乐观探索实验，但不能把未观测空间称为已验证安全。

搜索结果必须区分：

状态	含义
found	在当前栅格和规则下找到路径
invalid_start / invalid_goal	起点或终点越界、被堵或接触阻塞边界
unreachable	两点有效，但自由格图不连通

失败时返回空路径和无穷大代价。成功的 `grid_cost` 是中心节点图上的最短距离，不是含精确端点及后续简化的折线总长。两者的定义不能混用。

11.5 删掉折点之前，检查整条线段

贪心简化从当前点开始，寻找能直视的最远后续点，再继续。这是减少折点的实用方法，不保证连续空间的全局最短路线。检查线段 $p(t) = a + t(b - a)$, $t \in [0, 1]$ 与每个候选阻塞方格的闭区间是否相交。对每个轴施加 $l \leq p(t) \leq u$ ，把允许的 t 区间相交；非空就是接触或穿过障碍。

本实现遍历线段包围框中的格子，采用解析区间裁剪。它检查细障碍、擦角和沿边接触，不用固定间距采样“猜测安全”。代价是长斜线可能检查较多格子；之后可优化为 `supercover DDA`，但应保留当前实现作为容易审查的对照。

建图射线遍历不能直接当作碰撞检查 第七章 DDA 在恰好经过角点时同时跨两个轴，跳过只被角点触到的旁边格子。这适合那里的观测更新约定，但会漏掉规划中的角点接触。相同的网格不表示相同的遍历语义。

常见错误：只看对角终点；把未知默认当作自由；只查简化线段上的少量采样点；扩大半径后仍使用旧的膨胀图；将找到的一串点当成满足速度、加速度和控制约束的轨迹。本章输出只有几何折线，时间分配和连续曲线将在第十四章开始。

11.6 半径、邻接与窄门实验

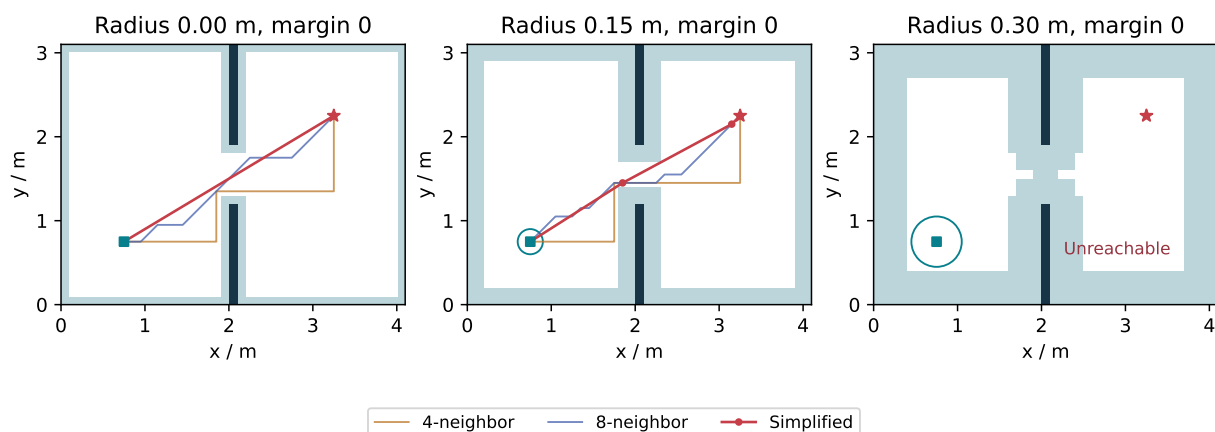


图 11.1: 深色是给定隔墙, 浅蓝是额外阻塞的配置空间格子, 白色可规划。绿方块为起点, 红星为目标; 绿圆保持真实半径与坐标等比例。红线为八邻接结果的简化。

半径/m	邻接	状态	图代价/m	扩展节点	简化点数
0	4	找到	4	317	2
0	8	找到	3.12132	142	2
0.15	4	找到	4	281	4
0.15	8	找到	3.12132	114	4
0.3	4	不可达	—	283	0
0.3	8	不可达	—	283	0

本实验裕量为零, 半径分别为 0、0.15、0.30 m。四邻接成功时图代价为 4 m, 八邻接约为 3.12132 m。两种半径的成功代价相同不表示路径完全相同: 栅格中可以存在多个等长路线。扩展节点数同时受启发式、可用区域和并列排序影响, 不能当作一般性能承诺。

0.7 m 门能在连续几何中容纳直径 0.6 m 的圆盘, 但本例找不到整格都安全的贯通路径。这是明确的离散保守性, 不是把真实机器人偷偷变大; 分辨率、半径与整格安全要求共同决定结果。下一步接入观测地图后, 还要区分几何变窄与尚未看见造成的封闭。

- 理解:** 在小网格上手算 g, h, f , 解释四邻接与八邻接启发式的不同。
- 代码:** 完成邻居检查, 覆盖外边界、两侧被挡和合法对角。
- 实验:** 改变半径、分辨率或未知策略, 记录可达性、代价与简化点数。

验收: 30 组 $\text{seed}=1111$ 随机栅格与独立 Dijkstra 的最短代价一致; 另检查已知四/八邻接距离、精确端点、同格路径、窄门、未知、外界、穿角、线段擦边/擦角与简化后安全。下面接入 ROS 观测地图和点击目标。

11.7 从观测地图接入 ROS 与 RViz

```
1 ros2 launch motion2d_bringup ch11.launch.py
2 ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
3   '{header: {frame_id: map}, pose: {position: {x: -7.5, y: -8.0}, orientation: {w: 1.0}}}'
4 ros2 topic echo /plan/status
```

本章默认使用 **ideal** 模型，机器人静止扫描；IMU 关闭，因为本章定位直接来自显式真值适配器。启动后等几帧观测累积，再在 RViz 的 2D Goal Pose 工具点击白色自由区域。工具发布到 `/goal_pose`，规划器只生成几何路径；执行轨迹留到第十四章。目标朝向在本章忽略，未来轨迹和控制器分别处理 yaw。

方向	接口	含义
输入	<code>/map</code>	仅凭观测累积的占据栅格
输入	<code>/odometry</code> 、TF	当前位姿与 <code>map/odom</code> 关系
输入	<code>/goal_pose</code>	<code>map</code> 中的二维目标位置
输出	<code>/planning/grid</code>	0 自由、100 阻塞的配置空间掩码
输出	<code>/plan/path</code> 、 <code>/plan/status</code>	<code>map</code> 折线与显式规划状态

规划器不订阅障碍物列表、仿真位姿或真值里程计。它使用选源后的 `/odometry`；此启动配置的定位来源仍是**真值**，地图来源则是**激光观测**。后续替换定位来源无需修改 A* 内核。

$${}^{map}T_{base} = {}^{map}T_{odom} {}^{odom}T_{base}. \quad (11.6)$$

TF 按里程计采样时刻查询；未能获取时发布 `waiting_transform` 并等待下一次尝试。不能把 `odom` 坐标直接当作 `map` 坐标。观测地图只接受 `map` 中轴对齐的栅格；原点含旋转时明确拒绝，不能悄悄丢掉旋转。

节点用 5 Hz 定时器合并输入更新：新目标、新地图或起点移动至少 0.05 m 时重新规划。一次搜索始终使用同一张配置空间栅格。地图和状态保留最后一条消息，便于晚加入的 RViz 读取。地图掩码不是占据概率；不要把它再喂给占据建图或重复膨胀。

```
106 const auto result = astar(*grid_, start.position, *goal_, diagonal_);
107 if (!result.success) {publish({}, result.status); return;}
108 const auto path = simplifyPath(*grid_, result.path);
109 publish(path, "found");
110 //
```

失败状态同时发布空 Path，清除旧结果；错误地图、错误目标坐标系与变换等待也如此。检测到地图或里程计时间回退时清除目标，`reset` 后须重新发送目标。已知完整世界中的远目标可能可达，但如果从当前观测地图看尚被未知区域隔开，本章应报告不可达或无效目标。探索和局部目标选择将在第十九章处理。

11.8 复现一条来自真实消息的路径

```
1 # Run ch11.launch.py rviz:=false in a separate terminal first.
2 /usr/bin/python3 scripts/check_ch11.py
3 /usr/bin/python3 scripts/plot_ch11_observed.py
```

检查脚本暂停、reset, 再执行 100 个 5 ms 单步, 固定在 0.5 s。默认 world/noise seed=42/4242, 720 束、10 Hz、测距标准差 0.01 m。地图 220×220 格, 分辨率 0.1 m; 圆盘半径 0.20 m, 额外裕量 0.05 m。脚本保存实际 /map、/planning/grid 与 /plan/path 消息, 再绘制下图。

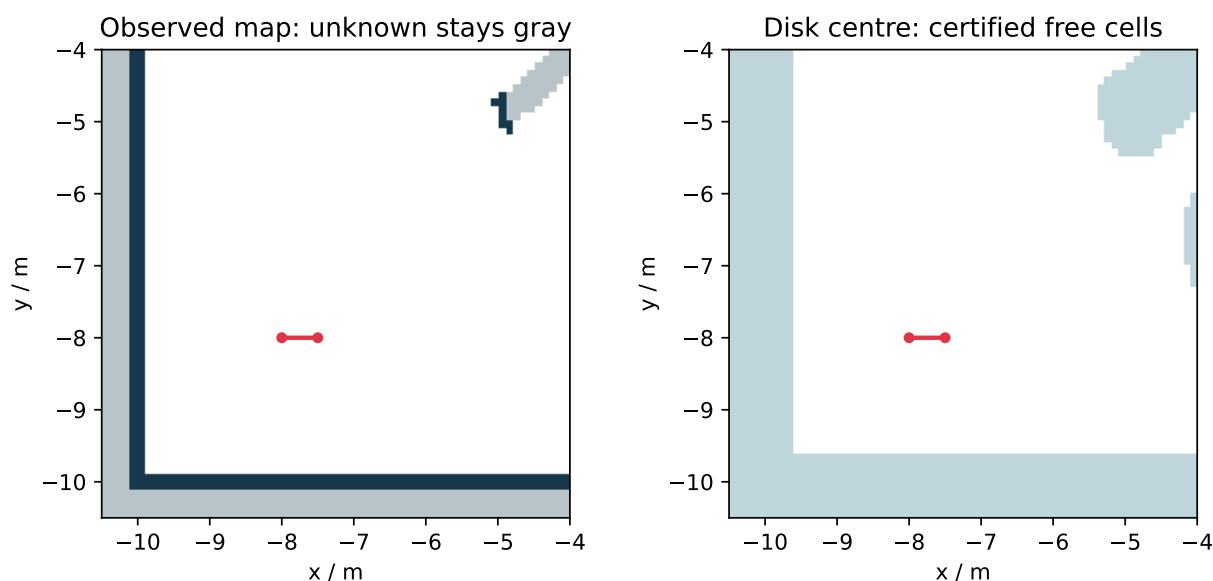


图 11.2: 默认世界左下角的局部视图。左侧灰色为未知, 深色为不满足自由阈值的观测; 右侧浅蓝含额外膨胀。红线从 $(-8, -8)$ 到 $(-7.5, -8)$ m, 两侧都保持米制等比例。

本次得到 6872 个整格安全的自由格, 简化路径含两个端点。机器人仍停在起点; 给 $(8, 8)$ m 的目标返回 `invalid_goal`, 错误 frame 返回 `invalid_goal_frame_or_position`, 两种失败都清空路径。还检查输入隔离、时间重置和唯一 TF 来源; 独立测试用旋转 90° 、平移 $(1, 2)$ m 的变换验证坐标组合。RViz 配置已提供, 本次主机验收为无界面 ROS 运行, 未将 GUI 画面列为已验收。

练习 ch11-2: 实现 `pathLength`, 累计相邻实际坐标之间的欧氏距离。空路径和单点长度为零; 含重复点时不要额外增加距离。用 3-4-5 三角形与多段折线验证, 比较 A* 的 `grid_cost`、真实端点折线长度与简化长度。

排查: 看不到路径时, 先读 `/plan/status`, 再检查目标 frame、观测是否足够、半径及未知策略。在 RViz 勾选 `Configuration grid` 可查看规划用的掩码。不要为了得到路径而把未知策略悄悄改为乐观。

第十二章 离障碍多远：二维距离场

目标 把占据标签变成距离，查询连续位置的值与梯度，明确哪些距离可以用于安全判定。

先修 第 07、11 章；范数、偏导数和双线性插值。

12.1 先看一个圆的距离场

```
1 ros2 run motion2d esdf_demo tmp/ch12_esdf
2 /usr/bin/python3 scripts/plot_ch12_esdf.py
```

实验把半径 1 m 的圆按格子中心采样为占据图，再比较三种分辨率。连续圆的解析有符号距离是 $d_{circle}(p) = \|p\| - 1$ ：圆外正、圆内负、边界零。除圆心外，其梯度为 $p/\|p\|$ ；圆心没有唯一的梯度方向。

12.2 先确定你计算的是哪一种距离

本章输入是未膨胀的观测栅格。沿用上一章标签策略：已观测且编码不超过 35 才自由，未知默认阻塞；外围补一圈阻塞格，表达地图外部不安全。在格子中心 c_i 定义离散场

$$d_i = \begin{cases} \min_{j \in O_c} \|c_i - c_j\|, & i \text{ 自由}, \\ -\min_{j \in F_c} \|c_i - c_j\|, & i \text{ 阻塞}. \end{cases} \quad (12.1)$$

O_c, F_c 是阻塞与自由格中心集合。这是有符号中心距离，不是到完整方格边界的精确距离。例如宽 ρ 的直墙格外，相邻自由中心距墙格中心为 ρ ，但距墙边只有 $\rho/2$ 。整张地图阻塞时没有自由中心，内部值为 $-\infty$ ，查询明确返回不可用。

12.3 欧氏距离变换：先平方，后开方

平方距离可以分离成 x、y 两项。先逐列，再逐行求最小值：

$$D(x, y) = \min_{i,j} [(x-i)^2 + (y-j)^2 + f(i, j)], \quad f = \begin{cases} 0 & \text{种子格}, \\ +\infty & \text{其他格}. \end{cases} \quad (12.2)$$

$d = \rho\sqrt{D}$ 才是米制距离，不能直接把 D 当作 ESDF。以阻塞格为种子得到外部距离，以自由格为种子得到内部距离；随后按标签赋予正负号。二维变换复用同一段一维算法；源码入口见本章运行说明。

12.4 一维最小值为什么可以快速求

给定数组 $f(q)$ ，每个有限元素贡献一条抛物线 $f(q) + (x - q)^2$ 。我们要的是这些曲线中最低的一条。按中心位置递增加入曲线，保存仍有机会成为最小值的中心索引，以及每条曲线从哪里开始胜出。新曲线与旧曲线 v 的交点为

$$s = \frac{f(q) + q^2 - f(v) - v^2}{2(q - v)}. \quad (12.3)$$

若交点早于旧曲线开始胜出的区间，旧曲线永远不再最低，可以删除。每条曲线最多加入和删除一次；得到分界点后，再从左到右读取最小值。这个下包络算法来自 Felzenszwalb 与 Huttenlocher 的 EDT 论文，正文与原始链接见 REFERENCES.md。本课程按公式独立实现，并用暴力最近中心搜索核对结果。

```

22 while (last >= 0) {
23     const int v = sites[last];
24     crossing = ((f[q]+double(q)*q)-(f[v]+double(v)*v))/(2.0*(q-v));
25     if (crossing > begin[last]) {break;}
26     --last;
27 }
28 ++last; sites[last] = q; begin[last] = last == 0 ? -inf : crossing;
29 //

```

12.5 连续位置的值与梯度

先换算到以格子中心为采样点的坐标：

$$\xi = \frac{x - o_x}{\rho} - \frac{1}{2}, \quad \eta = \frac{y - o_y}{\rho} - \frac{1}{2}. \quad (12.4)$$

令 $i = \lfloor \xi \rfloor, j = \lfloor \eta \rfloor$ ，局部比例 $u = \xi - i, v = \eta - j$ 。四个中心值按 00, 10, 01, 11 排列：

$$\tilde{d} = (1 - v)[(1 - u)d_{00} + ud_{10}] + v[(1 - u)d_{01} + ud_{11}]. \quad (12.5)$$

对权重求导，并用 $du/dx = dv/dy = 1/\rho$ 转为米制梯度。

```

12 const double value = (1-v)*((1-u)*d[0]+u*d[1]) + v*((1-u)*d[2]+u*d[3]);
13 const double dx = ((1-v)*(d[1]-d[0])+v*(d[3]-d[2]))/resolution;
14 const double dy = ((1-u)*(d[2]-d[0])+u*(d[3]-d[1]))/resolution;
15 return {value, {dx, dy}};
16 //

```

查询范围为最外围格子中心组成的闭矩形；越界或角点距离无穷时返回空。梯度在插值块边界可能不连续；跨自由/阻塞相邻中心时，值从 $-\rho$ 变到 $+\rho$ ，斜率为 2。所以不要强行把本章梯度解释成处处单位长度的精确 SDF 法向。

12.6 把距离与安全判定分开

插值会产生误差，障碍还有方格面积。仅使用 $\tilde{d} > r + m$ 可能高估可用净空。设当前查询点的四个插值中心为 c_k ，权重为 w_k ，且四个中心均自由。无符号最近阻塞中心距离 d_c 满足

$$|d_c(p) - d_c(c_k)| \leq \|p - c_k\|. \quad (12.6)$$

对四个不等式加权求和，可从插值值减去 $E = \sum_k w_k \|p - c_k\|$ 得到 $d_c(p)$ 的下界。每个障碍方格离中心最远为半对角线 $h = \rho/\sqrt{2}$ ，再由三角不等式得到

$$\underline{d}(p) = \max \left(0, \tilde{d}(p) - \sum_{k=1}^4 w_k \|p - c_k\| - \frac{\rho}{\sqrt{2}} \right). \quad (12.7)$$

这就是 `sample` 返回的 `clearance_lower_bound`，针对**方格并集与地图外部**。四个角中存在阻塞格时返回 0；0 只说明未给出正净空证书，不表示一定碰撞。此下界是本课程的保守选择，可能比精确方格距离小很多。

圆盘半径只处理一次 本章从未膨胀图计算距离，用 $d > r + m$ 验证单个圆心位置。严格大于对应接触也算不安全。若使用第十一章已膨胀的配置空间，则按中心点检查，不再扣一次物理半径。

返回的 `gradient` 是 $\tilde{d}$ 的导数，**不是这个下界的导数**。轨迹优化可以借助连续插值的值和梯度获得改进方向；优化完成后必须单独验证安全。离散采样满足净空不保证采样点之间的曲线安全，第十三、十六章会引入凸走廊与整段包含性检查。

12.7 核对距离、梯度与离散误差

三个层次分别检查：

1. **算法**：50 张 `seed=1212` 小栅格，逐格枚举所有种子，核对平方距离。无种子、单行及稀疏整列也覆盖。
2. **数值**：直墙的符号/米制距离有解析值；对双线性函数和实际距离场做中心有限差分，梯度误差要求小于 10^{-9} ，查询点避开插值块边界。
3. **几何**：1000 个 `seed=1213` 查询点，逐个障碍方格算点到闭方格的精确距离，再取与外部距离的最小值，确认下界从不超过它，容差 10^{-12} m。

这些检查针对栅格所表达的几何，不能消除传感器噪声、漏检和建图错误。若显式允许未知自由，证书只针对这张乐观地图，不能声称验证过真实未知空间。

12.8 分辨率实验与练习

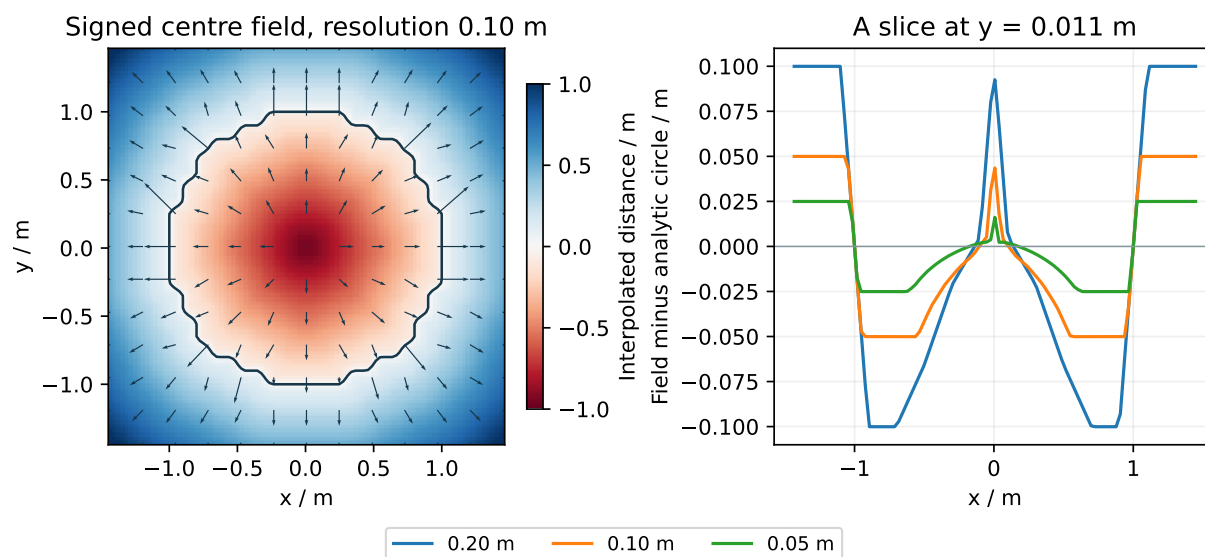


图 12.1: 左: 0.1 m 格的有符号场与插值梯度, 坐标等比例。右: 固定 y 的距离误差切片。不同分辨率都存在栅格化和中心距离误差, 靠近界面时插值也影响结果。

ρ/m	格数	平均误差/m	最大误差/m	中位数/ μs	P95/ μs
0.2	1,600	0.0662	0.1865	39.9	41.1
0.1	6,400	0.0315	0.0895	97.2	156.1
0.05	25,600	0.0132	0.0457	431.1	442.6

给定地图为 $8 \times 8 \text{ m}$; 中心位于半径 1 m 圆内的格子标为占据。三组都使用同一查询晶格, 统计 $0.3 \leq \|p\| \leq 1.6 \text{ m}$ 环带内与解析圆距离的绝对误差。这是**栅格对圆的近似实验**, 不是同一方格几何的精确距离误差; 原始圆与方格并集不完全相同。每个分辨率预热 5 次, 计时 100 次, 含分类、补边界和两个 EDT, 不含 ROS 序列化。本机更细网格平均误差更小, 但构建更慢; 单个点的误差未必单调, 耗时也不是跨机器承诺。

练习 ch12-1: 补全 `bilinearGradient`。给定 $f(x, y) = 2x + 3y + xy$ 在 $0, \rho$ 四角的值, 验证 $\rho = 0.2 \text{ m}$ 、 $u = 0.3, v = 0.7$ 时梯度为 $(2.14, 3.06)$ 。再验证常数场梯度为零。提示、评分和参考解分别位于 `exercises/ch12` 下, 练习失败不影响主演示。

理解与实验: 画解析圆场; 解释圆心处不可微、中心距与边界距的差别。改变分辨率, 报告误差、格数和构建时间, 保留相同查询点与计时范围。常见错误是忘记开方、忘记乘分辨率、把索引坐标当米数, 以及使用已经膨胀的地图再扣半径。下面将距离场接入 ROS 可视化。

12.9 ROS 距离通道与梯度显示

```
1 ros2 launch motion2d_bringup ch12.launch.py
2 ros2 topic echo /esdf/status
3 # In a separate terminal, with the same ROS_DOMAIN_ID:
4 /usr/bin/python3 scripts/check_ch12.py
5 /usr/bin/python3 scripts/plot_ch12_observed.py
```

`esdf_node` 只订阅 `/map`，每张观测地图独立构建一个距离场。本章启动保持上一章的真值定位和观测建图；ESDF 节点本身不读真值、激光、里程计或 TF。输入与 A* 共用同一套自由阈值、未知策略和轴对齐地图验证，避免两者解释不同的地图。

话题	类型	语义
<code>/esdf/cloud</code>	<code>PointCloud2</code>	x、y、z、distance 四个 FLOAT32
<code>/esdf/gradients</code>	<code>MarkerArray</code>	map 中的梯度方向箭头
<code>/esdf/status</code>	<code>String</code>	ready / no_finite_field / invalid_map

点云保持原地图的 `width`、`height`，按 $i = yW + x$ 行主序存储；每点 16 字节，行步长为 $16W$ 。x、y 是格子中心的米制坐标，z 固定为零。**distance** 单独存有符号中心距离，单位 **m**，不是 z 高度，也不是占据概率。全阻塞时保留 $-\infty$ 并设置 `is_dense=false`，同时发布 `no_finite_field` 和空箭头。地图格式错误时清空点云与标记，不保留上一次的有效显示。

RViz 的 Color Transformer 选择 Intensity, Channel Name 选择 distance。预设用 $[-0.5, 2.0]$ m 固定颜色范围，超出范围只在显示颜色上饱和，消息里的数值不变。Size (m) 默认匹配 0.1 m 分辨率；修改地图分辨率后应相应调整显示方块大小。配置空间掩码、观测图与 ESDF 是不同产物，可在 Displays 中切换比较。

梯度默认每隔 10 格显示一个箭头，箭头方向朝距离增加处。显示长度为 $\min(0.4, 0.2\|\nabla d\|)$ m，箭头头部随短箭头缩放；这只是可视化约定，不能当作速度或控制力。

```
84 const Eigen::Vector2d direction = s->gradient.normalized();
85 const Eigen::Vector2d normal(-direction.y(), direction.x());
86 const double length = std::min(.4, .2*s->gradient.norm());
87 const double head = std::min(.06, .3*length);
88 const Eigen::Vector2d end = p+length*direction;
89 add(p); add(end);
90 add(end); add(end-head*direction+.4*head*normal);
91 add(end); add(end-head*direction-.4*head*normal);
92 //
```

输出继承地图的采样时刻和 `map` 坐标系，采用保留最后一条消息的 QoS。重置不需要维护旧距离场：时间回退的新地图到达后直接重建；晚加入的 RViz 仍能读取当前快照。

12.10 读一次真实距离消息

下图来自实际 ROS 的距离点云与箭头消息，固定为 `reset` 后 100 步、0.5 s。图仅截取默认随机世界左下部分；负值同时包含占据、不确定和按策略阻塞的未知区域。颜色不能代替原始占据图中的观测来源信息。

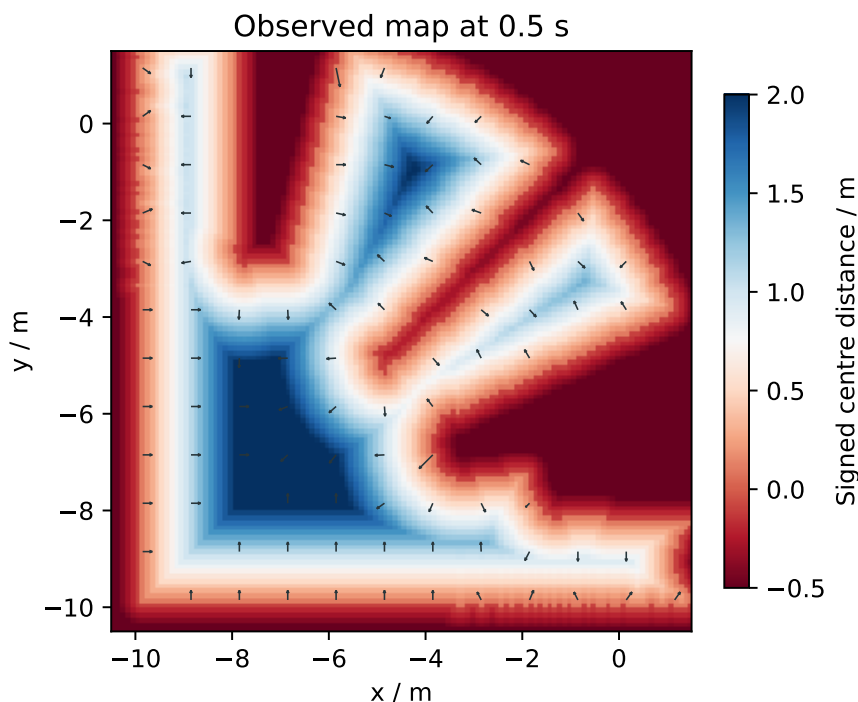


图 12.2: 实际观测距离场与消息中的梯度箭头。坐标为米且等比例，颜色在固定范围内显示。这张图由消息数据绘制，不作为 RViz GUI 截图。

220 × 220 点的独立抽查使用 `seed=1214` 随机选择 100 格，逐个枚举最近阻塞或自由中心，包含外部补边界。消息 `FLOAT32` 与独立计算的最大差为 4.3×10^{-7} m，验收阈值为 10^{-5} m。93 个箭头的长度、方向与字段梯度一致；还验证地图时间戳、输入隔离和 `reset` 后数据重放。

额外独立测试给 `esdf_node` 发送全未知图、全自由图、含旋转的错误图。分别检查 $-\infty$ 、外部距离和清除旧显示，不能用空图的零距离冒充有效场。实际主机已完成无界面 ROS 验收；RViz 配置已提供，GUI 画面未验收。

消息练习：

1. 从 `/esdf/cloud` 读取一格的四个字段，手算中心坐标和行字节偏移。
2. 把 `esdf.gradient_stride` 从 10 改为 5，比较箭头密度，说明为何数值距离不变。
3. 对全自由小地图，解释为什么越靠近地图边界，正距离越小。

独立检查入口、命令和评分见本章练习说明。